

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI



BÁO CÁO TỔNG KẾT

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN
NĂM HỌC 2025 – 2026

NGHIÊN CỨU VÀ ỨNG DỤNG XỬ LÝ NGÔN NGỮ TỰ NHIÊN TRONG PHÁT TRIỂN HỆ THỐNG AI TẠO LẬP CV THÔNG MINH VÀ TƯ VẤN CÁ NHÂN HÓA LỘ TRÌNH NGHỀ NGHIỆP

Sinh viên thực hiện:

1. Trần Đình Võ	CQ.64.CNTT	Bộ môn Công nghệ thông tin
2. Hà Văn Dũng	CQ.63.CNTT	Bộ môn Công nghệ thông tin
3. Cao Xuân Điệp	CQ.64.CNTT	Bộ môn Công nghệ thông tin
4. Đặng Ngọc Hiếu	CQ.64.CNTT	Bộ môn Công nghệ thông tin
5. Phạm Hải Yến	CQ.64.CNTT	Bộ môn Công nghệ thông tin

Người hướng dẫn: KS. Trần Quốc Khánh

TP. Hồ Chí Minh, tháng 5 năm 2026

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI



BÁO CÁO TỔNG KẾT

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN
NĂM HỌC 2025 – 2026

NGHIÊN CỨU VÀ ỨNG DỤNG XỬ LÝ NGÔN NGỮ TỰ NHIÊN TRONG PHÁT TRIỂN HỆ THỐNG AI TẠO LẬP CV THÔNG MINH VÀ TƯ VẤN CÁ NHÂN HÓA LỘ TRÌNH NGHỀ NGHIỆP

Sinh viên thực hiện:

1. Trần Đình Võ	Nam, Nữ: Nam	Dân tộc: Kinh
Lớp: CQ.64.CNTT	Ngành học: Công nghệ thông tin	Năm thứ: 3/4
2. Hà Văn Dũng	Nam, Nữ: Nam	Dân tộc: Kinh
Lớp: CQ.63.CNTT	Ngành học: Công nghệ thông tin	Năm thứ: 4/4
3. Cao Xuân Điệp	Nam, Nữ: Nam	Dân tộc: Kinh
Lớp: CQ.64.CNTT	Ngành học: Công nghệ thông tin	Năm thứ: 3/4
4. Đặng Ngọc Hiếu	Nam, Nữ: Nam	Dân tộc: Kinh
Lớp: CQ.64.CNTT	Ngành học: Công nghệ thông tin	Năm thứ: 3/4
5. Phạm Hải Yến	Nam, Nữ: Nữ	Dân tộc: Kinh
Lớp: CQ.64.CNTT	Ngành học: Công nghệ thông tin	Năm thứ: 3/4

Người hướng dẫn: KS. Trần Quốc Khánh

MỤC LỤC

MỤC LỤC	i
DANH MỤC HÌNH ẢNH.....	vi
DANH MỤC BẢNG BIỂU.....	vii
DANH MỤC VIẾT TẮT	viii
LỜI MỞ ĐẦU	xi
CHƯƠNG 1: TỔNG QUAN	1
1.1 Tình hình nghiên cứu.....	1
1.1.1 Các nghiên cứu quốc tế	1
1.1.2 Các nghiên cứu trong nước.....	3
1.2 Lý do chọn đề tài	4
1.3 Nhận xét và định vị đề tài.....	5
1.4 Mục tiêu, nội dung, phương pháp nghiên cứu của đề tài	7
1.4.1 Mục tiêu nghiên cứu	7
1.4.2 Nội dung nghiên cứu	7
1.4.3 Phương pháp nghiên cứu	8
1.5 Đối tượng và phạm vi nghiên cứu	10
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	12
2.1 Nền tảng NLP và các mô hình BERT	12
2.1.1 Tổng quan NLP và kiến trúc Transformer.....	12
2.1.2 Mô hình BERT và các biến thể.....	13
2.1.3 Sentence-BERT và biểu diễn ngữ nghĩa.....	14
2.2 Named Entity Recognition và BIO Tagging.....	15
2.2.1 Bài toán NER và biểu diễn nhãn BIO	15

2.2.2 Mô hình BERT cho NER và cách thức tokenization	15
2.2.3 Đánh giá mô hình NER	16
2.3 Skill Ontology và Skill Matching.....	17
2.3.1 Khái niệm Ontology trong lĩnh vực nghề nghiệp.....	17
2.3.2 Các bộ Ontology kỹ năng tiêu biểu	18
2.3.3 Bài toán Skill Matching và các tầng tiếp cận	19
2.3.4 ATS Score - điểm phù hợp CV với JD	20
2.4 Kiến trúc RAG và mô hình ngôn ngữ lớn	20
2.4.1 Mô hình ngôn ngữ lớn và kiến trúc RAG.....	20
2.4.2 Hạ tầng RAG trong hệ thống.....	21
2.4.3 Triển khai và tối ưu trong đề tài	22
2.5 Kiến trúc microservices và sinh dữ liệu synthetic.....	22
2.5.1 Kiến trúc microservices.....	22
2.5.2 Sinh dữ liệu synthetic bằng LLM.....	23
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG.....	25
3.1 Kiến trúc tổng thể hệ thống	25
3.1.1 Tổng quan thiết kế	25
3.1.2 Luồng dữ liệu tổng thể	27
3.2 Dữ liệu và pipeline huấn luyện NER.....	28
3.2.1 Tổng quan bài toán dữ liệu	28
3.2.2 Pipeline sinh dữ liệu CV Synthetic	29
3.2.3 Thống kê bộ dữ liệu.....	30
3.2.4 Pipeline gán nhãn BIO.....	31
3.2.5 Cấu trúc dữ liệu lưu trữ	31
3.3 NER Service - Thiết kế và huấn luyện	32
3.3.1 Tổng quan và kiến trúc NER Service	32

3.3.2 Xử lý suy luận và pipeline dữ liệu.....	34
3.3.3 Cấu hình và huấn luyện mô hình.....	34
3.4 Skill Matching Service - Thiết kế và triển khai.....	35
3.4.1 Tổng quan Skill Service	35
3.4.2 SkillOntology - Nền tảng tri thức kỹ năng	35
3.4.3 SkillMatcher - Logic so khớp 3 tầng.....	36
3.4.4 ATSScoringEngine - Đánh giá CV toàn diện 8 tiêu chí	37
3.4.5 Tích hợp O*NET và Market Analysis	38
3.5 Chatbot Service - RAG Pipeline và tư vấn nghề nghiệp.....	38
3.5.1 Tổng quan Chatbot Service	38
3.5.2 Kiến trúc RAG pipeline chi tiết.....	39
3.5.3 Xây dựng prompt và cơ chế cá nhân hóa	40
3.5.4 Dual LLM Backend: Ollama và Groq.....	40
3.5.5 User Memory Service.....	41
3.6 API Gateway (.NET 9) và Frontend React 18.....	41
3.6.1 Kiến trúc hệ thống và xử lý backend.....	41
3.6.2 Dữ liệu và xác thực.....	44
3.6.3 Frontend (React 18).....	46
3.6.4 Triển khai hệ thống.....	48
CHƯƠNG 4: THỰC NGHIỆM VÀ KẾT QUẢ	49
4.1 Môi trường thực nghiệm.....	49
4.1.1 Cấu hình phần cứng.....	49
4.1.2 Môi trường phần mềm.....	49
4.1.3 Cấu hình mô hình NER	50
4.1.4 Thiết kế thực nghiệm.....	51
4.2 Kết quả NER - Phân tích và đánh giá.....	51

4.2.1 Đánh giá định lượng trên Silver Labels.....	51
4.2.2 Phân tích lỗi Tokenization Mismatch	52
4.2.3 Đánh giá định lượng trên tập thủ công (Manual Annotation)	54
4.2.4 Đánh giá định tính trên 12 CV thực tế	55
4.2.5 Đánh giá bán tự động trên 100 CV Groq (Semi-Automatic Evaluation).....	57
4.2.6 Tổng hợp pipeline đánh giá NER	59
4.2.7 Ví dụ output NER trực quan.....	59
4.3 Kết quả Skill Matching - Đánh giá và phân tích	65
4.3.1 Thiết kế bộ test cases	65
4.3.2 Kết quả tổng hợp	66
4.3.3 Phân tích chi tiết từng test case	67
4.3.4 Phân tích ngưỡng Semantic Similarity	68
4.4 Demo hệ thống End-to-End.....	69
4.4.1 Môi trường demo	69
4.4.2 Luồng 1 - Upload và phân tích CV	70
4.4.3 Luồng 2 - ATS Score và Skill Gap Analysis	71
4.4.4 Luồng 3 - Tư vấn nghề nghiệp qua chatbot RAG	73
4.4.5 Luồng 4 - CV Builder với AI Assistance.....	74
4.4.6 Giao diện người dùng - Tổng quan	76
4.5 Thảo luận và đánh giá tổng thể.....	77
4.5.1 Tổng hợp kết quả thực nghiệm.....	77
4.5.2 Hạn chế và nguyên nhân.....	77
4.5.3 Định hướng cải tiến	78
4.5.4 Tổng hợp kết quả và định hướng.....	79
KẾT LUẬN VÀ KIẾN NGHỊ	82
1. Kết luận.....	82

2. Hạn chế	83
3. Kiến nghị	83
TÀI LIỆU THAM KHẢO	84
Bài báo Khoa học và Hội nghị	84
Tài liệu Kỹ thuật và Báo cáo	86
Văn bản Pháp luật.....	86
Dữ liệu và Bộ dữ liệu	87
Tài liệu Nghề nghiệp và Thị trường	87

DANH MỤC HÌNH ẢNH

Hình 3.1: Kiến trúc microservices hệ thống CV Assistant.....	25
Hình 3.2: Sơ đồ luồng dữ liệu tổng thể	27
Hình 3.3: Pipeline sinh dữ liệu CV	29
Hình 3.4: Kiến trúc nội bộ NER Service.....	33
Hình 3.5: Logic so khớp 3 tầng	36
Hình 3.6: Kiến trúc RAG chi tiết.....	39
Hình 3.7: Xây dựng prompt với cơ chế cá nhân hóa.....	40
Hình 3.8: Nguyên tắc Clean Architecture.....	42
Hình 3.9: Sơ đồ luồng đăng ký và đăng nhập	45
Hình 4.1: Cấu trúc thực nghiệm đề tài	51
Hình 4.2: Pipeline và kết quả đánh giá mô hình NER theo 4 phương pháp	59
Hình 4.3: Kết quả trích xuất thực tế trực tiếp trên CV mẫu 1	60
Hình 4.4: Kết quả trích xuất thực tế trực tiếp trên CV mẫu 1	61
Hình 4.5: Kết quả trích xuất thực tế trực tiếp trên CV mẫu 2	62
Hình 4.6: Kết quả trích xuất thực tế trực tiếp trên CV mẫu 3	63
Hình 4.7: Kết quả trích xuất thực tế trực tiếp trên CV mẫu 3	64
Hình 4.8: Cosine similarity các cặp kỹ năng so với ngưỡng $\tau = 0.65$ (đường xanh)	69
Hình 4.9: Giao diện upload CV (dropzone)	70
Hình 4.10: Nhập JD để phân tích Skill Gap	71
Hình 4.11: Kết quả Skill Matrix matching	71
Hình 4.12: ATS Score breakdown	72
Hình 4.13: Chatbot với Knowledge Graph (1)	73
Hình 4.14: Chatbot với Knowledge Graph (2)	73
Hình 4.15: CV Builder bước 1: thông tin cá nhân (Header)	74
Hình 4.16: CV Builder bước 2: chatbot thu thập thông tin	75
Hình 4.17: CV Builder bước 3: form nhập học vấn	75
Hình 4.18: Tổng quan giao diện hệ thống CV Assistant	76
Hình 4.19: Điểm kết quả và tiềm năng cải tiến của từng module trong hệ thống.....	80

DANH MỤC BẢNG BIỂU

Bảng 1.1: Tổng quan các công trình liên quan	6
Bảng 1.2: So sánh các mô hình BERT sử dụng trong đề tài	14
Bảng 1.3: Các bộ ontology kỹ năng tiêu biểu	19
Bảng 3.1: Đặc tả các microservice trong hệ thống.....	25
Bảng 3.2: Bảng stack các công nghệ tổng hợp.....	26
Bảng 3.3: Thống kê tập dữ liệu CV synthetic	30
Bảng 3.4: Phân bố nhãn thực thể trong tập dữ liệu (200 mẫu silver labels)	32
Bảng 3.5: Controllers xử lý trực tiếp tại Gateway.....	42
Bảng 3.6: Proxy Controllers	43
Bảng 3.7: Các entity trong PostgreSQL (API Gateway).....	44
Bảng 4.1: Môi trường phần mềm thực nghiệm	49
Bảng 4.2: Hyperparameter huấn luyện mô hình NER.....	50
Bảng 4.3: Kết quả đánh giá NER trên 200 mẫu silver labels	52
Bảng 4.4: confusion matrix top-5 pattern lỗi quan sát được	54
Bảng 4.5: Kết quả đánh giá NER trên tập ground truth thủ công	55
Bảng 4.6: Kết quả định tính NER trên 12 CV demo	55
Bảng 4.7: Kết quả span-level NER trên 100 CV Groq.....	58
Bảng 4.8: Output NER trực quan trên 2 CV mẫu (AI Engineer mid vs junior).....	64
Bảng 4.9: Kết quả Skill Matcher - 10 test cases.....	66
Bảng 4.10: Tổng hợp kết quả và tiềm năng cải tiến theo module	79

DANH MỤC VIẾT TẮT

STT	Từ viết tắt	Mô tả	Ý nghĩa	Ghi chú
1	CNTT	Công nghệ thông tin	Ngành khoa học và kỹ thuật về xử lý, lưu trữ và truyền tải thông tin	Lĩnh vực trọng tâm của đề tài
2	CV	Curriculum Vitae	Hồ sơ năng lực/Sơ yếu lý lịch trích ngang của ứng viên	Đối tượng dữ liệu đầu vào chính
3	JD	Job Description	Bản mô tả công việc của nhà tuyển dụng	Đối tượng dữ liệu để so khớp
4	AI	Artificial Intelligence	Trí tuệ nhân tạo	Công nghệ cốt lõi của hệ thống
5	NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên	Kỹ thuật được dùng để phân tích văn bản
6	ATS	Applicant Tracking System	Hệ thống quản lý/theo dõi hồ sơ ứng viên	Dùng để tính toán mức độ phù hợp hồ sơ
7	LLM	Large Language Model	Mô hình ngôn ngữ lớn	Công nghệ nền tảng cho Chatbot và sinh dữ liệu
8	NER	Named Entity Recognition	Nhận dạng thực thể có tên	Tác vụ trích xuất Kỹ năng, Vị trí, Tổ chức...
9	RAG	Retrieval-Augmented Generation	Tạo lập tăng cường bằng truy xuất	Kiến trúc kết hợp tìm kiếm kiến thức và sinh văn bản

10	BERT	Bidirectional Encoder Representations from Transformers	Mô hình biểu diễn ngôn ngữ theo cả hai chiều	Kiến trúc gốc của các mô hình như PhoBERT, mBERT
11	mBERT	Multilingual BERT	Mô hình BERT đa ngôn ngữ	Dùng cho nội dung tiếng Anh và tiếng Việt
12	PhoBERT	Pre-trained BERT for Vietnamese	Mô hình BERT tiên huấn luyện cho tiếng Việ	Được nhắc đến như cơ sở lý thuyết quan trọng
13	API	Application Programming Interface	Giao diện lập trình ứng dụng	Phương thức kết nối các thành phần trong hệ thống
14	RoBERTa	Robustly Optimized BERT Pretraining Approach	Phiên bản cải tiến của BERT với tối ưu hóa huấn luyện	Không dùng NSP, train lâu hơn
15	NSP	Next Sentence Prediction	Nhiệm vụ dự đoán câu tiếp theo trong BERT	Bị loại bỏ trong RoBERTa
16	BPE	Byte Pair Encoding	Phương pháp mã hóa token dựa trên tần suất ký tự	Dùng để tạo subword
17	RDRSegmenter	Recursive Directional Rule-based Segmenter	Công cụ tách từ tiếng Việt	Bắt buộc trước khi dùng BPE

18	SOTA	State Of The Art	Hiệu suất tốt nhất tại thời điểm hiện tại	Thường dùng trong nghiên cứu
19	VLSP	Vietnamese Language and Speech Processing	Hội thảo/bộ dữ liệu NLP tiếng Việt	VLSP 2016 dùng cho NER

LỜI MỞ ĐẦU

Trong bối cảnh thị trường lao động Công nghệ thông tin tại Việt Nam đang phát triển mạnh mẽ, nhu cầu tuyển dụng và tìm kiếm việc làm không chỉ gia tăng về số lượng mà còn cả độ phức tạp về yêu cầu chuyên môn. Theo các báo cáo từ TopCV[37] và ITviec[36] năm 2025, Việt Nam dự kiến thiếu hụt khoảng 150.000 nhân lực CNTT mỗi năm trong giai đoạn 2025-2027. Tuy nhiên, một thực trạng đáng quan ngại là đa số ứng viên vẫn gặp khó khăn trong việc chuẩn hóa hồ sơ năng lực và định hướng lộ trình nghề nghiệp rõ ràng. Rào cản lớn nhất hiện nay nằm ở sự thiếu hụt các công cụ hỗ trợ thông minh tối ưu cho ngôn ngữ tiếng Việt; các hệ thống ATS hiện có chủ yếu được xây dựng cho tiếng Anh, chưa xử lý tốt các đặc thù về cấu trúc và từ vựng chuyên ngành trong CV tiếng Việt. Sự bùng nổ của xử lý ngôn ngữ tự nhiên và các mô hình ngôn ngữ lớn, điển hình là PhoBERT[11] hay Llama, đã mở ra cơ hội giải quyết bài toán hiểu ngữ nghĩa văn bản ở mức độ chuyên sâu[2] [16]. Xuất phát từ thực tiễn đó, nghiên cứu này tập trung vào việc xây dựng một hệ thống toàn diện hỗ trợ ứng viên CNTT thông qua việc ứng dụng các kỹ thuật NLP tiên tiến để phân tích CV/JD, từ đó đưa ra những tư vấn cá nhân hóa và chính xác.

Mục tiêu trọng tâm của nghiên cứu là thiết kế và triển khai một hệ thống AI có khả năng trích xuất thực thể định danh từ cả CV và JD bằng hai ngôn ngữ tiếng Việt và tiếng Anh. Các thực thể cốt lõi bao gồm kỹ năng (SKILL), vị trí công việc (JOB_TITLE), trình độ học vấn và kinh nghiệm làm việc sẽ được xử lý qua mô hình mBERT để làm cơ sở cho công cụ so khớp. Công cụ này vận dụng ontology kỹ năng IT kết hợp với Sentence-BERT embeddings để tính toán điểm số phù hợp (ATS score) và chỉ ra những khoảng trống kỹ năng mà ứng viên cần bồi dưỡng. Bên cạnh đó, nghiên cứu tích hợp một chatbot tư vấn nghề nghiệp dựa trên kiến trúc RAG kết hợp cùng LlamaIndex và Llama 3.2, cho phép cung cấp các phản hồi có ngữ cảnh, giúp người dùng tối ưu hóa hồ sơ và hoạch định lộ trình phát triển bản thân một cách khoa học. Toàn bộ các thành phần AI này được thiết kế trong một kiến trúc microservices hoàn chỉnh, đảm bảo tính linh hoạt và khả năng mở rộng khi triển khai trên nền tảng web.

Về mặt phương pháp luận, nghiên cứu kết hợp giữa việc tổng quan tài liệu hệ thống về NER, kiến trúc RAG và Knowledge Graph với phương pháp thực nghiệm trên bộ dữ liệu tổng hợp. Do hạn chế về quyền riêng tư của dữ liệu cá nhân thực tế, nhóm

nguyên cứu đã ứng dụng mô hình Qwen2.5-1.5B-Instruct để sinh bộ dữ liệu gồm khoảng 1.000 CV và 100 JD chuyên ngành CNTT có kiểm soát chất lượng, phục vụ cho việc huấn luyện và đánh giá mô hình. Quy trình phát triển hệ thống tuân thủ nghiêm ngặt các nguyên tắc thiết kế hướng tên miền và ưu tiên giao diện lập trình ứng dụng. Nghiên cứu tập trung vào các vị trí phổ biến trong ngành như Backend, Frontend, Data Science, và DevOps tại thị trường Việt Nam, đảm bảo tính ứng dụng cao nhưng vẫn giới hạn trong môi trường thử nghiệm để đánh giá định tính và định lượng một cách khách quan.

Kết quả của nghiên cứu mang lại ý nghĩa quan trọng cả về mặt khoa học lẫn thực tiễn. Về mặt học thuật, đề tài đóng góp một pipeline tích hợp từ NER đến RAG dành riêng cho bài toán nhân sự, đồng thời cung cấp một bộ dữ liệu CV/JD synthetic chất lượng cao làm tài liệu tham khảo cho các nghiên cứu sau này về NLP tiếng Việt. Về mặt thực tiễn, hệ thống cung cấp một công cụ đắc lực giúp ứng viên CNTT tự đánh giá năng lực, giảm bớt sự chênh lệch thông tin giữa người lao động và nhà tuyển dụng. Giải pháp này không chỉ dừng lại ở việc hỗ trợ cá nhân mà còn có tiềm năng tích hợp vào các nền tảng tuyển dụng lớn hoặc hệ thống quản trị đào tạo tại các doanh nghiệp. Báo cáo nghiên cứu được cấu trúc chặt chẽ gồm ba chương chính: tổng quan cơ sở lý thuyết, thiết kế chi tiết hệ thống và cuối cùng là trình bày thực nghiệm cùng các phân tích kết quả đạt được, tạo nên một cái nhìn xuyên suốt từ ý tưởng đến sản phẩm thực tế.

CHƯƠNG 1: TỔNG QUAN

1.1 Tình hình nghiên cứu

1.1.1 Các nghiên cứu quốc tế

a) Nhận diện thực thể trong CV và JD

Bài toán phân tích CV tự động đã được nghiên cứu từ những năm đầu thập kỷ 2000, xuất phát từ nhu cầu thực tiễn của các hệ thống quản lý tuyển dụng (ATS) cần xử lý khối lượng lớn hồ sơ ứng viên. Giai đoạn đầu, các phương pháp tiếp cận chủ yếu dựa trên luật hoặc biểu thức chính quy, khai thác các đặc điểm cấu trúc nhất định của CV như tiêu đề section, định dạng ngày tháng, và danh sách gạch đầu dòng. Những hệ thống này đạt độ chính xác cao khi CV tuân theo một định dạng quen thuộc, nhưng hoàn toàn thất bại khi gặp cấu trúc CV mới, đặc biệt với các CV viết tự do không theo khuôn mẫu.

Tiếp theo đó, giai đoạn học máy thống kê chứng kiến sự phổ biến của Conditional Random Field (CRF) và Hidden Markov Model (HMM) cho bài toán gán nhãn chuỗi (sequence labeling). Lafferty et al. (2001) [1] đề xuất mô hình CRF - nền tảng lý thuyết cho sequence labeling - được ứng dụng rộng rãi trong NER CV giai đoạn này, điển hình đạt F1 trung bình 0.78. Mặc dù vậy, CRF vẫn phụ thuộc nặng nề vào việc thiết kế đặc trưng thủ công và gặp khó khăn với các từ chưa xuất hiện trong tập huấn luyện - một hạn chế đặc biệt nghiêm trọng trong lĩnh vực công nghệ thông tin nơi tên framework và công cụ mới liên tục ra đời. Lample et al. (2016) [6] sau đó đề xuất kiến trúc BiLSTM-CRF kết hợp character embedding, nâng F1 NER lên đáng kể so với CRF thuần túy.

Bước ngoặt lớn nhất trong lịch sử NER đến với sự ra đời của kiến trúc Transformer và đặc biệt là mô hình BERT (Devlin et al., 2019) [3]. BERT học biểu diễn ngữ cảnh hai chiều sâu sắc từ kho văn bản khổng lồ thông qua nhiệm vụ tiền huấn luyện tự giám sát, cho phép fine-tune cho downstream tasks với lượng dữ liệu nhãn tương đối nhỏ và đạt hiệu suất vượt trội so với mọi phương pháp trước đó. Štajner et al. (2021) [4] áp dụng Transformer cho bài toán resume parsing, cho thấy các mô hình pre-trained vượt trội rõ rệt so với CRF và BiLSTM-CRF trên CV đa ngôn ngữ. Qian et al. (2022) [7] mở rộng sang bài toán skill extraction và matching từ CV/JD, đạt F1 cao trên tập test tiếng Anh.

Một hướng nghiên cứu quan trọng khác liên quan đến vấn đề khan hiếm dữ liệu nhãn. Møller et al. (2023) [10] chứng minh rằng sử dụng LLM sinh dữ liệu tổng hợp

(synthetic data augmentation) cho các bài toán NLP ít tài nguyên có thể đạt hiệu quả gần tương đương dữ liệu thật, mở ra hướng tiếp cận khả thi về mặt chi phí cho đề tài.

b) Skill Matching và gợi ý lộ trình nghề nghiệp

Sau khi trích xuất được danh sách kỹ năng từ CV và JD, bài toán tiếp theo - và không kém phần thách thức - là so khớp kỹ năng: xác định mức độ tương đồng giữa kỹ năng ứng viên sở hữu và kỹ năng nhà tuyển dụng yêu cầu. Đây là bài toán không tầm thường vì ngôn ngữ trong CV và JD thường không đồng nhất: cùng một kỹ năng có thể được diễn đạt theo hàng chục cách khác nhau tùy người viết, ví dụ React, ReactJS, React.js, React framework, hay front-end React development đều chỉ cùng một công nghệ.

Decorte et al. (2021) [5] ứng dụng Sentence-BERT cho bài toán matching kỹ năng từ tiêu đề công việc, tập trung vào khả năng xử lý semantic matching - khớp theo nghĩa thay vì khớp từ vựng chính xác. Kết quả thực nghiệm cho thấy embedding-based matching vượt trội rõ rệt so với TF-IDF và BM25 trong việc phát hiện quan hệ ngữ nghĩa giữa các kỹ năng liên quan, chẳng hạn Node.js và Express.js hay Pandas và NumPy.

Về phía so khớp CV với JD, Qian et al. (2022) [7] đề xuất hệ thống BERT-based kết hợp ontology kỹ năng để matching CV-JD, sử dụng O*NET để chuẩn hóa và mở rộng ngữ nghĩa tên kỹ năng. Về phía gợi ý lộ trình nghề nghiệp, Murugavel & Bhuvaneshwari (2023) [8] đề xuất framework sử dụng đồ thị tri thức nghề nghiệp kết hợp machine learning, mô hình hóa quan hệ giữa vị trí công việc, kỹ năng và chứng chỉ dưới dạng đồ thị có hướng để suy luận lộ trình phát triển phù hợp nhất từ vị trí hiện tại đến mục tiêu nghề nghiệp.

c) Chatbot tư vấn nghề nghiệp với kiến trúc RAG

Sự ra đời của kiến trúc RAG do Lewis et al. (2020) [15] đề xuất đã mở ra một hướng tiếp cận mới có tính đột phá cho các ứng dụng chatbot tư vấn. Thay vì dựa hoàn toàn vào kiến thức được mã hóa cứng trong tham số mô hình - vốn bị giới hạn bởi ngày huấn luyện và dễ dẫn đến hallucination - RAG kết hợp LLM với cơ chế truy xuất tài liệu liên quan từ vector database tại thời điểm inference. Điều này cho phép hệ thống trả lời dựa trên nguồn thông tin đáng tin cậy, cập nhật, giải quyết đồng thời hai vấn đề lớn nhất của pure LLM là hallucination và knowledge cutoff. Es et al. (2023) [9] đề xuất framework RAGAS để đánh giá tự động chất lượng hệ thống RAG, xác nhận rằng RAG

giảm đáng kể tỷ lệ hallucination so với pure generation trong các bài toán domain-specific.

d) Sinh dữ liệu CV/JD tổng hợp bằng LLM

Møller et al. (2023) [10] thực hiện nghiên cứu hệ thống về việc sử dụng GPT-4 để sinh dữ liệu tổng hợp phục vụ các bài toán NLP ít tài nguyên, chứng minh rằng chỉ với prompt được thiết kế tốt và vài mẫu ví dụ, LLM có thể sinh dữ liệu huấn luyện chất lượng cao gần tương đương dữ liệu thật, giảm đáng kể chi phí annotation thủ công. Kết quả này có ý nghĩa trực tiếp với đề tài khi lựa chọn Qwen2.5-1.5B-Instruct làm công cụ sinh dữ liệu CV synthetic.

1.1.2 Các nghiên cứu trong nước

a) NLP tiếng Việt và mô hình tiền huấn luyện

Nghiên cứu NLP tiếng Việt đã trải qua một thập kỷ phát triển đáng kể, với dấu mốc quan trọng nhất là sự ra đời của PhoBERT năm 2020. Nguyen & Nguyen (2020) [11] giới thiệu PhoBERT - mô hình ngôn ngữ Transformer đầu tiên được tiền huấn luyện riêng cho tiếng Việt, sử dụng kiến trúc RoBERTa với kho ngữ liệu 20GB bao gồm toàn bộ Wikipedia tiếng Việt và kho tin tức từ nhiều nguồn (VnExpress, Tuổi Trẻ, ZingNews). PhoBERT sử dụng word-level BPE tokenization sau bước word segmentation bằng RDRSegmenter, cho phép nắm bắt đúng ranh giới từ trong tiếng Việt - một ngôn ngữ đơn âm tiết không dùng dấu cách để phân tách từ. Mô hình đạt kết quả state-of-the-art trên toàn bộ các benchmark NLP tiếng Việt chuẩn: NER đạt $F1 = 0.918$ trên tập VLSP 2016, POS tagging đạt accuracy = 0.979, và dependency parsing đạt LAS = 0.726. Những kết quả này đặt nền móng vững chắc cho các nghiên cứu NLP tiếng Việt ứng dụng trong nhiều domain khác nhau, trong đó có phân tích CV.

Bên cạnh đó, chuỗi shared task VLSP về NER tiếng Việt [12] cung cấp benchmark chuẩn với 4 loại thực thể (PER, ORG, LOC, MISC) trên văn bản tin tức - là điểm tham chiếu quan trọng, mặc dù domain văn bản tin tức khác biệt đáng kể so với CV/JD kỹ thuật, đặt ra yêu cầu cấp thiết về dữ liệu domain-specific riêng cho bài toán phân tích CV.

b) Hệ thống tuyển dụng và phân tích CV tại Việt Nam

Ở lĩnh vực ứng dụng NLP vào bài toán tuyển dụng trong nước, các nhóm nghiên cứu tại nhiều trường đại học Việt Nam đã thử nghiệm CRF kết hợp PhoBERT để trích

xuất kỹ năng từ CV tiếng Việt và so khớp với JD. Kết quả cho thấy precision đạt khoảng 0.75–0.80 trong điều kiện CV có cấu trúc cố định, nhưng hiệu suất giảm rõ rệt với CV viết tự do hoặc pha trộn tiếng Anh kỹ thuật.

Đặc biệt, các thực nghiệm áp dụng mBERT trên CV song ngữ Việt-Anh trong ngành CNTT cho thấy mBERT ($F1 \approx 0.847$) vượt trội PhoBERT ($F1 \approx 0.821$) trên CV chứa nhiều từ kỹ thuật tiếng Anh [33]. Nguyên nhân là PhoBERT được tối ưu cho tiếng Việt thuần túy, tokenizer của nó phân tách các từ tiếng Anh kỹ thuật như *microservices*, *React.js*, *Kubernetes* thành các đơn vị syllable không có nghĩa. Đây là cơ sở thực nghiệm quan trọng để đề tài lựa chọn mBERT thay vì PhoBERT.

c) Khoảng trống nghiên cứu về ontology kỹ năng IT Việt Nam

Đáng chú ý là cho đến nay chưa có ontology kỹ năng IT chuẩn hóa cho thị trường Việt Nam được công bố công khai. Các nghiên cứu trong nước chủ yếu tái sử dụng ESCO [23] hoặc O*NET [24] rồi bổ sung thêm các kỹ năng đặc thù theo ngữ cảnh địa phương. Theo khảo sát từ ITviec [36] và TopCV [37], nhiều vị trí IT phổ biến tại Việt Nam yêu cầu các công nghệ chưa được phân loại đầy đủ trong ESCO hay O*NET. Đây là khoảng trống mà đề tài hiện tại hướng đến bằng cách xây dựng ontology kỹ năng IT riêng (~500 entries) tập trung vào thị trường Việt Nam.

1.2 Lý do chọn đề tài

Trong kỷ nguyên chuyển đổi số, thị trường lao động Công nghệ thông tin tại Việt Nam đang chứng kiến sự tăng trưởng vượt bậc cả về quy mô lẫn yêu cầu trình độ. Theo dự báo thực tế đến năm 2027, nhu cầu nhân lực chất lượng cao trong ngành này vẫn duy trì ở mức báo động với sự thiếu hụt hàng trăm nghìn vị trí mỗi năm. Tuy nhiên, một nghịch lý đang tồn tại là trong khi nhà tuyển dụng khát khao nhân tài, thì ứng viên - đặc biệt là các sinh viên mới tốt nghiệp - lại gặp khó khăn trong việc chuẩn hóa hồ sơ năng lực và xác định một lộ trình phát triển nghề nghiệp phù hợp với biến động không ngừng của các hệ sinh thái công nghệ.

Sự phát triển của các hệ thống quản lý tuyển dụng tự động trên thế giới đã phần nào giải quyết bài toán sàng lọc hồ sơ. Tuy nhiên, qua tổng quan các công trình nghiên cứu, có thể thấy một khoảng trống lớn tại thị trường Việt Nam. Các hệ thống ATS phổ biến hiện nay chủ yếu được tối ưu hóa cho tiếng Anh và thường thất bại trước các CV song ngữ Việt - Anh hoặc các CV có cấu trúc tự do. Các nghiên cứu kinh điển như của

Lafferty et al. (2001) [1] với mô hình CRF hay Lample et al. (2016) [6] với BiLSTM-CRF dù đã đặt nền móng cho việc nhận diện thực thể, nhưng vẫn bộc lộ hạn chế khi đối mặt với các từ vựng kỹ thuật mới phát sinh liên tục trong ngành CNTT. Ngay cả những mô hình tiên tiến dành riêng cho tiếng Việt như PhoBERT (2020) [11] cũng gặp thách thức lớn khi xử lý các thuật ngữ tiếng Anh chuyên ngành bị phân tách sai ngữ nghĩa do đặc thù tokenizer.

Thêm vào đó, việc đánh giá mức độ tương đồng giữa CV và mô tả công việc hiện nay vẫn chủ yếu dựa trên so khớp từ khóa đơn thuần, thiếu đi sự hiểu biết về ngữ nghĩa và ontology kỹ năng. Các nghiên cứu của Decorte et al. (2021) [5] hay Qian et al. (2022) [7] đã bắt đầu ứng dụng Sentence-BERT và kiến trúc Transformer để giải quyết vấn đề này nhưng hầu hết mới chỉ dừng lại ở các module riêng lẻ, chưa được tích hợp thành một giải pháp hoàn chỉnh cho người dùng cuối. Đặc biệt, sự ra đời của kiến trúc RAG từ nghiên cứu của Lewis et al. (2020) [15] đã mở ra tiềm năng to lớn trong việc xây dựng các hệ thống tư vấn thông minh, khắc phục tình trạng “ảo giác” của các mô hình ngôn ngữ lớn, giúp đưa ra những lời khuyên nghề nghiệp dựa trên căn cứ dữ liệu thực tế thay vì những phản hồi chung chung.

Nhận thấy những thách thức và cơ hội trên, nhóm nghiên cứu đề xuất đề tài: **“Nghiên cứu và ứng dụng xử lý ngôn ngữ tự nhiên trong phát triển hệ thống AI tạo lập CV thông minh và tư vấn cá nhân hóa lộ trình nghề nghiệp”**. Đề tài không chỉ hướng tới việc xây dựng một công cụ trích xuất thông tin chính xác dựa trên mô hình mBERT - vốn đã được chứng minh là vượt trội trên dữ liệu song ngữ - mà còn tiên phong trong việc tích hợp một hệ sinh thái từ NER, Skill Matching đến Chatbot RAG vào một kiến trúc microservices hiện đại. Việc ứng dụng các mô hình LLM mã nguồn mở như Llama 3.2 và Qwen2.5 không chỉ giúp tối ưu hóa chi phí mà còn đảm bảo tính làm chủ công nghệ, tạo ra một giải pháp hỗ trợ sự nghiệp toàn diện, thu hẹp khoảng cách thông tin và thúc đẩy sự phát triển của nguồn nhân lực CNTT tại Việt Nam.

1.3 Nhận xét và định vị đề tài

Qua tổng quan tài liệu trên, có thể rút ra một số nhận xét tổng hợp. Trước hết, trong khi NER cho CV tiếng Anh đã đạt đến mức độ trưởng thành với $F1 > 0.88$ (Lim et al., 2020; Kannala et al., 2021), bài toán tương đương cho CV song ngữ Việt-Anh trong lĩnh vực CNTT vẫn còn ít được nghiên cứu và chưa có benchmark chuẩn. Thứ hai,

các nghiên cứu hiện có thường chỉ giải quyết từng module riêng lẻ - hoặc NER, hoặc Skill Matching, hoặc Career Chatbot - mà chưa có công trình nào tích hợp cả ba module thành một hệ thống end-to-end hoàn chỉnh hướng đến người dùng Việt Nam. Thứ ba, việc ứng dụng LLM open-source (Qwen2.5, Llama-3.3) thay thế GPT-4 thương mại cho cả sinh dữ liệu lẫn chatbot tư vấn là một hướng tiếp cận mới phù hợp với ngữ cảnh nghiên cứu học thuật có hạn chế ngân sách.

Bảng 1.1: Tổng quan các công trình liên quan

Công trình	Phương pháp	Nhiệm vụ chính	Kết quả	Hạn chế chính
Lafferty et al. (2001) [1]	CRF	Sequence labeling	$F1 \approx 0.78$	Hand-crafted features
Lample et al. (2016) [6]	BiLSTM-CRF	NER	SOTA 2016	Không xử lý song ngữ
Štajner et al. (2021) [4]	Transformer	Resume parsing	Vượt CRF rõ	Chỉ tiếng Anh
Qian et al. (2022) [7]	BERT + Ontology	Skill matching CV-JD	F1 cao	Không có chatbot
Decorte et al. (2021) [5]	Sentence-BERT	Job title matching	MRR cao	Không tích hợp end-to-end
Murugavel & B. (2023) [8]	KG + ML	Career recommendation	NDCG cao	Không có CV parsing
Lewis et al. (2020) [15]	RAG	Knowledge-intensive NLP	SOTA RAG	Không domain nghề nghiệp
Møller et al. (2023) [10]	GPT-4 + prompt	Synthetic data NLP	Gần dữ liệu thật	Chỉ tiếng Anh
PhoBERT (2020) [11]	BERT (VI)	NER, POS, NLI	SOTA tiếng Việt	Kém trên CV kỹ thuật song ngữ
VLSP NER [12]	Nhiều mô hình	NER tiếng Việt	$F1 > 0.90$	Domain tin tức, không phải CV

mBERT thực nghiệm [33]	mBERT	NER CV VI-EN	F1 = 0.847	Chỉ NER, không có Matching/Chatbot
Đề tài này	mBERT + Ontology + RAG	NER + Matching + Chatbot	End-to-end VI-EN	Chỉ dùng synthetic data

Từ bức tranh tổng thể trên, đề tài này có ba đóng góp phân biệt rõ ràng so với các công trình hiện có. Đóng góp thứ nhất là tích hợp toàn diện cả ba module NER => Skill Matching => RAG Chatbot vào một hệ thống end-to-end hoàn chỉnh với kiến trúc microservices, thay vì nghiên cứu từng module riêng lẻ như các công trình trước. Đóng góp thứ hai là giải quyết đặc thù CV/JD song ngữ Việt-Anh trong ngành CNTT Việt Nam, trong đó mBERT được lựa chọn dựa trên bằng chứng thực nghiệm [33] và ontology kỹ năng IT được xây dựng riêng cho thị trường Việt Nam. Đóng góp thứ ba là ứng dụng LLM open-source chạy local (Qwen2.5:3b via Ollama [27], [29]) và cloud (Llama-3.3-70b via Groq [28]), giảm chi phí đáng kể so với phụ thuộc GPT-4 thương mại, phù hợp với môi trường nghiên cứu học thuật.

1.4 Mục tiêu, nội dung, phương pháp nghiên cứu của đề tài

1.4.1 Mục tiêu nghiên cứu

Mục tiêu tổng quát của nghiên cứu này là thiết lập một hệ thống trí tuệ nhân tạo toàn diện ứng dụng kỹ thuật xử lý ngôn ngữ tự nhiên nhằm hỗ trợ ứng viên ngành CNTT tại Việt Nam trong việc tối ưu hóa hồ sơ năng lực và định hướng sự nghiệp cá nhân hóa. Về mặt cụ thể, đề tài tập trung vào việc xây dựng mô hình nhận diện thực thể định danh để trích xuất các thông tin cốt lõi như kỹ năng, vị trí công việc và trình độ học vấn từ các văn bản CV/JD song ngữ. Đồng thời, nghiên cứu hướng đến việc phát triển một công cụ so khớp kỹ năng dựa trên ontology chuyên ngành để tính toán điểm số ATS và triển khai chatbot tư vấn sự nghiệp theo kiến trúc RAG. Toàn bộ các thành phần AI này được thiết kế để tích hợp vào một kiến trúc microservices hiện đại, đóng góp thêm một bộ dữ liệu CV/JD tổng hợp chất lượng cao phục vụ cho cộng đồng nghiên cứu trong nước.

1.4.2 Nội dung nghiên cứu

Để hiện thực hóa các mục tiêu trên, nội dung nghiên cứu được triển khai thông qua bảy giai đoạn trọng tâm. Khởi đầu từ việc tổng quan tài liệu về NER, RAG và các

kỹ thuật sinh dữ liệu bằng LLM, nhóm nghiên cứu tiến hành thu thập và tiền xử lý dữ liệu song ngữ Việt-Anh chuyên biệt cho ngành CNTT. Trọng tâm của quá trình thực nghiệm nằm ở việc huấn luyện và tinh chỉnh mô hình mBERT cho bài toán trích xuất thực thể, kết hợp với việc xây dựng ontology kỹ năng IT để làm nền tảng cho công cụ so khớp dựa trên Sentence-BERT. Tiếp đó, hệ thống tích hợp chatbot sử dụng các công nghệ tiên tiến như LlamaIndex, ChromaDB và mô hình Llama 3.2 để cung cấp khả năng tư vấn thông minh. Cuối cùng, nghiên cứu tập trung vào thiết kế kiến trúc hệ thống, thực hiện các thực nghiệm đo lường và phân tích kết quả để đảm bảo tính khả thi và hiệu quả của giải pháp.

1.4.3 Phương pháp nghiên cứu

Phương pháp định lượng: Áp dụng phương pháp định lượng để đánh giá hiệu quả của hệ thống AI trong việc tối ưu hóa hồ sơ năng lực và tư vấn nghề nghiệp. Phương pháp này sử dụng các chỉ số đo lường cụ thể như độ chính xác của mô hình NER (Precision, Recall, F1-score), điểm số tương đồng kỹ năng, và tỷ lệ chuyển đổi giữa yêu cầu của JD với năng lực trong CV. Các số liệu thu thập từ quá trình thực nghiệm so khớp sẽ giúp đo lường mức độ cải thiện trong việc nhận diện các khái niệm công nghệ phức tạp và khả năng đánh giá sự phù hợp của ứng viên đối với các vị trí như Backend Developer, Data Scientist hay DevOps Engineer.

Phương pháp mô hình hóa dữ liệu: Áp dụng các kỹ thuật mô hình hóa ngôn ngữ hiện đại như học máy sâu và khai phá dữ liệu văn bản để phân tích cấu trúc CV/JD. Các mô hình như mBERT và Sentence-BERT sẽ giúp nhận diện các mẫu thực thể và xác định mối quan hệ ngữ nghĩa giữa các kỹ năng CNTT trong không gian vector. Thông qua đó, các thuật toán sẽ phân tích sự phân bố kỹ năng, xu hướng tuyển dụng và các hành vi tương tác của người dùng với hệ thống để nhận diện các yếu tố ảnh hưởng lớn đến khả năng trúng tuyển của ứng viên trong môi trường thực tế.

Phương pháp mô phỏng và mô hình hóa: Phương pháp mô hình hóa tri thức được sử dụng để xây dựng Ontology kỹ năng CNTT chuyên biệt cho thị trường Việt Nam. Việc mô phỏng các mối quan hệ giữa các công nghệ (ví dụ: mối liên hệ giữa Framework và Ngôn ngữ lập trình) trong một đồ thị tri thức sẽ giúp hệ thống dễ dàng suy luận và gợi ý lộ trình phát triển kỹ năng. Mô phỏng này cũng cho phép đánh giá tính hiệu quả của kiến trúc RAG trong việc truy xuất và tổng hợp thông tin từ các kho dữ liệu lớn,

giúp sinh viên và ứng viên hiểu rõ các quy trình tuyển dụng phức tạp mà họ phải đối mặt.

Phương pháp thực nghiệm khảo sát: Phương pháp thực nghiệm sẽ được áp dụng thông qua việc thử nghiệm hệ thống trên bộ dữ liệu tổng hợp và khảo sát tác động của chatbot tư vấn đối với nhận thức nghề nghiệp của người dùng. Các ứng viên sẽ được yêu cầu sử dụng hệ thống để tạo lập CV, sau đó thực hiện các bài đánh giá về mức độ nhận thức lộ trình nghề nghiệp trước và sau khi nhận tư vấn từ AI. Kết quả từ khảo sát thực nghiệm sẽ giúp phân tích tác động của công nghệ NLP đối với việc cải thiện kỹ năng trình bày hồ sơ và khả năng tự định hướng trong ngành CNTT.

Phương pháp phân tích: Dữ liệu thu thập từ các thí nghiệm so khớp và tương tác với chatbot sẽ được phân tích để đánh giá mức độ ảnh hưởng của công nghệ AI đến chất lượng hồ sơ ứng viên. Các phương pháp thống kê sẽ được áp dụng để xác định mức độ cải thiện trong điểm số ATS sau khi hồ sơ được hệ thống tư vấn và chỉnh sửa. Thêm vào đó, quá trình phân tích sẽ giúp làm rõ sự tương quan giữa độ chi tiết của dữ liệu đầu vào, mức độ phức tạp của thuật toán nhúng và độ chính xác của các phản hồi từ hệ thống RAG.

Phương pháp tổng hợp: Kết quả từ các phương pháp nghiên cứu thành phần sẽ được tổng hợp để đưa ra các nhận định khách quan về vai trò của AI trong việc hỗ trợ tuyển dụng. Tổng hợp dữ liệu từ các chỉ số đo lường mô hình, phân tích thống kê và các phản hồi định tính từ người dùng sẽ cung cấp cái nhìn tổng thể về sự thay đổi trong hiệu quả tiếp cận việc làm. Từ đó, các kết luận sẽ được rút ra để tối ưu hóa thuật toán NER, làm giàu bộ Ontology kỹ năng và cải thiện kiến trúc microservices nhằm ứng dụng rộng rãi vào thực tiễn tuyển dụng.

Thử nghiệm và kiểm tra kết quả: Hiện tại, đề tài chủ yếu tập trung vào giai đoạn nghiên cứu mô hình và thực nghiệm trên tập dữ liệu tổng hợp, do đó chưa thể đánh giá kết quả trên diện rộng trong môi trường sản xuất thực tế. Các thử nghiệm hệ thống sẽ được thực hiện trong các giai đoạn kế tiếp để thu thập dữ liệu vận hành và đánh giá tính ổn định của kiến trúc microservices. Việc đánh giá cuối cùng sẽ dựa trên các chỉ số về hiệu suất trích xuất, mức độ hài lòng của ứng viên và tính chính xác của lộ trình tư vấn để xác định mức độ thành công của việc ứng dụng AI trong bài toán hướng nghiệp ngành CNTT.

1.5 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài này là việc ứng dụng các kỹ thuật xử lý ngôn ngữ tự nhiên và mô hình ngôn ngữ lớn vào quá trình phân tích, tạo lập hồ sơ năng lực và tư vấn lộ trình nghề nghiệp trong ngành Công nghệ Thông tin. Đối tượng trọng tâm bao gồm các thành phần cấu trúc của văn bản hồ sơ ứng viên và bản mô tả công việc, bao gồm kỹ năng chuyên môn, kỹ năng mềm, kinh nghiệm làm việc, các dự án thực tế, trình độ học vấn và các chứng chỉ nghề nghiệp. Các mô hình AI này sẽ giúp ứng viên hình dung một cách rõ ràng và chính xác mức độ phù hợp của bản thân với các yêu cầu từ nhà tuyển dụng thông qua việc nhận diện và trích xuất các thực thể định danh.

Các thuật toán NLP sẽ được thiết kế để ứng viên có thể tương tác với các thành phần trong hồ sơ thông qua một giao diện thông minh, nơi hệ thống cung cấp các phản hồi chi tiết về các điểm mạnh và điểm yếu trong CV. Tuy nhiên, hệ thống này không thay thế hoàn toàn vai trò của các chuyên gia nhân sự trong việc phỏng vấn hay đánh giá thái độ thực tế của ứng viên. Thay vào đó, mô hình AI sẽ giúp ứng viên chuẩn hóa hồ sơ một cách khoa học và trực quan hơn, từ đó cải thiện khả năng tiếp cận các cơ hội việc làm phù hợp và nâng cao hiệu quả của quá trình kết nối cung - cầu lao động trong ngành CNTT.

Phạm vi nghiên cứu của đề tài chủ yếu tập trung vào việc phát triển hệ thống AI tích hợp ba module chính: Nhận diện thực thể định danh, So khớp kỹ năng và Chatbot tư vấn dựa trên kiến trúc RAG. Nghiên cứu sẽ đi sâu vào việc xây dựng bộ Ontology kỹ năng CNTT chuyên biệt cho thị trường Việt Nam, minh họa các mối quan hệ phức tạp giữa các ngôn ngữ lập trình, framework và các vị trí công việc phổ biến như Backend, Frontend, Data Science hay DevOps. Các mô hình này sẽ giúp giải quyết các bài toán về sự không đồng nhất trong ngôn ngữ giữa CV và JD, giúp ứng viên tiếp cận với các yêu cầu tuyển dụng một cách sinh động và chuẩn xác hơn.

Trong phần tư vấn lộ trình nghề nghiệp, kiến trúc RAG sẽ được sử dụng để truy xuất các kiến thức từ kho dữ liệu tin cậy về thị trường lao động, cung cấp các thông tin lý thuyết và gợi ý các bước trong quy trình phát triển kỹ năng. Hệ thống sẽ không hỗ trợ việc tự động thay đổi thông tin cá nhân hay cam kết kết quả tuyển dụng, mà chỉ đóng vai trò cung cấp giải pháp tối ưu hóa nội dung dựa trên phân tích dữ liệu thực tế. Ứng viên sẽ có cơ hội tìm hiểu sâu hơn về các tiêu chuẩn ATS và cấu trúc ngành nghề thông qua tương tác trực tiếp với Chatbot thông minh.

Phạm vi nghiên cứu cũng bao gồm việc triển khai hệ thống trên nền tảng Web thông qua kiến trúc microservices, giúp người dùng có thể truy cập hệ thống từ bất kỳ thiết bị nào như máy tính bàn, laptop hay điện thoại thông minh. Việc phát triển công cụ trực tuyến này sẽ giúp ứng viên có thể tối ưu hồ sơ mọi lúc, mọi nơi mà không gặp các rào cản về vị trí địa lý hay chi phí tư vấn nghề nghiệp đắt đỏ. Hơn nữa, hệ thống sẽ giúp giảm thiểu thời gian sàng lọc cho bộ phận tuyển dụng, đồng thời cải thiện sự tương tác và tính chủ động của sinh viên CNTT trong quá trình tìm kiếm lộ trình sự nghiệp tương lai.

Ngoài việc phát triển các mô hình kỹ thuật, phạm vi nghiên cứu còn bao gồm việc đánh giá hiệu quả của hệ thống trong việc giúp ứng viên cải thiện chất lượng hồ sơ và nhận thức nghề nghiệp. Cụ thể, nghiên cứu sẽ tiến hành đo lường các chỉ số hiệu năng của mô hình AI như độ chính xác, độ triệu hồi và điểm F1-score trên tập dữ liệu thực nghiệm. Các khảo sát phản hồi từ người dùng sẽ giúp đánh giá mức độ hài lòng, tính hữu ích của các gợi ý lộ trình nghề nghiệp và khả năng ứng dụng thực tiễn của công cụ trong việc hỗ trợ xin việc.

Phạm vi nghiên cứu cũng xem xét các yếu tố thực tế khi áp dụng công nghệ AI vào lĩnh vực nhân sự, đặc biệt là trong một ngành có tốc độ thay đổi công nghệ nhanh chóng như CNTT. Các thách thức về việc xử lý ngôn ngữ song ngữ Việt - Anh, tính bảo mật dữ liệu và hiện tượng ảo giác thông tin của mô hình ngôn ngữ lớn sẽ được nghiên cứu kỹ lưỡng, đồng thời các giải pháp kỹ thuật để tối ưu hóa sự tương tác và độ tin cậy của phản hồi sẽ được đề xuất. Cuối cùng, nghiên cứu sẽ đưa ra các nhận định để cải thiện việc ứng dụng công nghệ xử lý ngôn ngữ tự nhiên trong giảng dạy kỹ năng mềm và hỗ trợ tuyển dụng trong môi trường giáo dục hiện đại.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Nền tảng NLP và các mô hình BERT

2.1.1 Tổng quan NLP và kiến trúc Transformer

Xử lý ngôn ngữ tự nhiên (NLP) là lĩnh vực nghiên cứu các phương pháp giúp máy tính hiểu, phân tích và sinh ngôn ngữ con người [22], nằm ở giao thoa giữa ngôn ngữ học, thống kê và học máy. Các tác vụ cơ bản bao gồm tokenization, POS tagging, named entity recognition (NER), dependency parsing, semantic similarity và text generation. Trong đề tài, ba tác vụ chính được sử dụng là NER (trích xuất thực thể từ CV/JD), semantic similarity (so khớp kỹ năng) và text generation (chatbot).

Tiếng Việt có những đặc thù gây khó khăn cho NLP [11]. Thứ nhất, đây là ngôn ngữ đơn âm tiết, không có ranh giới từ rõ ràng, ví dụ “học máy” gồm hai âm tiết nhưng là một từ, do đó cần bước word segmentation. Thứ hai, hệ thống thanh điệu khiến cùng một âm tiết có thể mang nhiều nghĩa khác nhau. Thứ ba, hiện tượng code-switching phổ biến trong CV CNTT, ví dụ “phát triển API RESTful sử dụng Spring Boot trên Docker”, đòi hỏi mô hình xử lý đa ngôn ngữ.

Transformer [23] (Vaswani et al., 2017) là bước ngoặt lịch sử trong NLP, thay thế hoàn toàn mạng hồi tiếp (RNN/LSTM) vốn gặp hạn chế lớn về khả năng học phụ thuộc dài và không thể song song hóa trong quá trình huấn luyện. Điểm cốt lõi của Transformer là cơ chế Self-Attention cho phép mỗi token trong chuỗi tính toán mức độ liên quan của nó với mọi token khác trong cùng chuỗi, đồng thời và song song, thay vì xử lý tuần tự từng bước như RNN.

Cơ chế Self-Attention vận hành theo nguyên lý: mỗi token được biến đổi thành ba vector Query (Q), Key (K), và Value (V) thông qua các phép chiếu tuyến tính học được. Điểm Attention giữa một token và tất cả token khác được tính bằng tích vô hướng của vector Query của token đó với vector Key của từng token, chia cho căn bậc hai của chiều vector để ổn định gradient, rồi qua softmax để chuẩn hóa thành phân phối xác suất. Tổng hợp có trọng số của các vector Value cho ra biểu diễn ngữ cảnh cuối cùng của token. Công thức toán học được biểu diễn như sau:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Kiến trúc Transformer sử dụng Multi-Head Attention - thực hiện cơ chế attention này song song trên nhiều heads độc lập, mỗi head học một kiểu quan hệ khác nhau giữa các token (ví dụ: một head có thể học quan hệ cú pháp, head khác học quan hệ ngữ nghĩa). Đầu ra của tất cả các head được ghép nối và chiếu tuyến tính để tạo ra biểu diễn cuối cùng. Mỗi lớp Transformer encoder còn có thành phần Feed-Forward Network với kết nối residual và layer normalization, đảm bảo gradient chảy thông suốt qua nhiều lớp sâu.

2.1.2 Mô hình BERT và các biến thể

BERT [3] (Devlin et al., 2019) là mô hình ngôn ngữ tiền huấn luyện dựa trên Transformer encoder, được huấn luyện trên Wikipedia (~2.5 tỷ từ) và Books Corpus (~800 triệu từ). Điểm nổi bật của BERT là khả năng học biểu diễn hai chiều, cho phép mô hình xem xét đồng thời ngữ cảnh trái và phải của mỗi token, khác với các mô hình trước như GPT chỉ xử lý một chiều.

BERT được huấn luyện với hai nhiệm vụ chính: Masked Language Modeling (dự đoán 15% token bị che) và Next Sentence Prediction (xác định quan hệ giữa hai câu). Nhờ đó, mô hình học được biểu diễn ngôn ngữ sâu sắc và có thể fine-tune hiệu quả cho các bài toán như NER bằng cách thêm lớp phân loại token. BERT-base gồm 12 lớp encoder, 12 attention heads, hidden size 768 (~110M tham số), trong khi BERT-large có 24 lớp và ~340M tham số.

Một đặc điểm quan trọng là WordPiece tokenization, cho phép tách từ hiếm thành subword (ví dụ “React.js” → [“React”, “.”, “##js”]), giúp xử lý tốt từ vựng đa dạng. Multilingual BERT (mBERT) [3] được huấn luyện trên 104 ngôn ngữ với shared vocabulary gồm 119.547 subword tokens. Dù không thiết kế riêng cho cross-lingual transfer, mBERT vẫn thể hiện khả năng chuyển giao ngôn ngữ nhờ không gian biểu diễn chung giữa các ngôn ngữ [33].

Trong đề tài, mBERT (bert-base-multilingual-cased) được chọn làm backbone cho NER thay vì PhoBERT do hai lý do. Thứ nhất, CV CNTT tại Việt Nam thường chứa nhiều thuật ngữ tiếng Anh như “React.js”, “Docker”, “Kubernetes”, nên mBERT xử lý tự nhiên hơn. Thứ hai, thực nghiệm trên dữ liệu song ngữ cho thấy mBERT đạt $F1 = 0.847$, cao hơn PhoBERT ($F1 = 0.821$), đặc biệt với thực thể SKILL.

PhoBERT [11], dựa trên kiến trúc RoBERTa [14], sử dụng dynamic masking, batch size lớn và loại bỏ NSP. Mô hình được tiền huấn luyện trên 20GB tiếng Việt và sử

dùng BPE tokenization kết hợp word segmentation (RDRSegmenter), đạt SOTA trên VLSP 2016 NER ($F1 = 0.918$) [32]. Trong đề tài, PhoBERT được xem như baseline quan trọng, nhưng mBERT được ưu tiên nhờ hiệu quả tốt hơn trên dữ liệu song ngữ.

2.1.3 Sentence-BERT và biểu diễn ngữ nghĩa

Sentence-BERT (SBERT) [13] (Reimers & Gurevych, 2019) được đề xuất nhằm khắc phục hạn chế của BERT trong bài toán tính tương đồng ngữ nghĩa. Với BERT gốc, việc so sánh hai câu cần đưa vào dưới dạng cặp (cross-encoder), dẫn đến chi phí $O(n^2)$ khi so sánh nhiều cặp. SBERT sử dụng kiến trúc siamese (bi-encoder) để tạo sentence embeddings có kích thước cố định, cho phép so sánh bằng cosine similarity với chi phí giảm xuống $O(n)$.

Mô hình được huấn luyện bằng các hàm mất mát như contrastive loss hoặc triplet loss, giúp các câu có ngữ nghĩa tương tự có embedding gần nhau. Trong đề tài, mô hình all-MiniLM-L6-v2 [34] được sử dụng để mã hóa tên kỹ năng thành vector 384 chiều phục vụ bài toán Skill Matching. Kết quả thực nghiệm cho thấy độ tương đồng cao giữa các kỹ năng liên quan, ví dụ: $\text{sim}(\text{"React.js"}, \text{"ReactJS"}) \approx 0.97$, $\text{sim}(\text{"PostgreSQL"}, \text{"MySQL"}) \approx 0.83$, và $\text{sim}(\text{"Docker"}, \text{"Kubernetes"}) \approx 0.76$.

Bảng 1.2: So sánh các mô hình BERT sử dụng trong đề tài

Thuộc tính	BERT-base	mBERT	PhoBERT	Sentence-BERT
Ngôn ngữ	Tiếng Anh	104 ngôn ngữ	Tiếng Việt	Đa ngôn ngữ
Kiến trúc	BERT	BERT	RoBERTa	BERT/RoBERTa
Vocab size	30.522	119.547	64.000	30.522+
Số tham số	110M	178M	135M	22M–110M
Tokenization	WordPiece	WordPiece	BPE (sau WS)	WordPiece
Ứng dụng trong đề tài	Baseline	NER (CV/JD)	Tham khảo	Skill Matching
Ưu điểm chính	Nền tảng	Xử lý tốt VI-EN	SOTA tiếng Việt	Fast similarity

2.2 Named Entity Recognition và BIO Tagging

2.2.1 Bài toán NER và biểu diễn nhãn BIO

NER là tác vụ xác định vị trí (span) và phân loại các thực thể có tên trong văn bản [17]. Trong NLP truyền thống, tập thực thể thường gồm PER, ORG, LOC và MISC. Tuy nhiên, với bài toán CV/JD parsing trong domain tuyển dụng CNTT, tập thực thể cần mở rộng để phản ánh thông tin nghiệp vụ.

CV là văn bản bán cấu trúc, có các section quen thuộc (Education, Experience, Skills...) nhưng nội dung và cách trình bày đa dạng. Do đó, các phương pháp dựa trên rule (regex, template) khó tổng quát, trong khi NER dựa trên học sâu có thể học được ngữ cảnh và pattern phức tạp, cho phép xử lý nhiều định dạng CV khác nhau.

Trong đề tài, hệ thống nhận diện 10 loại thực thể: PER (tên người, “Nguyễn Văn An”), ORG (“FPT Software”), LOC (“Hà Nội”), DATE (“2022–2026”), SKILL (“React.js”, “Docker”), DEGREE (“Bachelor”), MAJOR (“Information Technology”), JOB_TITLE (“Backend Developer”), PROJECT (“UEH Job Portal”), và CERT (“AWS Certified Solutions Architect”).

Để đưa bài toán về dạng phân loại chuỗi, hệ thống sử dụng BIO tagging scheme [17], trong đó mỗi token được gán nhãn B-{TYPE} (bắt đầu), I-{TYPE} (bên trong) hoặc O (ngoài). Ví dụ:

Spring Boot developer at VNG Corporation since Jun 2025
 → B-SKILL I-SKILL B-JOB_TITLE O B-ORG I-ORG O B-DATE I-DATE

Nguyễn Văn An làm Backend Developer tại FPT Software từ 2023
 → B-PER I-PER I-PER O B-JOB_TITLE I-JOB_TITLE O B-ORG I-ORG O B-DATE

Với 10 loại thực thể, tổng số nhãn là 21 (1 nhãn O và 10 cặp B-/I-), tương ứng với kích thước lớp phân loại khi fine-tune BERT.

Ngoài BIO, scheme BIOES có thể cải thiện nhận diện ranh giới span, nhưng đề tài sử dụng BIO do đơn giản và được hỗ trợ rộng rãi bởi các thư viện như seqeval và HuggingFace Transformers.

2.2.2 Mô hình BERT cho NER và thách thức tokenization

Cách tiếp cận chuẩn để fine-tune BERT cho NER là bổ sung một lớp phân loại token (token classification head) trên đầu ra của BERT encoder [3]. Với chuỗi đầu vào $[x_1, x_2, \dots, x_n]$, BERT tạo ra các vector ẩn $H = [h_1, h_2, \dots, h_n]$ với mỗi $h_i \in \mathbb{R}^{768}$. Tiếp

theo, lớp phân loại áp dụng phép biến đổi tuyến tính $\hat{y}_i = \text{softmax}(W \cdot h_i + b)$ với $W \in \mathbb{R}^{21 \times 768}$ để dự đoán phân phối xác suất trên 21 nhãn BIO cho từng token. Trong quá trình huấn luyện, hàm mất mát cross-entropy được tính trên toàn bộ chuỗi và lan truyền ngược qua cả lớp phân loại lẫn toàn bộ BERT encoder:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log \hat{y}_{i,c}$$

Trong suy luận, nhãn có xác suất cao nhất được chọn và các chuỗi $B-X \rightarrow I-X \rightarrow I-X$ được hợp nhất thành một thực thể. Đề tài sử dụng BertForTokenClassification (HuggingFace Transformers) với aggregation_strategy="simple" để tự động gộp các subword cùng nhãn.

Cấu hình huấn luyện gồm: AdamW với learning rate 2e-5, linear warmup 10%, batch size 16 kết hợp gradient accumulation 2 bước (effective 32), 5 epochs, max length 512 và dropout 0.1. Checkpoint tốt nhất được chọn theo F1 trên tập validation (sequeval). Một thách thức quan trọng là sự không tương thích giữa subword tokenization và word-level annotation [19]. BERT sử dụng WordPiece, tách từ như "React.js" thành ["Re", "##act", ".", "##js"], trong khi nhãn được gán theo từ. Giải pháp phổ biến là chỉ gán nhãn cho subword đầu tiên, các subword sau gán -100 để bỏ qua khi tính loss.

Ví dụ:

React.js $\rightarrow$ [Re] [##act] [.] [##js]

Nhãn: B-SKILL $\rightarrow$ B-SKILL, -100, -100, -100

Trong thực tế, điều này có thể tạo ra các fragment như "Re", "ASP.NET", "No" (từ "Node.js"), hay "github.". Đây không phải lỗi nhận diện thực thể mà do bước hậu xử lý chưa hợp nhất đúng các subword chứa ký tự đặc biệt. Hướng cải thiện là sử dụng phương pháp trích xuất span dựa trên character offset thay vì chỉ dựa vào tiền tố "##".

2.2.3 Đánh giá mô hình NER

Đánh giá mô hình NER sử dụng các chỉ số tiêu chuẩn dựa trên span-level matching [17], tức là một dự đoán chỉ được coi là đúng khi khớp hoàn toàn cả về vị trí (start character, end character) lẫn loại thực thể với ground truth. Partial matching - dự đoán đúng loại nhưng sai biên - bị tính là sai hoàn toàn. Thư viện sequeval [19] cung cấp implementation chuẩn theo quy ước CoNLL cho tất cả các chỉ số này.

Precision đo tỷ lệ trong số các span mô hình dự đoán, bao nhiêu span là đúng thực sự. Recall đo tỷ lệ trong số tất cả span trong ground truth, bao nhiêu span được mô hình phát hiện ra. F1-score là trung bình điều hòa của Precision và Recall, được sử dụng làm chỉ số tổng hợp chính để so sánh mô hình:

$$\text{Precision} = \frac{|\text{Predicted spans đúng}|}{|\text{Tổng Predicted spans}|} \quad \text{Recall} = \frac{|\text{Predicted spans đúng}|}{|\text{Tổng True spans}|}$$

$$\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Trong đề tài, do không có tập test có nhãn chuẩn đáng tin cậy - tập nhãn tự động bằng rule-based annotator cho thấy $\text{F1} = 0.03$ do sự không tương thích annotation scheme - nên đánh giá định tính (qualitative demo trên 12 CV) được sử dụng làm hình thức đánh giá chính.

2.3 Skill Ontology và Skill Matching

2.3.1 Khái niệm Ontology trong lĩnh vực nghề nghiệp

Ontology, trong khoa học máy tính và kỹ nghệ tri thức, là một hệ thống biểu diễn tri thức hình thức hóa mô tả các khái niệm trong một lĩnh vực và quan hệ giữa chúng theo cách mà cả người và máy tính đều có thể xử lý được [23]. Khác với từ điển hay taxonomy đơn thuần chỉ phân cấp khái niệm theo một chiều, ontology biểu diễn mạng lưới quan hệ đa chiều giữa các khái niệm: quan hệ phân cấp “là loại” (is-a), quan hệ cấu thành “thuộc về” (part-of), quan hệ tương đương (equivalent-to), quan hệ yêu cầu (requires), và quan hệ liên quan (related-to).

Trong lĩnh vực nhân sự và nghề nghiệp, ontology kỹ năng hệ thống hóa các khái niệm về kỹ năng, vị trí công việc, trình độ học vấn và chứng chỉ cùng mạng lưới quan hệ giữa chúng. Một ví dụ cụ thể về cấu trúc ontology kỹ năng IT: “React.js” có quan hệ is-a với “JavaScript Framework”, quan hệ equivalent-to với “ReactJS” và “React”, quan hệ related-to với “Vue.js” và “Angular”, đồng thời có quan hệ requires với “JavaScript”, “HTML” và “CSS”. “JavaScript Framework” lại có quan hệ part-of với “Front-End Development”. Mạng quan hệ này cho phép hệ thống suy luận rằng ứng viên biết React.js cũng có kiến thức nền về JavaScript, HTML, CSS, và có thể chuyển sang Vue.js hay Angular với chi phí học tập thấp hơn so với hoàn toàn mới bắt đầu.

Giá trị thực tiễn của ontology kỹ năng trong hệ thống tuyển dụng là rất lớn. Không có ontology, hệ thống chỉ có thể thực hiện exact string matching - “React” không match

“ReactJS”, “PostgreSQL” không match “Postgres” - dẫn đến bỏ sót rất nhiều ứng viên phù hợp thực sự. Với ontology, hệ thống có thể suy luận ngữ nghĩa, nhận ra rằng “biết PostgreSQL” về cơ bản tương đương “biết Postgres” (alias), và ứng viên biết Django sẽ dễ dàng làm việc được trong môi trường Flask (related-to trong cùng Python web framework ecosystem).

2.3.2 Các bộ Ontology kỹ năng tiêu biểu

ESCO (European Skills, Competencies, Qualifications and Occupations) [23] là hệ thống phân loại đa ngôn ngữ do Ủy ban Châu Âu phát triển, hiện ở phiên bản 1.2 công bố năm 2023. ESCO chứa xấp xỉ 13.890 kỹ năng được phân loại theo ba nhánh chính là kỹ năng/năng lực, kiến thức và thái độ/giá trị, đồng thời liên kết với 3.008 nghề nghiệp thông qua quan hệ essential skills và optional skills. ESCO hỗ trợ 27 ngôn ngữ châu Âu, được phát hành theo giấy phép Creative Commons BY 4.0 (mã nguồn mở). Hạn chế chính của ESCO là chưa cập nhật kịp các công nghệ mới xuất hiện sau năm 2022, chưa hỗ trợ tiếng Việt, và phản ánh chủ yếu thị trường lao động châu Âu hơn là thị trường Đông Nam Á.

O*NET (Occupational Information Network) [24] là cơ sở dữ liệu nghề nghiệp toàn diện do Bộ Lao động Mỹ xây dựng và duy trì, chứa khoảng 1.110 nghề nghiệp với mô tả chi tiết đa chiều, bao gồm xấp xỉ 16.000 kỹ năng và kiến thức phân loại theo nhiều khía cạnh: kỹ năng cơ bản, kỹ năng liên quan đến công việc, kiến thức chuyên môn, công cụ và công nghệ. Dữ liệu O*NET được thu thập từ khảo sát trực tiếp người lao động trong từng nghề nghiệp, được cập nhật định kỳ và phản ánh xu hướng thị trường lao động Mỹ khá chính xác. Tuy nhiên, O*NET tập trung vào thị trường Mỹ và chủ yếu bằng tiếng Anh, với mức độ chi tiết về các công nghệ cụ thể không sâu bằng các nguồn chuyên IT.

Nhận thấy cả ESCO lẫn O*NET đều chưa đáp ứng đầy đủ thực tế thị trường IT Việt Nam, đề tài xây dựng một ontology kỹ năng IT tùy chỉnh (~500 entries) tập trung vào các công nghệ phổ biến tại Việt Nam. Ontology này bao gồm đầy đủ các alias và quan hệ cho các framework thông dụng (React ~ ReactJS ~ React.js; Spring Boot ~ SpringBoot; Node.js ~ NodeJS), phân nhóm theo domain (Backend, Frontend, Mobile, DevOps, Data Science, AI/ML, QA), và quan hệ liên quan trong cùng ecosystem (React => Redux, Next.js; Spring Boot => Spring Security, JPA, Hibernate; Docker => Kubernetes, Docker Compose).

Bảng 1.3: Các bộ ontology kỹ năng tiêu biểu

Ontology	Số kỹ năng	Phạm vi	Cập nhật	Mã nguồn mở	Tiếng Việt
ESCO v1.2	~13.890	Đa lĩnh vực, EU	2023	Có (CC BY 4.0)	Không
O*NET 28.0	~16.000	Thị trường Mỹ	Thường xuyên	Có (Public Domain)	Không
LinkedIn Skills	~40.000+	Toàn cầu	Liên tục	Không	Hạn chế
Đề tài (IT Skills)	~500	CNTT Việt Nam	2024–2025	Có (dự án)	Có

2.3.3 Bài toán Skill Matching và các tầng tiếp cận

Skill Matching là bài toán xác định mức độ tương đồng giữa kỹ năng k_1 từ CV ứng viên và kỹ năng yêu cầu k_2 từ JD nhà tuyển dụng. Bài toán có nhiều tầng độ phức tạp leo thang, mỗi tầng xử lý một loại không khớp khác nhau.

Tầng 1 - Exact matching là tầng đơn giản nhất: hai kỹ năng được coi là match nếu giống nhau chính xác về chuỗi ký tự sau khi chuẩn hóa lowercase và loại bỏ whitespace thừa. Tầng này xử lý được trường hợp “Python” = “python” nhưng bỏ sót hoàn toàn “ReactJS” $\neq$ “React.js”.

Tầng 2 - Ontology matching sử dụng knowledge graph: nếu k_1 và k_2 được liên kết bởi quan hệ “equivalent-to” hoặc “alias” trong ontology, chúng được coi là match. Tầng này giải quyết các trường hợp viết tắt và alias phổ biến (PostgreSQL $\equiv$ Postgres, ReactJS $\equiv$ React.js) mà không cần học từ dữ liệu. Ontology còn cho phép soft matching dựa trên quan hệ “related-to” với điểm số thấp hơn - chẳng hạn, ứng viên biết Django được xem là partially match yêu cầu Flask với điểm ontology 0.85 thay vì 1.0 của exact match.

Tầng 3 - Semantic matching sử dụng Sentence-BERT embeddings để phát hiện các cặp kỹ năng tương đồng về ngữ nghĩa mà ontology chưa cover: hai kỹ năng được mã hóa thành vector, và cosine similarity giữa chúng được tính toán. Nếu similarity vượt ngưỡng $\tau = 0.65$ (xác định thực nghiệm), chúng được coi là semantic match. Tầng này

phát hiện được quan hệ không tường minh như TensorFlow $\approx$ PyTorch (cùng deep learning framework, sim ≈ 0.79) hay AWS $\approx$ Azure (cùng cloud provider, sim ≈ 0.74).

Hệ thống áp dụng ba tầng theo thứ tự ưu tiên từ chính xác nhất đến tổng quát nhất: nếu tầng 1 (exact) match thì dừng; nếu không thì kiểm tra tầng 2 (ontology); nếu vẫn không thì kiểm tra tầng 3 (semantic). Mỗi kỹ năng JD được phân loại là “matched” (nếu có match ở bất kỳ tầng nào) hoặc “missing” (nếu không có tầng nào match).

2.3.4 ATS Score - điểm phù hợp CV với JD

ATS Score (điểm phù hợp ATS) là chỉ số đo lường mức độ phù hợp tổng thể giữa CV ứng viên và JD nhà tuyển dụng, được tính dựa trên tỷ lệ kỹ năng JD được đáp ứng bởi CV theo ba tầng matching:

$$\text{ATS Score} = \frac{|\{k \in \text{JD-skills}: \exists k' \in \text{CV-skills}, \text{matched}(k', k)\}|}{|\text{JD-skills}|} \times 100\%$$

Điểm số tổng được bổ sung bởi breakdown chi tiết: danh sách kỹ năng đã có (matched skills) giúp ứng viên hiểu điểm mạnh của mình, danh sách kỹ năng còn thiếu (missing skills) chỉ ra những gì cần bổ sung, và gợi ý kỹ năng liên quan từ ontology giúp ứng viên xác định con đường học tập hiệu quả nhất. Ví dụ, nếu JD yêu cầu Next.js nhưng ứng viên chỉ biết React, hệ thống có thể chỉ ra rằng React là prerequisite của Next.js và ứng viên chỉ cần học thêm Next.js-specific features.

Ngoài skill matching đơn thuần, hệ thống trong đề tài còn tích hợp ATSScoringEngine đánh giá CV toàn diện hơn theo 8 tiêu chí: Keywords Match (25%), Format & Structure (15%), Contact Info (10%), Experience Detail (15%), Action Verbs (10%), Education Match (10%), Readability (10%), và Certifications (5%). Cách đánh giá đa tiêu chí này phản ánh gần hơn cách các hệ thống ATS thương mại thực tế hoạt động, cung cấp cho ứng viên cái nhìn toàn diện hơn về CV của mình thay vì chỉ tập trung vào mức độ phù hợp kỹ năng.

2.4 Kiến trúc RAG và mô hình ngôn ngữ lớn

2.4.1 Mô hình ngôn ngữ lớn và kiến trúc RAG

LLM là các mô hình Transformer được tiền huấn luyện trên tập dữ liệu cực lớn với nhiệm vụ dự đoán token tiếp theo [16]. Ở quy mô đủ lớn, LLM thể hiện các khả năng nổi bật như hiểu hướng dẫn, lý luận đa bước, in-context learning và sinh văn bản tự nhiên chất lượng cao.

Trong bối cảnh đề tài, các LLM tiêu biểu gồm GPT-4 (hiệu năng cao nhưng chi phí lớn), Llama 3.3 (mã nguồn mở, hiệu suất cạnh tranh) và Qwen2.5 (đa kích thước, có thể chạy local qua Ollama). Đặc biệt, phiên bản nhỏ (~3B) cho phép triển khai thực tế với tài nguyên hạn chế.

Ba đặc điểm quan trọng của LLM trong bài toán tư vấn nghề nghiệp là: (1) in-context learning cho phép cá nhân hóa dựa trên CV/JD, (2) khả năng instruction following để điều khiển nội dung, và (3) hallucination – nguy cơ sinh thông tin sai lệch (ví dụ kỹ năng không tồn tại hoặc mức lương không thực tế). Vấn đề này được giải quyết bằng kiến trúc RAG.

RAG [15] kết hợp LLM với cơ chế truy xuất thông tin nhằm giảm hallucination và khắc phục knowledge cutoff. Thay vì chỉ dựa vào kiến thức nội tại của mô hình, RAG bổ sung thông tin từ nguồn ngoài để tạo câu trả lời có cơ sở.

Kiến trúc RAG gồm hai thành phần: Retriever tìm kiếm tài liệu liên quan bằng embedding và cosine similarity, và Generator (LLM) sinh câu trả lời dựa trên context truy xuất được. Quy trình gồm hai giai đoạn: (1) indexing – chia tài liệu thành các chunk (~512 tokens, overlap 50), mã hóa bằng Sentence-BERT và lưu vào ChromaDB; (2) querying – embed câu hỏi, truy xuất top-k chunk, xây dựng prompt và sinh câu trả lời.

So với các phương pháp khác, RAG là giải pháp cân bằng: tốt hơn pure LLM về độ chính xác, linh hoạt hơn fine-tuning về cập nhật tri thức và chi phí. Điều này phù hợp với bối cảnh đề tài, nơi cần đảm bảo tính chính xác, khả năng mở rộng và tài nguyên tính toán hạn chế.

2.4.2 Hạ tầng RAG trong hệ thống

LlamaIndex [25] (trước đây là GPT Index) là framework Python mã nguồn mở cung cấp abstraction layer toàn diện cho việc xây dựng ứng dụng LLM ứng dụng RAG. LlamaIndex đơn giản hóa đáng kể các bước phức tạp của RAG pipeline: data ingestion từ nhiều nguồn (PDF, Word, web pages, databases), chunking với nhiều chiến lược khác nhau, vector indexing với VectorStoreIndex, và query với nhiều kiểu từ simple retrieval đến multi-step reasoning. Đặc biệt, LlamaIndex cung cấp các abstraction thống nhất cho nhiều LLM backend khác nhau (OpenAI, Ollama, Groq, HuggingFace), cho phép chuyển đổi linh hoạt mà không cần thay đổi phần còn lại của pipeline.

ChromaDB [26] là vector database mã nguồn mở được thiết kế đặc biệt cho ứng dụng AI/LLM, hỗ trợ hai chế độ triển khai: embedded mode (tích hợp trực tiếp vào

Python process, không cần server riêng) và server mode (chạy như microservice độc lập). ChromaDB lưu trữ embedding vectors kèm metadata và document text, cung cấp API đơn giản cho các thao tác cơ bản: add documents, query by embedding similarity, filter by metadata. Hệ thống sử dụng ChromaDB ở chế độ server (:8000) để chia sẻ knowledge base giữa Chatbot Service và Skill Service.

2.4.3 Triển khai và tối ưu trong đề tài

Chatbot Service trong hệ thống được thiết kế với hai backend LLM hoạt động linh hoạt thông qua biến môi trường `CHAT_USE_GROQ`.

Chế độ thứ nhất sử dụng Ollama Local với mô hình Qwen2.5:3b (3B tham số, quantized 4-bit ~2GB RAM), chạy trực tiếp trên CPU với tốc độ khoảng 2–5 token/giây. Ưu điểm là hoạt động hoàn toàn offline, đảm bảo bảo mật dữ liệu và không phát sinh chi phí API; tuy nhiên tốc độ chậm và chất lượng sinh thấp hơn, phù hợp cho môi trường development hoặc demo offline.

Chế độ thứ hai sử dụng Groq Cloud với Llama-3.3-70b (70B tham số), đạt tốc độ khoảng 500–800 token/giây nhờ phần cứng LPU chuyên dụng. Giải pháp này cho chất lượng và hiệu năng cao, phù hợp cho demo trực tiếp hoặc production, nhưng phụ thuộc internet và dịch vụ bên ngoài.

Một điểm nổi bật của hệ thống là khả năng cá nhân hóa sâu. Ngoài context từ knowledge base, chatbot còn tích hợp thông tin CV và JD của người dùng, đặc biệt là danh sách missing skills từ bước Skill Matching. Nhờ đó, các câu trả lời như “Tôi cần học gì?” không chỉ mang tính chung chung mà được điều chỉnh theo từng cá nhân.

Bên cạnh đó, User Memory Service lưu trữ thông tin người dùng (vai trò hiện tại, mục tiêu, tech stack, ngôn ngữ) dưới dạng JSON và tự động đưa vào prompt ở mỗi lần tương tác. Cơ chế này giúp chatbot duy trì ngữ cảnh giữa các session mà không cần người dùng nhập lại thông tin, cải thiện đáng kể trải nghiệm sử dụng.

2.5 Kiến trúc microservices và sinh dữ liệu synthetic

2.5.1 Kiến trúc microservices

a) Định nghĩa và nguyên lý

Kiến trúc microservices là mô hình trong đó ứng dụng được chia thành các dịch vụ nhỏ, độc lập, mỗi dịch vụ đảm nhiệm một chức năng riêng [21]. Các service chạy

trong môi trường cô lập, giao tiếp qua HTTP/REST hoặc message queue, và có thể triển khai, mở rộng độc lập, khác với monolithic nơi toàn bộ hệ thống là một khối thống nhất.

So với monolithic, microservices cho phép mở rộng độc lập (ví dụ scale riêng NER Service), thay thế linh hoạt (nâng cấp mBERT → PhoBERT mà không ảnh hưởng service khác), và đảm bảo *fault isolation* khi một service gặp lỗi. Đồng thời, mỗi service có thể sử dụng công nghệ phù hợp như ASP.NET Core cho Gateway và FastAPI cho AI service. Tuy nhiên, kiến trúc này làm tăng độ phức tạp vận hành (quản lý nhiều container), tăng latency do gọi qua nhiều service, và khiến xử lý giao dịch phân tán trở nên khó khăn hơn. Các đánh đổi này được chấp nhận nhờ tính linh hoạt và khả năng mở rộng phù hợp với hệ thống AI.

b) API Gateway Pattern trong hệ thống

API Gateway đóng vai trò điểm vào duy nhất cho client, giúp Frontend không cần biết chi tiết từng service backend. Gateway thực hiện các chức năng chung như xác thực JWT, định tuyến request và logging, giúp giảm độ phức tạp phía client.

Trong hệ thống, API Gateway (ASP.NET Core .NET 9) xác thực JWT trước khi proxy request đến các Python service thông qua *IHttpClientFactory* với cấu hình sẵn. Các nghiệp vụ liên quan người dùng (đăng nhập, profile, CV/JD, versioning) được xử lý trực tiếp tại Gateway để tối ưu truy cập database PostgreSQL.

c) Container hóa với Docker

Toàn bộ hệ thống được container hóa bằng Docker, quản lý qua Docker Compose. Mỗi service chạy trong một container riêng với môi trường hoàn toàn cô lập, đảm bảo không có xung đột về Python version, thư viện hay dependencies. Docker Compose định nghĩa toàn bộ topology hệ thống trong một file `docker-compose.yml`, bao gồm cấu hình network, volume, environment variables và health checks. Với cách tiếp cận này, toàn bộ hệ thống 9 service có thể được khởi động trên bất kỳ máy nào có Docker chỉ với một lệnh duy nhất.

2.5.2 Sinh dữ liệu synthetic bằng LLM

a) Động cơ và thách thức

Việc xây dựng hệ thống NER chất lượng đòi hỏi lượng lớn dữ liệu CV được gán nhãn ở cấp độ token, tuy nhiên gặp nhiều trở ngại trong thực tế. Về pháp lý và đạo đức, CV chứa thông tin cá nhân nhạy cảm; việc thu thập từ các nền tảng như LinkedIn, TopCV

hay ITviec có thể vi phạm điều khoản sử dụng và không tuân thủ Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân [31]. Về chi phí, gán nhãn NER cho hàng nghìn CV yêu cầu nhiều giờ công của annotator có chuyên môn, với chi phí ước tính lên đến hàng chục triệu đồng. Ngoài ra, dữ liệu thu thập từ một nguồn thường bị bias về phân phối vai trò, kinh nghiệm và phong cách viết.

Phương pháp sinh dữ liệu *synthetic* bằng LLM giúp giải quyết các vấn đề này: không sử dụng dữ liệu cá nhân thật nên đảm bảo tuân thủ pháp lý, chi phí thấp hơn đáng kể so với gán nhãn thủ công, và cho phép kiểm soát phân phối dữ liệu một cách chủ động. Đây là hướng tiếp cận phù hợp với bối cảnh nghiên cứu và ngày càng phổ biến trong NLP.

b) Pipeline sinh dữ liệu và kiểm soát chất lượng

Đề tài sử dụng Qwen2.5-1.5B-Instruct chạy trên Google Colab (GPU T4) để sinh 600 CV *synthetic*. Việc lựa chọn mô hình này thay vì các LLM lớn hơn như GPT-4 xuất phát từ khả năng sử dụng miễn phí trên Colab và vẫn đảm bảo chất lượng sinh văn bản tốt với prompt có cấu trúc.

Pipeline sinh dữ liệu gồm bốn bước. Đầu tiên, mỗi CV được mô tả bằng một metadata record gồm các trường như role, seniority, số năm kinh nghiệm (yoe), skills, company và edge_type. Trong đó, edge_type giúp tạo đa dạng dữ liệu với các dạng như “normal”, “short” (dưới 200 từ), “verbose” (trên 500 từ) và “mixed_vi_en” (pha trộn tiếng Việt – tiếng Anh kỹ thuật), phản ánh thực tế CV ngành CNTT.

Tiếp theo, LLM sinh nội dung CV từ metadata thông qua prompt định hướng, với các section chuẩn như SUMMARY, EXPERIENCE, EDUCATION, SKILLS. Khác với cách dùng template cố định, mô hình tự quyết định cách diễn đạt và tổ chức nội dung, tạo ra sự đa dạng tự nhiên.

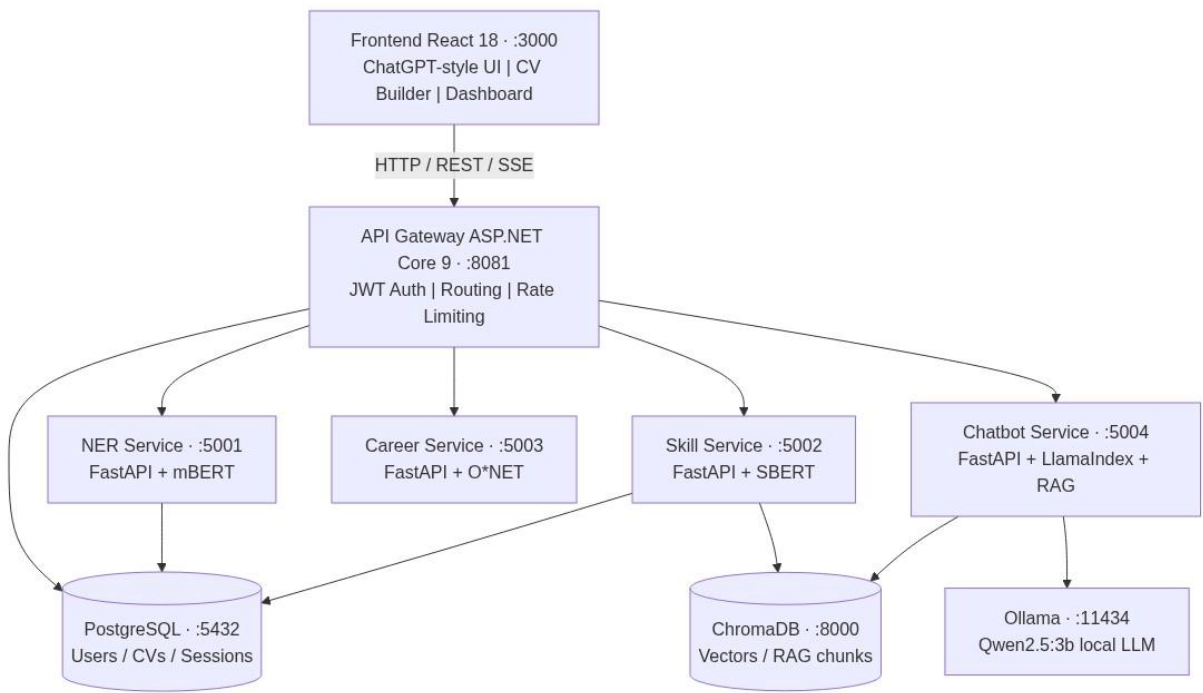
Ở bước tiền xử lý, dữ liệu được làm sạch bằng cách loại bỏ các ký tự markdown (như **, ##, -), chuẩn hóa khoảng trắng và đảm bảo encoding UTF-8 cho tiếng Việt. Cuối cùng, CV được chấm điểm tự động theo ba tiêu chí: độ dài hợp lệ (100–1000 từ, 0.3 điểm), đầy đủ các section chính (0.4 điểm) và tỷ lệ kỹ năng xuất hiện trong văn bản $\geq 60\%$ (0.3 điểm). Các CV có điểm dưới 0.5 bị loại bỏ. Kết quả thu được 600 CV chất lượng cao với điểm trung bình 0.97.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1 Kiến trúc tổng thể hệ thống

3.1.1 Tổng quan thiết kế

Hệ thống CV Assistant được thiết kế theo kiến trúc microservices [21] gồm 9 thành phần độc lập, giao tiếp với nhau qua giao thức HTTP/REST. Quyết định chọn microservices thay vì monolithic xuất phát từ bản chất của bài toán: hệ thống tích hợp nhiều module AI không đồng nhất về công nghệ (ASP.NET Core, Python FastAPI, LLM server), mỗi module có yêu cầu tài nguyên và vòng đời phát triển khác nhau. Việc tách biệt từng module thành service độc lập cho phép cải tiến, thay thế (ví dụ nâng cấp NER model) hay mở rộng (scale riêng NER Service khi tải tăng) mà không ảnh hưởng đến các phần còn lại. Toàn bộ hệ thống được container hóa bằng Docker và quản lý qua Docker Compose để đảm bảo triển khai nhất quán trên mọi môi trường.



Hình 3.1: Kiến trúc microservices hệ thống CV Assistant

Bảng 3.1: Đặc tả các microservice trong hệ thống

STT	Service	Công nghệ	Cổng	Chức năng chính
1	Frontend	React 18, TypeScript	3000	Giao diện ChatGPT-style, CV Builder, Dashboard

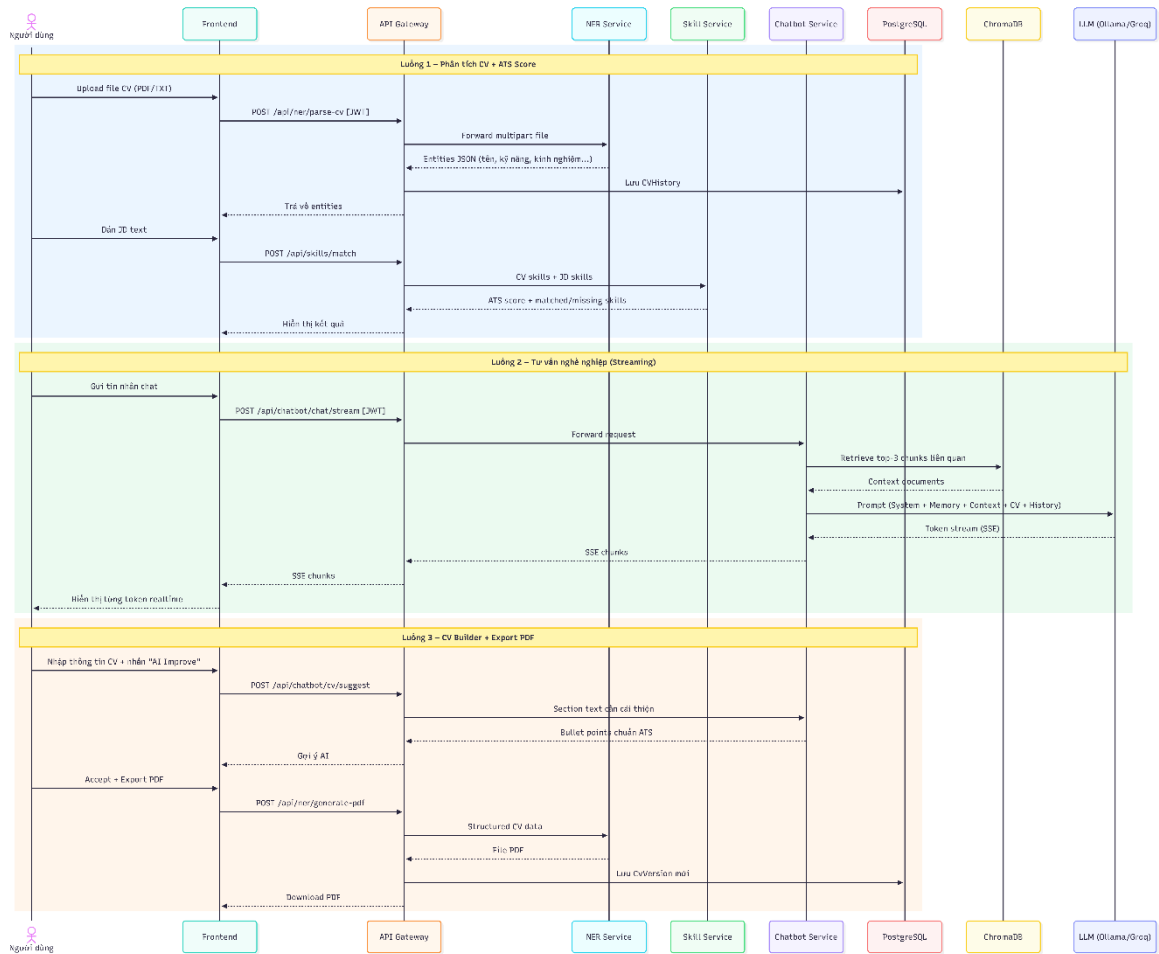
2	API Gateway	ASP.NET Core 9, C#	8081	JWT Auth, routing, quản lý user, CV versioning
3	NER Service	FastAPI, Python 3.10	5001	Trích xuất thực thể CV/JD bằng mBERT [33]
4	Skill Service	FastAPI, Python 3.10	5002	Skill matching, ATS score, phân tích thị trường
5	Career Service	FastAPI, Python 3.10	5003	Lộ trình nghề nghiệp, gợi ý bước tiếp theo
6	Chatbot Service	FastAPI, LlamaIndex	5004	RAG chatbot [15] , sinh và tối ưu CV
7	PostgreSQL	PostgreSQL 15	5432	Users, CV history, sessions, feedback
8	ChromaDB	ChromaDB 0.4	8000	Vector DB [26] cho RAG và lịch sử hội thoại
9	Ollama	Ollama + Qwen2.5:3b	11434	Serve LLM local [27] (chế độ offline)

Bảng 3.2: Bảng stack các công nghệ tổng hợp

Lớp	Công nghệ
Frontend	React 18, TypeScript, Tailwind CSS
API Gateway	ASP.NET Core 9, C#, .NET 9 [30] , EF Core, Npgsql, BCrypt
AI Services	Python 3.10, FastAPI, HuggingFace Transformers [19]
NLP Models	mBERT BertForTokenClassification (NER) [33] , Sentence-BERT all-MiniLM-L6-v2 (Skill Matching) [34]
LLM	Qwen2.5:3b via Ollama [27] (local), Llama-3.3-70b via Groq Cloud [28]
RAG Framework	LlamaIndex [25] , ChromaDB 0.4 [26]
Database	PostgreSQL 15 (relational), ChromaDB (vector)
DevOps	Docker, Docker Compose
Ngôn ngữ phát triển	Python, C# (.NET), TypeScript

3.1.2 Luồng dữ liệu tổng thể

Hệ thống hỗ trợ ba luồng sử dụng chính, mỗi luồng tương ứng với một use case cụ thể của người dùng.



Hình 3.2: Sơ đồ luồng dữ liệu tổng thể

Luồng 1 - Phân tích CV và tính ATS Score là luồng cốt lõi nhất: người dùng upload file CV lên Frontend, request được gửi qua API Gateway (xác thực JWT), API Gateway forward file multipart đến NER Service. NER Service load file, trích xuất text, chạy mBERT pipeline để nhận diện các thực thể (tên, kỹ năng, kinh nghiệm, học vấn...) rồi trả về danh sách entities có cấu trúc. API Gateway lưu kết quả vào CVHistory (PostgreSQL) và trả về Frontend để hiển thị. Nếu người dùng cung cấp thêm JD, Skill Service được gọi tiếp theo để thực hiện matching và tính ATS score.

Luồng 2 - Tư vấn nghề nghiệp qua Chatbot hoạt động theo mô hình streaming: người dùng gửi tin nhắn từ Frontend, API Gateway proxy request đến Chatbot Service. Chatbot Service thực hiện retrieval từ ChromaDB để lấy context liên quan, kết hợp với thông tin CV/JD của user từ User Memory, rồi gửi prompt tổng hợp cho LLM

(Qwen2.5:3b qua Ollama hoặc Llama-3.3-70b qua Groq [28]). LLM sinh response dạng stream (Server-Sent Events), được pipe qua Chatbot Service => API Gateway => Frontend và hiển thị từng token một giống ChatGPT.

Luồng 3 - CV Builder là luồng phức tạp nhất với nhiều bước: người dùng điền thông tin CV qua form, từng phần có thể được AI cải thiện qua Chatbot Service. Khi xuất PDF, dữ liệu CV được gửi đến NER Service để generate file PDF chuẩn ATS. API Gateway lưu mỗi lần chỉnh sửa như một version mới trong CvDocument versioning system (PostgreSQL), cho phép người dùng xem lịch sử và khôi phục bất kỳ version nào.

3.2 Dữ liệu và pipeline huấn luyện NER

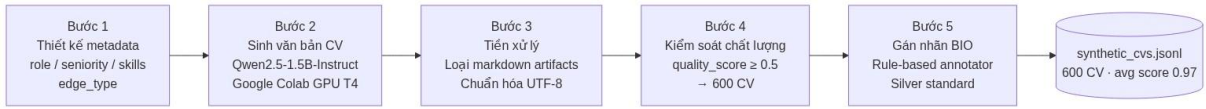
3.2.1 Tổng quan bài toán dữ liệu

Huấn luyện mô hình NER đòi hỏi một lượng đáng kể văn bản đã được gán nhãn BIO ở cấp độ token. Đối với bài toán phân tích CV và JD trong ngành CNTT, thách thức không chỉ nằm ở khối lượng dữ liệu mà còn ở chỗ không tồn tại bộ dữ liệu CV tiếng Việt có nhãn NER chi tiết nào được công bố công khai. Trong khi các bộ dữ liệu quốc tế như CoNLL-2003 hay bộ dữ liệu Resume-NER của Lim et al. [4] phục vụ cho tiếng Anh, thị trường Việt Nam thiếu hoàn toàn nguồn dữ liệu tương đương.

Phương án thu thập CV thực tế từ các nền tảng như LinkedIn, TopCV, hay ITviec gặp hai rào cản nghiêm trọng không thể vượt qua. Thứ nhất, CV chứa thông tin cá nhân nhạy cảm bao gồm họ tên, địa chỉ, số điện thoại và email - việc thu thập và sử dụng mà không có sự đồng ý của chủ thể vi phạm Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân [31] và vi phạm điều khoản sử dụng của các nền tảng nói trên. Thứ hai, chi phí gán nhãn NER ở cấp độ token cho một nghìn CV với mười loại thực thể đòi hỏi nhiều giờ công của annotator có chuyên môn, ước tính lên đến hàng chục triệu đồng - vượt xa nguồn lực của một đề tài nghiên cứu sinh viên.

Giải pháp được chọn là sinh dữ liệu CV synthetic có kiểm soát bằng LLM, kết hợp với quy trình tự động gán nhãn BIO theo phương pháp silver standard. Phương pháp này giải quyết đồng thời cả ba vấn đề: tuân thủ pháp lý vì không sử dụng dữ liệu cá nhân thật, chi phí tính toán thấp hơn nhiều so với gán nhãn thủ công, và phân phối dữ liệu có thể được kiểm soát chủ động để đảm bảo cân bằng về vai trò nghề nghiệp và cấp độ kinh nghiệm.

3.2.2 Pipeline sinh dữ liệu CV Synthetic



Hình 3.3: Pipeline sinh dữ liệu CV

Pipeline sinh dữ liệu được thực thi trên Google Colab với GPU T4 miễn phí, sử dụng mô hình Qwen2.5-1.5B-Instruct [35] làm engine sinh văn bản. Việc lựa chọn Qwen2.5-1.5B thay vì các LLM lớn hơn như GPT-4 hay Llama-3.3-70b xuất phát từ hai lý do thực tế: mô hình này hoàn toàn miễn phí để chạy trên Colab GPU và đủ năng lực sinh văn bản CV chất lượng tốt khi được cung cấp prompt có cấu trúc rõ ràng.

Bước đầu tiên của pipeline là thiết kế schema kiểm soát, trong đó mỗi CV được mô tả bởi một metadata record JSON với các trường định nghĩa rõ ràng. Trường role xác định vai trò nghề nghiệp seniority phân loại cấp độ (junior, mid, senior, fresher), years ghi số năm kinh nghiệm, skills liệt kê danh sách công nghệ cần xuất hiện trong CV, và company chỉ định tên công ty làm việc gần nhất. Đặc biệt, trường edge_type kiểm soát đa dạng hóa phong cách CV: giá trị normal tạo CV tiêu chuẩn đầy đủ các section, short tạo CV ngắn dưới 200 từ phù hợp với fresher hoặc CV tóm tắt, verbose tạo CV dài trên 500 từ cho ứng viên nhiều kinh nghiệm, và mixed_vi_en tạo CV xen lẫn tiếng Việt và thuật ngữ kỹ thuật tiếng Anh - phản ánh thực tế đa dạng của CV ngành CNTT tại Việt Nam nơi lập trình viên thường viết CV theo phong cách pha trộn ngôn ngữ.

Bước thứ hai là sinh nội dung văn bản từ metadata đầu vào. Từ mỗi metadata record, hệ thống tạo ra một prompt có cấu trúc hướng dẫn Qwen2.5-1.5B-Instruct viết CV theo định dạng plain text với các section SUMMARY, EXPERIENCE, EDUCATION, và SKILLS. Thay vì sử dụng template cố định với nội dung tĩnh, LLM được tự do quyết định phong cách diễn đạt, cách sắp xếp nội dung, và từ ngữ sử dụng trong mỗi CV - tạo ra sự đa dạng tự nhiên không thể đạt được bằng template. Mỗi CV được sinh trong khoảng 3–5 giây trên Colab GPU T4.

Bước thứ ba là tiền xử lý để tạo trường text_clean từ văn bản thô. Quá trình này loại bỏ các markdown artifacts còn sót lại từ quá trình sinh (ký tự **, ##, ---), chuẩn hóa khoảng trắng và ký tự xuống dòng thừa, đồng thời xử lý encoding UTF-8 đặc biệt quan trọng với tên người Việt có dấu. Trường text_clean được lưu song song với trường text gốc để phục vụ cho cả NER training và debugging.

Bước thứ tư là kiểm soát chất lượng tự động thông qua chỉ số `quality_score`. Mỗi CV được chấm điểm tổng hợp từ ba tiêu chí: độ dài văn bản nằm trong khoảng hợp lệ từ 100 đến 1000 từ được cộng 0.3 điểm, sự hiện diện đầy đủ của các section EXPERIENCE, SKILLS, và EDUCATION được cộng 0.4 điểm, và tỷ lệ kỹ năng trong metadata xuất hiện thực tế trong văn bản đạt từ 60% trở lên được cộng 0.3 điểm. Tổng `quality_score` dao động từ 0 đến 1.0; những CV có điểm dưới 0.5 bị loại khỏi bộ dữ liệu. Kết quả cuối cùng là 600 CV synthetic chất lượng cao với `quality_score` trung bình đạt 0.97 - con số này phản ánh Qwen2.5-1.5B-Instruct sinh ra hầu hết CV đáp ứng đầy đủ các tiêu chí chất lượng khi được cung cấp metadata đầu vào có cấu trúc rõ ràng.

3.2.3 Thống kê bộ dữ liệu

Bộ dữ liệu 600 CV được phân bố đều qua mười nhóm nghề nghiệp, mỗi nhóm gồm 60 CV với phân bố cấp độ seniority được thiết kế để phản ánh thực tế thị trường. Chi tiết phân bố được trình bày trong Bảng 2.2.

Bảng 3.3: Thống kê tập dữ liệu CV synthetic

Vai trò	Số CV	Quality Score TB
Backend Developer	61	0.997
Full-stack Developer	61	0.997
Data Analyst	61	0.997
Frontend Developer	60	0.997
Data Scientist	60	0.997
DevOps Engineer	60	0.997
AI Engineer	59	0.997
Mobile Developer	59	0.997
Project Manager	59	0.997
QA Engineer	60	0.997
Tổng cộng	600	0.996

Phân bố seniority thực tế trong bộ dữ liệu: Junior 216 CV (36%), Mid 235 CV (39%), Senior 113 CV (19%), Lead 25 CV (4%), Principal 11 CV (2%). Phân bố nghiêng về mid-level phản ánh thực tế thị trường tuyển dụng CNTT Việt Nam, nơi nhóm này chiếm đa số trong các tin đăng tuyển. `quality_score` trung bình 0.996 cho thấy Qwen2.5-1.5B-Instruct [35] sinh CV đạt chất lượng cao khi được cung cấp metadata có cấu trúc.

3.2.4 Pipeline gán nhãn BIO

Sau khi xây dựng bộ 600 CV synthetic, bước tiếp theo là gán nhãn BIO ở cấp độ token để tạo dữ liệu huấn luyện NER. Do không có annotator thủ công, đề tài sử dụng phương pháp silver standard, tức gán nhãn tự động bằng rule-based system và chấp nhận một tỷ lệ sai số nhất định.

Quy trình bắt đầu bằng việc tách text_clean thành các token theo khoảng trắng và lưu character offset. Mỗi loại thực thể được xử lý bằng rule riêng: PER khớp với trường name và regex tên người Việt/Anh; SKILL đối chiếu với metadata và ontology ~500 kỹ năng (bao gồm alias); ORG khớp với company và danh sách công ty IT; DATE nhận diện qua regex năm (2018–2026) và khoảng thời gian “Tháng Năm – Tháng Năm”; các thực thể khác như JOB_TITLE, DEGREE, MAJOR, LOC, CERT được gán nhãn theo danh sách từ khóa.

Pipeline hỗ trợ context-aware matching, ví dụ MAJOR chỉ được gán trong section EDUCATION (xác định qua SmartCVParser), giúp giảm nhầm lẫn ngữ cảnh. Sau đó, các entity span được chuyển đổi thành chuỗi nhãn BIO tương ứng với từng token.

Một hạn chế quan trọng là sự không tương thích giữa whitespace tokenization (trong gán nhãn) và WordPiece subword tokenization của mBERT khi suy luận. Do đó, khi đánh giá trên 200 mẫu, F1 chỉ đạt 0.03; kết quả này không phản ánh đúng hiệu suất mô hình mà chủ yếu do tokenization mismatch.

3.2.5 Cấu trúc dữ liệu lưu trữ

Toàn bộ bộ dữ liệu được lưu tại data/synthetic_cvs.jsonl (2.7 MB, 600 dòng) theo định dạng JSON Lines - mỗi dòng là một JSON object đại diện cho một CV với đầy đủ metadata, văn bản thô và văn bản đã làm sạch. Định dạng này cho phép đọc streaming từng CV mà không cần load toàn bộ file vào RAM - quan trọng khi xử lý batch cho 600 CV trong bộ nhớ giới hạn của Google Colab.

Phân tích phân bố nhãn thực thể trên 200 mẫu đầu tiên của bộ dữ liệu được trình bày trong Bảng 2.3. Kết quả cho thấy SKILL là loại thực thể chiếm tỷ lệ áp đảo với 3.799 instances, tương đương 58.8% tổng số thực thể - phản ánh đặc thù của CV ngành CNTT nơi phần Skills thường liệt kê danh sách dài các công nghệ. Đây cũng chính là loại thực thể thách thức nhất cho mô hình do sự đa dạng cực lớn trong cách viết tên công nghệ (ReactJS / React.js / React / react), sự xuất hiện của các từ viết tắt (k8s cho

Kubernetes, EKS cho Elastic Kubernetes Service), và vấn đề subword tokenization khiến những tên công nghệ có dấu chấm hoặc số bị tách thành nhiều subword.

Bảng 3.4: Phân bố nhãn thực thể trong tập dữ liệu (200 mẫu silver labels)

Loại thực thể	Số lượng	Tỷ lệ
SKILL	3.799	58.8%
DATE	693	10.7%
JOB_TITLE	560	8.7%
ORG	552	8.5%
LOC	300	4.6%
MAJOR	242	3.7%
DEGREE	168	2.6%
PER	116	1.8%
CERT	35	0.5%
Tổng	6.465	100%

Sự mất cân bằng nhãn rõ nét trong bộ dữ liệu - SKILL chiếm gần 59% trong khi CERT chỉ chiếm 0.5% - là một thực tế phản ánh đúng bản chất của CV ngành CNTT chứ không phải artefact của quá trình sinh dữ liệu. Trong training, mất cân bằng này được xử lý một phần bởi cross-entropy loss tiêu chuẩn, nhưng các thực thể hiếm như CERT và DEGREE có thể sẽ có F1 thấp hơn do ít dữ liệu huấn luyện hơn - điều này được xác nhận qua kết quả đánh giá định lượng ở Chương 4.

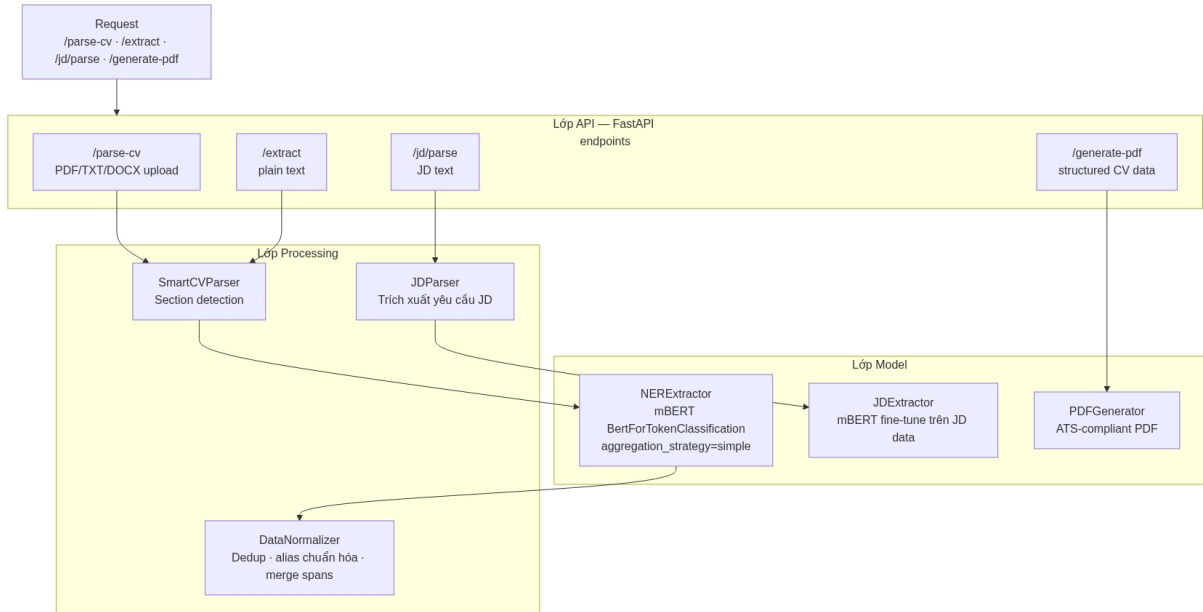
3.3 NER Service - Thiết kế và huấn luyện

3.3.1 Tổng quan và kiến trúc NER Service

NER Service là thành phần trung tâm trong pipeline phân tích, chịu trách nhiệm trích xuất các thực thể có cấu trúc từ văn bản CV và JD. Service được xây dựng bằng FastAPI (Python 3.10), chạy tại cổng 5001 và được API Gateway truy cập thông qua NerProxyController. Việc lựa chọn FastAPI thay vì Flask hay Django nhằm tận dụng khả năng async IO, tự động sinh OpenAPI documentation, tích hợp Pydantic cho validation và hỗ trợ async inference, cho phép xử lý nhiều request CV song song khi mô hình có độ trễ cao.

Service cung cấp nhiều endpoint phục vụ các mục đích khác nhau. Cụ thể, /parse-cv nhận file (PDF/TXT/DOCX) và trả về kết quả có cấu trúc theo từng section; /extract

xử lý plain text và trả về danh sách thực thể; /batch/extract hỗ trợ xử lý nhiều văn bản cùng lúc; /jd/parse áp dụng cho JD với mô hình riêng; và /generate-pdf xuất CV dạng PDF chuẩn ATS từ dữ liệu có cấu trúc. Việc phân tách các endpoint này phản ánh các kịch bản sử dụng thực tế như xử lý CV người dùng, gọi API giữa các service và xuất kết quả cuối cùng.



Hình 3.4: Kiến trúc nội bộ NER Service

Bên trong, NER Service được tổ chức thành ba lớp chức năng rõ ràng. Lớp model (services/extractor.py) chứa NERExtractor - class cốt lõi quản lý vòng đời của mBERT model và thực hiện inference. Lớp processing (services/normalizer.py, services/jd_parser.py) thực hiện post-processing: chuẩn hóa entities, gộp thực thể trùng, phân tích cấu trúc JD. Lớp output (services/pdf_generator.py) tạo file PDF từ structured CV data. Ngoài ra, shared/utils/cv_parser.py chứa SmartCVParser - một component chia sẻ giữa các service - thực hiện phân tích cấu trúc section của CV.

Một điểm thiết kế quan trọng là singleton pattern với lazy initialization cho model: _extractor global variable được khởi tạo chỉ khi có request đầu tiên, tránh load model (vài trăm MB) khi service khởi động và chỉ tốn thời gian load một lần duy nhất trong suốt vòng đời service. Tương tự, hệ thống có hai extractor riêng biệt: _extractor cho CV NER model và _jd_extractor cho JD NER model - hai model có cùng kiến trúc nhưng được fine-tune trên phân phối dữ liệu khác nhau, phản ánh sự khác biệt đáng kể về văn phong giữa CV (viết từ góc nhìn ứng viên, thường dùng bullet points, ngôn ngữ

thành tích) và JD (viết từ góc nhìn nhà tuyển dụng, thường dùng danh sách yêu cầu, ngôn ngữ hướng dẫn).

3.3.2 Xử lý suy luận và pipeline dữ liệu

NERExtractor sử dụng HuggingFace Transformers pipeline với `aggregation_strategy="simple"` nhằm tự động xử lý bài toán subword tokenization. Thay vì dự đoán nhãn cho từng subword riêng lẻ (ví dụ: “Re”, “##act”, “.”, “##js”), pipeline sẽ hợp nhất các subword liên tiếp có cùng loại thực thể thành một span duy nhất, trả về loại thực thể, nội dung văn bản, vị trí ký tự và độ tin cậy. Cách tiếp cận này giúp đơn giản hóa đáng kể việc xử lý subword chỉ với một cấu hình.

Sau khi thu được danh sách thực thể ban đầu, hệ thống thực hiện hậu xử lý để loại bỏ các thực thể không hợp lệ (ví dụ quá ngắn), chuẩn hóa độ tin cậy và vị trí ký tự, và trả về danh sách Entity hoàn chỉnh. Trong trường hợp mô hình không khả dụng, service chuyển sang chế độ mock bằng keyword matching, ví dụ “Python” → SKILL, “Google” → ORG, “Data Analyst” → JOB_TITLE, đảm bảo hệ thống vẫn hoạt động trong môi trường phát triển.

SmartCVParser được sử dụng để phân tích cấu trúc CV, nhận diện các section dựa trên từ khóa tiêu đề như “EXPERIENCE” hoặc “WORK HISTORY”. Thông tin này giúp cải thiện độ chính xác của NER, ví dụ thực thể MAJOR chỉ hợp lệ trong section EDUCATION, đồng thời cho phép tổ chức kết quả theo cấu trúc có ngữ nghĩa.

Cuối cùng, DataNormalizer thực hiện các bước làm sạch như loại bỏ trùng lặp, chuẩn hóa tên kỹ năng (ví dụ “ReactJS” → “React.js”, “k8s” → “Kubernetes”) và hợp nhất các thực thể liên kề bị tách sai (ví dụ “Ho” + “Chi” + “Minh” → “Ho Chi Minh City”), giúp nâng cao chất lượng đầu ra.

3.3.3 Cấu hình và huấn luyện mô hình

Mô hình CV NER gồm kiến trúc BertForTokenClassification, 21 nhãn phân loại, `vocab_size = 119.547` (mBERT), `hidden_size = 768`, 12 lớp Transformer encoder và 12 attention heads. Tổng số tham số bao gồm khoảng 178 triệu tham số của mBERT backbone và 16.128 tham số của lớp classification head. Mô hình JD NER có cùng kiến trúc nhưng được fine-tune trên dữ liệu JD (100 mẫu), dẫn đến khác biệt về trọng số.

Quá trình huấn luyện được thực hiện trên Google Colab với GPU T4. Dữ liệu gồm 600 CV được gán nhãn tự động bằng rule-based annotator (sử dụng dictionary kỹ

năng, regex ngày tháng, danh sách trường học), sau đó chia theo tỷ lệ 80/10/10 thành train (480 CV), validation và test (60 CV). Tập validation được sử dụng để chọn checkpoint tốt nhất.

Trong bước tokenization, nhãn ở mức từ được căn chỉnh với subword của mBERT: subword đầu tiên giữ nhãn gốc, các subword tiếp theo (có tiền tố “##”) được gán giá trị -100 để bỏ qua khi tính loss. Đây là phương pháp tiêu chuẩn cho bài toán token classification.

Cấu hình huấn luyện gồm AdamW với learning rate $2e-5$, weight decay 0.01, linear warmup 10% và linear decay; batch size 16 kết hợp gradient accumulation 2 bước (effective batch size 32); huấn luyện 5 epochs với early stopping theo Val F1; dropout 0.1. Checkpoint tốt nhất được lưu về máy cục bộ sau khi huấn luyện.

Một hạn chế thực tế là toàn bộ training log chi tiết (loss, F1 theo epoch) không được lưu do Colab session không mount Google Drive. Chỉ có model cuối cùng và một phần log được giữ lại, do đó việc đánh giá chủ yếu dựa trên quan sát định tính.

3.4 Skill Matching Service - Thiết kế và triển khai

3.4.1 Tổng quan Skill Service

Skill Service là service chịu trách nhiệm về toàn bộ lớp phân tích kỹ năng - từ so khớp kỹ năng CV với JD, tính điểm ATS, đến phân tích thị trường việc làm và gợi ý lộ trình học tập. Service chạy trên FastAPI (Python 3.10) tại cổng :5002 và được API Gateway proxy qua SkillProxyController. Ngoài các chức năng AI, Skill Service còn có một isolated PostgreSQL schema riêng (khởi tạo qua `init_db()`) để lưu trạng thái học tập và bookmark kỹ năng của từng user - phản ánh tính chất stateful của learning roadmap feature.

Skill Service được thiết kế như một engine phân tích độc lập, có thể được gọi bởi cả Frontend (qua API Gateway) lẫn các service nội bộ như Chatbot Service (để lấy context CV/JD của user khi sinh response). Kiến trúc này cho phép chatbot trả lời câu hỏi “Tôi đang thiếu kỹ năng gì?” với thông tin cụ thể về gap analysis của user, thay vì chỉ trả lời chung chung.

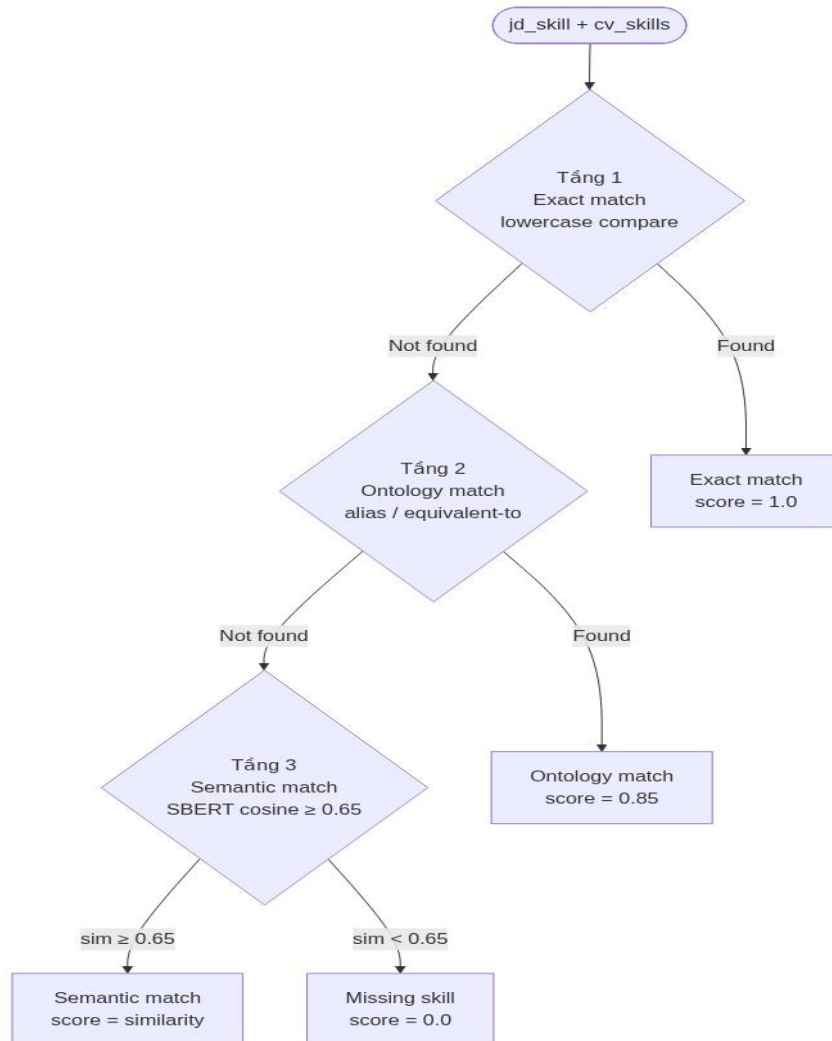
3.4.2 SkillOntology - Nền tảng tri thức kỹ năng

SkillOntology là class quản lý ontology kỹ năng IT, được định nghĩa hardcoded trong file `services/skill_service/services/ontology.py` (474 entries, 1112 dòng). Ontology được tổ chức phân cấp theo domain (frontend, backend, mobile, devops, data, ai), trong

mỗi domain là các nhóm công nghệ, và trong mỗi nhóm là các entry với đầy đủ metadata: aliases (các cách viết tương đương), related (kỹ năng liên quan trong cùng ecosystem), và requires (kỹ năng prerequisite). Cấu trúc JSON phân cấp này cho phép query nhanh theo nhiều chiều: tìm tất cả aliases của một kỹ năng, tìm các kỹ năng trong cùng category, hay kiểm tra hai kỹ năng có tương đương không.

Ontology kỹ năng IT được xây dựng dựa trên ba nguồn chính: tái sử dụng một phần ESCO [23] và O*NET [24] cho các kỹ năng universal (SQL, Python, Java), bổ sung các công nghệ đặc thù thị trường Việt Nam (Laravel, Flutter, .NET Core, Spring Boot), và cập nhật theo dữ liệu tuyển dụng thực tế từ ITviec và TopCV. Với khoảng 500 entries, ontology cover được hầu hết các kỹ năng xuất hiện trong CV/JD ngành CNTT tại Việt Nam hiện nay.

3.4.3 SkillMatcher - Logic so khớp 3 tầng



Hình 3.5: Logic so khớp 3 tầng

SkillMatcher là class trung tâm của Skill Service, triển khai logic matching 3 tầng theo thứ tự từ chính xác nhất đến tổng quát nhất. Đây là cách tiếp cận “cascade” - chỉ khi tầng cao hơn (chính xác hơn) không tìm được match mới thử tầng thấp hơn (tổng quát hơn), đảm bảo kết quả cuối có precision cao nhất có thể.

Ở tầng 1 (exact match), tất cả kỹ năng được chuẩn hóa lowercase trước khi so sánh chuỗi. Nếu `jd_skill.lower()` tìm thấy trong `cv_skills_lower`, đây là exact match với `score = 1.0`. Ở tầng 2 (ontology match), `SkillOntology.find_match()` kiểm tra xem `jd_skill` và bất kỳ kỹ năng nào trong CV có quan hệ `equivalent-to` hay `alias` với nhau không. Nếu có, đây là ontology match với `score = 0.85`. Ở tầng 3 (semantic match), cả `jd_skill` lẫn từng kỹ năng trong CV được encode thành embedding vector bởi Sentence-BERT model `all-MiniLM-L6-v2` [13] [34], và cosine similarity được tính giữa `jd_skill` embedding và từng CV skill embedding. Kỹ năng CV có similarity cao nhất được chọn; nếu `similarity ≥ threshold  $\tau = 0.65$` , đây là semantic match với score bằng giá trị similarity đó (0.65–1.0).

Threshold $\tau = 0.65$ được chọn dựa trên thực nghiệm với các cặp kỹ năng mẫu: “Vue.js” và “React” có `sim ≈ 0.72` (cùng frontend framework ecosystem, nên match), “TensorFlow” và “PyTorch” có `sim ≈ 0.79` (cùng deep learning framework, nên match), “Python” và “Java” có `sim ≈ 0.58` (đều là programming language nhưng khác nhau cơ bản, không nên match). Ngưỡng 0.65 phân biệt chính xác hai nhóm này trong các thực nghiệm kiểm tra.

Kết quả cuối cùng của `match()` là dictionary gồm danh sách `matched` (các kỹ năng JD đã được cover với loại match và score), danh sách `missing` (các kỹ năng JD chưa được cover với gap score), danh sách `extra` (kỹ năng CV có nhưng JD không yêu cầu - “điểm cộng”), và `ats_score` là tỷ lệ phần trăm kỹ năng JD được cover.

3.4.4 ATSScoringEngine - Đánh giá CV toàn diện 8 tiêu chí

ATSScoringEngine vượt ra ngoài skill matching đơn thuần để đánh giá CV theo 8 tiêu chí phản ánh cách các hệ thống ATS thương mại thực tế vận hành. Tiêu chí `Keywords Match` (25%) đo tỷ lệ kỹ năng JD được cover theo ba tầng matching. Tiêu chí `Format & Structure` (15%) kiểm tra CV có đủ các section quan trọng, có dùng bullet points cho kinh nghiệm, và độ dài hợp lý (400–1200 từ). Tiêu chí `Contact Info` (10%) xác nhận CV có đầy đủ email, số điện thoại, LinkedIn và GitHub. Tiêu chí `Experience`

Detail (15%) kiểm tra mỗi mục kinh nghiệm có chứa số liệu định lượng (percentages, dollar amounts, user counts) hay không - yếu tố phân biệt CV trung bình và CV chuyên nghiệp. Tiêu chí Action Verbs (10%) đếm tỷ lệ bullet points bắt đầu bằng động từ hành động mạnh từ danh sách 80+ verbs (developed, led, optimized, spearheaded...). Tiêu chí Education Match (10%) kiểm tra bằng cấp có đáp ứng yêu cầu JD. Tiêu chí Readability (10%) đánh giá độ dài câu trung bình và phát hiện keyword stuffing. Tiêu chí Certifications (5%) kiểm tra CV có chứng chỉ liên quan đến domain của JD.

Kèm theo điểm số tổng là danh sách issues - các vấn đề cụ thể được diễn đạt bằng ngôn ngữ tự nhiên và được ưu tiên theo impact, ví dụ “Thiếu từ khóa quan trọng: Docker, Kubernetes (ảnh hưởng 25% điểm)”, “Không có số liệu định lượng trong phần Experience - thêm metrics cụ thể để tăng điểm”, hay “GitHub profile chưa được đề cập - bổ sung để tăng tín nhiệm”. Danh sách issues này là phần hữu ích nhất với người dùng vì nó chỉ ra đúng những gì cần làm tiếp theo.

3.4.5 Tích hợp O*NET và Market Analysis

Skill Service sử dụng dữ liệu O*NET [24] từ knowledge_base/onet/db_28_1_text/ được index vào ChromaDB [26] collection COLLECTION_JOBS để hỗ trợ hai chức năng bổ sung. Thứ nhất, khi JD text được cung cấp nhưng danh sách kỹ năng không được tách biệt rõ ràng, Skill Service có thể tự động trích xuất kỹ năng bằng cách tìm kiếm O*NET occupation embeddings gần với JD text nhất và lấy required skills của những occupation đó. Thứ hai, MarketAnalyzer truy vấn O*NET data để phân tích xu hướng kỹ năng theo industry domain, cung cấp thông tin thị trường như “Top 10 kỹ năng được yêu cầu nhiều nhất trong tuần qua” hay “Lương trung bình cho Backend Developer tại Việt Nam”.

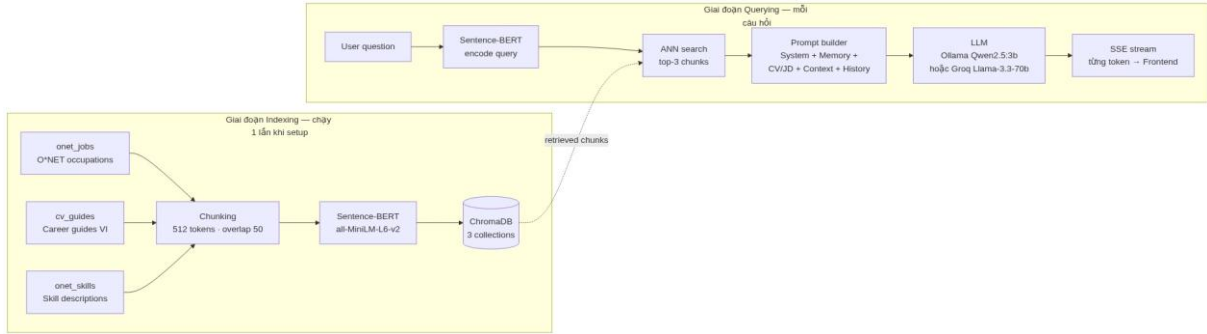
3.5 Chatbot Service - RAG Pipeline và tư vấn nghề nghiệp

3.5.1 Tổng quan Chatbot Service

Chatbot Service là thành phần phức tạp và sáng tạo nhất trong hệ thống, tích hợp kiến trúc RAG [15] với LLM để cung cấp tư vấn nghề nghiệp cá nhân hóa thực sự. Service chạy trên FastAPI tại cổng :5004 với một đặc điểm kỹ thuật quan trọng phân biệt nó với các service còn lại: hỗ trợ **Server-Sent Events (SSE)** để streaming response từng token về Frontend - tạo trải nghiệm người dùng giống hệt ChatGPT với chữ xuất hiện dần dần thay vì chờ toàn bộ response.

Ngoài chức năng chatbot tư vấn cốt lõi, Chatbot Service còn đảm nhận các chức năng liên quan đến tạo nội dung CV: viết lại từng section CV theo chuẩn ATS (/cv/rewrite), sinh CV hoàn chỉnh từ thông tin đầu vào (/cv/generate), tối ưu toàn bộ CV theo một JD cụ thể (/cv/optimize), gợi ý bullet points chuyên nghiệp cho từng vị trí kinh nghiệm (/cv/suggest). Tất cả những chức năng này đều sử dụng cùng một LLM backend, chỉ khác nhau ở prompt template và cách tổ chức input/output.

3.5.2 Kiến trúc RAG pipeline chi tiết



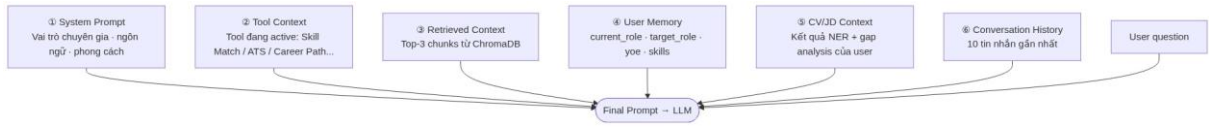
Hình 3.6: Kiến trúc RAG chi tiết

Knowledge base của Chatbot Service được xây dựng từ nhiều nguồn và lưu trữ trong ChromaDB [26] (thư mục `knowledge_base/chatbot_chroma/`) dưới ba collection chính. Collection `onet_jobs` chứa dữ liệu từ O*NET về yêu cầu kỹ năng, kinh nghiệm và đặc điểm công việc cho nhiều vị trí IT. Collection `cv_guides` bao gồm các hướng dẫn nghề nghiệp tự xây dựng cho 10 vai trò IT tại Việt Nam, với lộ trình kỹ năng, chứng chỉ và mức lương theo seniority. Collection `onet_skills` cung cấp mô tả chi tiết từng kỹ năng, phục vụ các câu hỏi như “Kỹ năng X là gì và học ở đâu?”.

Trong giai đoạn indexing, tài liệu được chia thành các đoạn 512 tokens với overlap 50 tokens để giữ ngữ cảnh, sau đó được mã hóa thành embedding bằng Sentence-BERT [13] và lưu vào ChromaDB kèm metadata (source, document_type, role...). Quá trình này chỉ thực hiện một lần khi khởi tạo hệ thống.

Khi người dùng gửi truy vấn, ChatService thực hiện retrieval gồm ba bước: mã hóa câu hỏi thành embedding, tìm kiếm các đoạn gần nhất bằng approximate nearest neighbor, và re-ranking theo độ liên quan. Thông thường, top-3 kết quả bao gồm một đoạn từ `cv_guides`, một từ `onet_jobs` và một từ `onet_skills`. Sự kết hợp này cung cấp ngữ cảnh đầy đủ để mô hình ngôn ngữ trả lời chính xác và có cơ sở.

3.5.3 Xây dựng prompt và cơ chế cá nhân hóa



Hình 3.7: Xây dựng prompt với cơ chế cá nhân hóa

Điểm phân biệt lớn nhất của hệ thống so với chatbot RAG thông thường là cơ chế cá nhân hóa sâu thông qua việc tổng hợp nhiều nguồn context khác nhau vào một prompt duy nhất. Prompt cuối cùng gửi cho LLM có cấu trúc phân lớp rõ ràng.

Lớp đầu tiên là System Prompt định nghĩa vai trò của assistant là chuyên gia tư vấn nghề nghiệp IT, phong cách giao tiếp và ngôn ngữ (tiếng Việt hoặc tiếng Anh tùy preference của user). Lớp thứ hai là Tool Context Prompt - đây là tính năng đặc biệt của hệ thống: chatbot biết tool nào đang active trên panel phải của Frontend (Skill Match, Career Path, CV Upload, ATS Score, JD Analysis, Skill Graph, hay Market Dashboard) và điều chỉnh hành vi tương ứng. Ví dụ khi Skill Match tool đang active, chatbot chủ động hỏi về vị trí target và tech stack của user thay vì chờ user hỏi - tạo trải nghiệm proactive assistant thay vì passive Q&A bot. Lớp thứ ba là Retrieved Context gồm các document chunks từ ChromaDB liên quan đến câu hỏi. Lớp thứ tư là User Memory - thông tin đã lưu về user từ các session trước (current role, target role, skills, số năm kinh nghiệm). Lớp thứ năm là CV/JD Context - tóm tắt kết quả phân tích CV và JD hiện tại của user, nếu có. Lớp cuối cùng là Conversation History - 10 tin nhắn gần nhất để đảm bảo tính nhất quán của hội thoại.

Tổng hợp tất cả các lớp này, mỗi câu trả lời của chatbot không chỉ dựa trên knowledge base chung mà còn được điều chỉnh phù hợp với tình trạng cụ thể của từng user: một fresher Backend Developer với 0 năm kinh nghiệm sẽ nhận được tư vấn hoàn toàn khác so với một senior developer 5 năm kinh nghiệm cùng hỏi câu hỏi “Tôi nên học gì tiếp theo?”.

3.5.4 Dual LLM Backend: Ollama và Groq

ChatService hỗ trợ chuyển đổi giữa hai LLM backend thông qua dependency injection được cấu hình từ environment variable `CHAT_USE_GROQ`. Khi `use_groq=False` (mặc định), hệ thống khởi tạo `OllamaClient` pointing đến local Ollama server tại `http://localhost:11434` với model `qwen2.5:3b` [29] [27]. Khi `use_groq=True`,

hệ thống khởi tạo GroqClient với API key từ environment variable, sử dụng model llama-3.3-70b-versatile trên Groq Cloud [28].

Cả hai client đều implement cùng interface với method chat_stream() trả về iterator của string chunks, cho phép phần còn lại của code hoàn toàn trong suốt với backend LLM đang dùng. Chatbot Service stream response bằng cách yield từng chunk dưới dạng SSE events data: {"content": "..."}, API Gateway forward stream này về Frontend, và JavaScript trên Frontend parse từng event và append vào UI - chuỗi 3 lớp streaming này tạo ra trải nghiệm real-time mượt mà.

3.5.5 User Memory Service

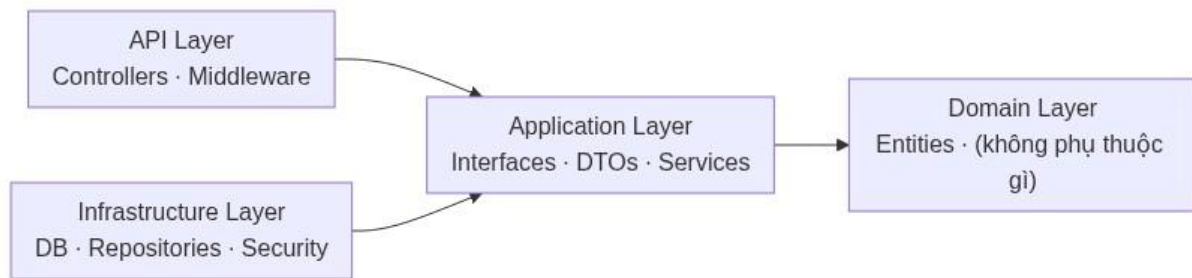
MemoryService lưu trữ thông tin cá nhân hóa của từng user tại data/user_memory/{user_id}.json. Mỗi khi chatbot thu thập được thông tin mới về user qua hội thoại (ví dụ user đề cập “Tôi đang làm Backend Developer được 3 năm”), memory service tự động update profile. Khi session mới bắt đầu, memory được load và inject vào prompt. Cơ chế này tạo ra “bộ nhớ dài hạn” cho chatbot - không phải bộ nhớ phiên (session memory dựa trên conversation history) mà là bộ nhớ về con người và tình huống nghề nghiệp của user, tồn tại qua nhiều session khác nhau.

3.6 API Gateway (.NET 9) và Frontend React 18

3.6.1 Kiến trúc hệ thống và xử lý backend

API Gateway là điểm vào duy nhất của toàn bộ hệ thống, được xây dựng bằng ASP.NET Core 9 (.NET 9 [30], C#) theo kiến trúc Clean Architecture phân tầng rõ ràng. Lắng nghe tại cổng :8081.

Lý do chọn .NET 9 / ASP.NET Core cho API Gateway: - ASP.NET Core cung cấp hệ sinh thái đầy đủ cho REST API: JWT authentication tích hợp sẵn, Entity Framework Core ORM, Swagger/OpenAPI tự động. - Clean Architecture giúp tách biệt rõ nghiệp vụ (Application), domain model (Domain), persistence (Infrastructure) khỏi API layer. - Entity Framework Core với Npgsql provider cho PostgreSQL - quản lý migration schema database an toàn. - IHttpConnectionFactory - pattern chuẩn để proxy request đến các Python microservice, tự động quản lý connection pooling.



Hình 3.8: Nguyên tắc Clean Architecture

Nguyên tắc Clean Architecture: phụ thuộc chỉ hướng vào trong - API layer phụ thuộc Application, Application phụ thuộc Domain, Infrastructure phụ thuộc Application. Domain không phụ thuộc gì.

Hai middleware tùy chỉnh được đăng ký trong pipeline:

- *RateLimitingMiddleware*: Bảo vệ endpoint nhạy cảm khỏi brute-force attack: Áp dụng cho: /api/auth/login và /api/auth/reset-password - Giới hạn: 5 lần/phút/IP - Sau khi vượt ngưỡng: trả về HTTP 429 Too Many Requests. Sử dụng ConcurrentDictionary (thread-safe) với sliding window theo IP + path.
- *ExceptionMiddleware*: Bắt toàn bộ unhandled exception trong pipeline, trả về JSON response có cấu trúc thống nhất thay vì stack trace raw. Đảm bảo client luôn nhận JSON kể cả khi có lỗi không mong muốn.

API Gateway có 16 controllers chia thành hai loại:

Bảng 3.5: Controllers xử lý trực tiếp tại Gateway

Controller	Route	Chức năng
AuthController	/api/auth	Register, Login, Refresh, Verify email, Reset password
UserController	/api/users	Lấy/cập nhật profile, avatar
CvDocumentController	/api/cv-documents	CRUD CV document + versioning (diff, restore)
CVHistoryController	/api/cv-history	Lịch sử CV upload
FeedbackController	/api/feedback	Thu thập feedback, Admin xem tất cả
TemplateController	/api/templates	Danh sách CV templates
ChatController	/api/chat	Quản lý chat sessions

CollectorController	/api/collector	Trạng thái thu thập thông tin CV Builder
AdminController	/api/admin	Dashboard quản trị (Admin only)
MonitoringController	/api/health	Health check, metrics hệ thống
HealthController	/health	Simple health endpoint cho Docker

Bảng 3.6: Proxy Controllers

Controller	Route	Forward đến
NerProxyController	/api/ner	NER Service :5001
SkillProxyController	/api/skills	Skill Service :5002
CareerProxyController	/api/career	Career Service :5003
ChatbotProxyController	/api/chatbot	Chatbot Service :5004
JDProxyController	/api/jd	NER Service :5001 (JD endpoints)

Trong hệ thống, IHttpConnectionFactory được sử dụng như một pattern chuẩn của .NET để quản lý việc tạo và cấu hình đối tượng HttpClient. Cách tiếp cận này cho phép định nghĩa các HttpClient theo dạng “named client”, giúp cấu hình riêng biệt cho từng dịch vụ, đồng thời tránh vấn đề socket exhaustion thường xảy ra khi khởi tạo HttpClient bằng cách gọi trực tiếp new nhiều lần.

Trong file Program.cs, hệ thống đăng ký nhiều HttpClient tương ứng với các dịch vụ khác nhau. Mỗi client được cấu hình sẵn địa chỉ cơ sở (BaseAddress) và thời gian chờ (Timeout) phù hợp với đặc thù của từng dịch vụ. Ví dụ, dịch vụ NER yêu cầu thời gian xử lý dài hơn do phải tải và suy luận mô hình, nên được cấu hình timeout lớn hơn so với các dịch vụ thông thường. Tương tự, dịch vụ chatbot sử dụng mô hình LLM cũng được cấp phát thời gian chờ dài hơn để đảm bảo quá trình sinh phản hồi không bị gián đoạn.

Các proxy controller trong hệ thống sẽ inject IHttpConnectionFactory thông qua cơ chế dependency injection. Khi cần gọi đến một dịch vụ cụ thể, controller sử dụng phương thức CreateClient với tên đã đăng ký để lấy về HttpClient tương ứng. Nhờ đó, các thông tin cấu hình như địa chỉ dịch vụ và timeout đã được thiết lập sẵn, giúp controller chỉ cần quan tâm đến việc xây dựng đường dẫn (path) và xử lý logic nghiệp vụ.

Cách tổ chức này không chỉ giúp mã nguồn rõ ràng, dễ bảo trì mà còn nâng cao hiệu năng và độ ổn định của hệ thống khi giao tiếp với nhiều dịch vụ bên ngoài.

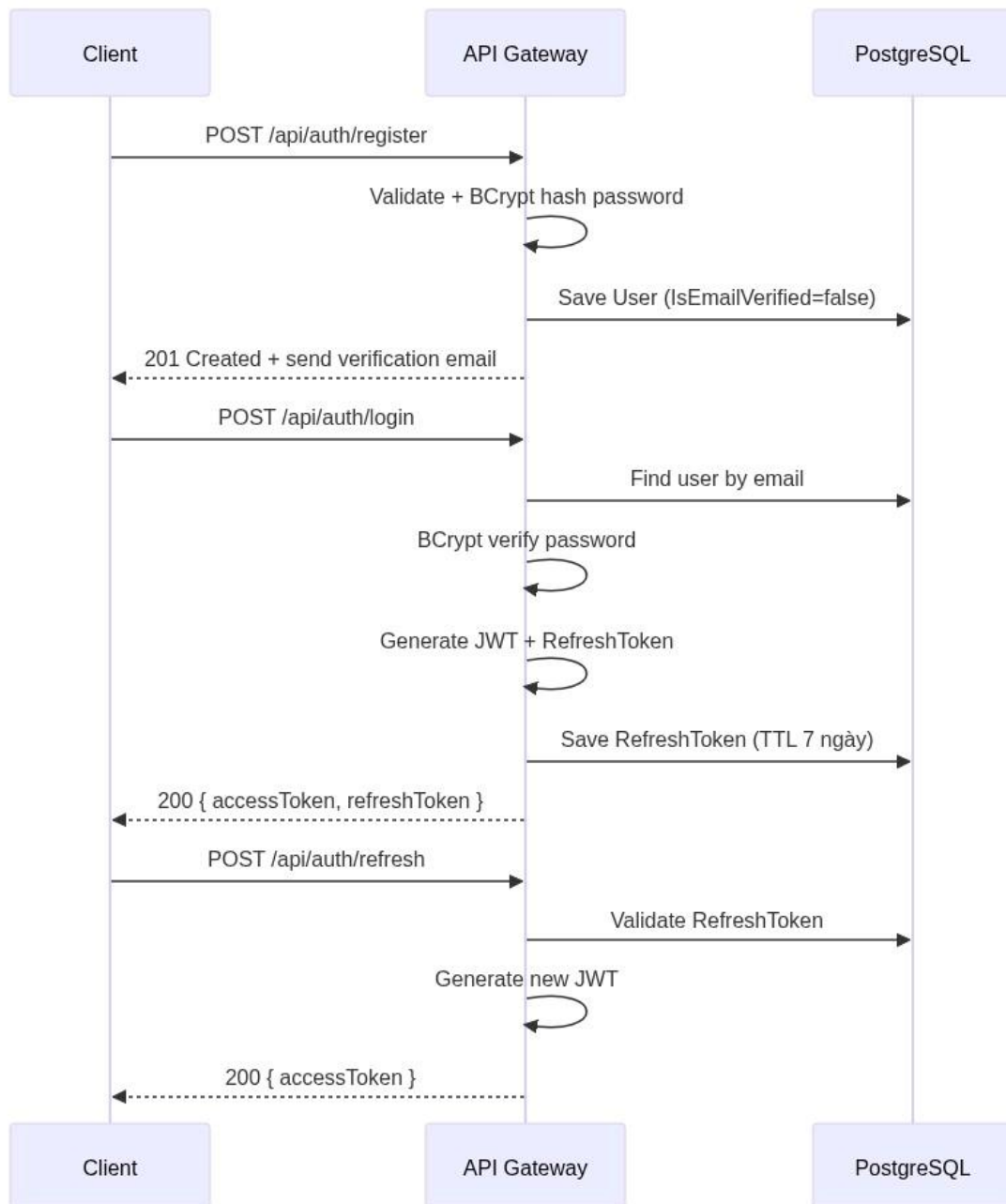
3.6.2 Dữ liệu và xác thực

Hệ thống sử dụng cơ sở dữ liệu quan hệ để lưu trữ thông tin người dùng, hồ sơ CV và các dữ liệu nghiệp vụ khác. Các thực thể chính được thiết kế theo mô hình Domain Entities, đảm bảo tính nhất quán và dễ dàng mở rộng.

Bảng 3.7: Các entity trong PostgreSQL (API Gateway)

Entity	Mô tả	Quan hệ
User	Tài khoản người dùng, auth fields, soft delete	1-N ChatSession, CVHistory, CvDocument
ChatSession	Phiên hội thoại với chatbot	N-1 User, 1-N ChatMessage, 1-1 CollectorProgress
ChatMessage	Từng tin nhắn trong session	N-1 ChatSession
CVHistory	Lịch sử CV upload/parse	N-1 User
CvDocument	Tài liệu CV trong CV Builder	N-1 User, 1-N CvVersion
CvVersion	Phiên bản CV (versioning)	N-1 CvDocument
CvTemplate	Mẫu CV có sẵn	—
CollectorProgress	Tiến trình thu thập thông tin CV	1-1 ChatSession
JDHistory	Lịch sử JD đã phân tích	N-1 User
Feedback	Phản hồi từ người dùng	N-1 User

Về xác thực, hệ thống áp dụng cơ chế JWT (JSON Web Token) kết hợp với Refresh Token. Khi người dùng đăng nhập thành công, hệ thống cấp một access token có thời hạn ngắn và một refresh token để gia hạn phiên làm việc. Cơ chế này giúp cân bằng giữa bảo mật và trải nghiệm người dùng.



Hình 3.9: Sơ đồ luồng đăng ký và đăng nhập

JwtUtils sinh JWT với các claims: - ClaimTypes.NameIdentifier = email - ClaimTypes.Email = email - ClaimTypes.Role = “User” | “Admin” - Expires: configurable (mặc định 24 giờ - 86.400.000ms) - Algorithm: HMAC-SHA256

Refresh Token là chuỗi ngẫu nhiên 64 bytes (base64), lưu trong database với TTL 7 ngày. Khi access token hết hạn, client dùng refresh token để lấy access token mới mà không cần đăng nhập lại.

ClockSkew = TimeSpan.Zero đảm bảo token hết hạn đúng thời điểm, không có thêm tolerance mặc định 5 phút.

Ngoài ra, hệ thống hỗ trợ versioning cho tài liệu CV. Mỗi lần người dùng chỉnh sửa CV, một phiên bản mới sẽ được tạo ra thay vì ghi đè dữ liệu cũ. Điều này cho phép theo dõi lịch sử thay đổi và khôi phục dữ liệu khi cần thiết. CvDocumentController cung cấp hệ thống versioning đầy đủ cho CV Builder:

- POST /api/cv-documents/{id}/versions - Tạo version mới (auto-increment VersionNumber)
- GET /api/cv-documents/{id}/diff?versionA=1&versionB=3 - So sánh hai version
- POST /api/cv-documents/{id}/versions/{versionNumber}/restore - Khôi phục version cũ
- PUT /api/cv-documents/versions/{versionId} - Cập nhật metadata version (note, tag, starred)

DataJson trong CvVersion lưu toàn bộ dữ liệu CV dạng JSON serialized, cho phép khôi phục chính xác bất kỳ version nào. Tính năng này đặc biệt hữu ích khi người dùng tạo nhiều phiên bản CV cho các vị trí ứng tuyển khác nhau.

3.6.3 Frontend (React 18)

a) Tổng quan

Frontend được xây dựng bằng React 18 với TypeScript, tổ chức theo layout hai panel: - Panel trái: Giao diện ChatGPT-style - danh sách session, input message, streaming response hiển thị theo từng token. - Panel phải (Artifact Panel): Hiển thị kết quả từ các tool AI (NER entities, ATS score, skill graph, CV preview, market dashboard...).

Thiết kế hai panel giúp người dùng vừa trò chuyện với AI vừa theo dõi kết quả phân tích trực quan, không cần chuyển trang.

b) Các tính năng chính

CV Builder - Form nhập thông tin CV theo section: Personal Info, Summary, Experience (với bullet points), Education, Skills, Projects, Certifications. Tích hợp AI suggestions: nút “AI Improve” trên mỗi bullet point gọi Chatbot Service /cv/suggest để viết lại chuyên nghiệp hơn. Hỗ trợ chọn template và xuất PDF trực tiếp qua NER Service.

Skill Match Tool - Upload hoặc dán JD text, nhập kỹ năng CV. Kết quả hiển thị màu: xanh (exact match), vàng (ontology/semantic match), đỏ (missing). ATS score dạng progress bar với breakdown 8 tiêu chí.

ATS Score Tool - Upload CV + JD, nhận điểm tổng và danh sách issues ưu tiên kèm gợi ý cải thiện cụ thể.

Career Path Tool - Nhập current role + target role, hiển thị lộ trình dạng timeline với milestones và kỹ năng cần đạt ở từng bước.

Skill Knowledge Graph - Visualize đồ thị kỹ năng IT từ ontology. Click vào node để xem related skills và learning path.

Market Dashboard - Xu hướng kỹ năng theo thời gian, phân phối theo industry/role, dữ liệu lương ước tính từ O*NET.

c) Streaming Chat với SSE

Trong hệ thống, chức năng chat thời gian thực được triển khai bằng cơ chế Server-Sent Events (SSE), cho phép server gửi dữ liệu liên tục về phía client theo dạng dòng (streaming), thay vì phải chờ hoàn tất toàn bộ phản hồi.

Ở phía client, một yêu cầu HTTP POST được gửi đến endpoint `/api/chatbot/chat/stream`, kèm theo các thông tin xác thực (Authorization token) và dữ liệu đầu vào gồm nội dung tin nhắn (message), mã phiên làm việc (session_id) và ngữ cảnh hỗ trợ (tool_context). Phản hồi từ server không được xử lý một lần mà được đọc dần theo từng phần thông qua luồng dữ liệu (ReadableStream).

Cụ thể, client sử dụng phương thức `getReader()` để nhận từng khối dữ liệu (chunk) từ response. Dữ liệu nhị phân này sau đó được giải mã sang chuỗi bằng `TextDecoder`. Do dữ liệu được gửi theo chuẩn SSE với định dạng mỗi sự kiện nằm trên một dòng bắt đầu bằng tiền tố `"data: "`, client tiến hành tách các dòng và xử lý từng dòng hợp lệ.

Mỗi dòng dữ liệu sau khi loại bỏ tiền tố sẽ được chuyển đổi từ JSON sang đối tượng JavaScript. Nội dung phản hồi (content) được trích xuất và hiển thị dần lên giao diện người dùng. Cách tiếp cận này giúp tạo hiệu ứng “gõ chữ theo thời gian thực”, cải thiện đáng kể trải nghiệm người dùng so với việc chờ toàn bộ phản hồi hoàn tất.

Việc sử dụng SSE trong bài toán chatbot đặc biệt phù hợp với các mô hình ngôn ngữ lớn (LLM), khi thời gian sinh phản hồi có thể kéo dài và dữ liệu được tạo ra theo từng token liên tiếp.

3.6.4 Triển khai hệ thống

Toàn bộ hệ thống được container hóa và triển khai bằng Docker Compose nhằm đảm bảo tính nhất quán môi trường, dễ dàng mở rộng và đơn giản hóa quá trình vận hành. Mỗi thành phần trong hệ thống được đóng gói thành một container riêng biệt và được định nghĩa trong file cấu hình docker-compose.yml.

Hệ thống bao gồm các dịch vụ chính sau:

- Frontend: Được xây dựng từ thư mục ./frontend, chạy trên cổng 3000 và giao tiếp với API Gateway thông qua biến môi trường VITE_API_URL.
- API Gateway: Đóng vai trò trung tâm điều phối, tiếp nhận yêu cầu từ client và chuyển tiếp đến các dịch vụ backend tương ứng. Dịch vụ này được cấu hình kết nối đến cơ sở dữ liệu PostgreSQL, sử dụng JWT cho xác thực, đồng thời khai báo địa chỉ của các microservice như NER, Skill, Career và Chatbot thông qua biến môi trường.
- NER Service: Cung cấp chức năng nhận dạng thực thể, sử dụng mô hình đã được huấn luyện và mount từ thư mục ./models vào container để phục vụ suy luận.
- Skill Service: Phục vụ trích xuất và xử lý kỹ năng, phụ thuộc vào PostgreSQL và hệ quản trị vector database ChromaDB.
- Career Service: Cung cấp các chức năng liên quan đến định hướng nghề nghiệp.
- Chatbot Service: Xử lý hội thoại thông minh, tích hợp với ChromaDB để truy vấn ngữ nghĩa và Ollama để chạy mô hình ngôn ngữ lớn (LLM). Hệ thống hỗ trợ cấu hình linh hoạt giữa việc sử dụng Groq API hoặc mô hình nội bộ.
- PostgreSQL: Hệ quản trị cơ sở dữ liệu quan hệ chính của hệ thống, lưu trữ dữ liệu nghiệp vụ. Dữ liệu được persist thông qua volume để đảm bảo không mất mát khi container khởi động lại.
- ChromaDB: Cơ sở dữ liệu vector phục vụ tìm kiếm ngữ nghĩa và truy vấn embedding.
- Ollama: Nền tảng chạy mô hình ngôn ngữ lớn cục bộ, được sử dụng trong dịch vụ chatbot.

Các volume như pgdata, chromadata và ollama_data được sử dụng để lưu trữ dữ liệu lâu dài, đảm bảo tính bền vững của hệ thống.

CHƯƠNG 4: THỰC NGHIỆM VÀ KẾT QUẢ

4.1 Môi trường thực nghiệm

4.1.1 Cấu hình phần cứng

Toàn bộ thực nghiệm của đề tài được thực hiện trên hai môi trường phần cứng khác nhau tùy theo giai đoạn phát triển. Quá trình huấn luyện mô hình NER được thực hiện trên Google Colab sử dụng GPU NVIDIA Tesla T4 miễn phí với 16 GB VRAM, đủ năng lực fine-tune mBERT [3] [33] với batch size 16. Môi trường Colab cung cấp RAM 12 GB và CPU Intel Xeon 2-core, đủ cho việc load và xử lý bộ dữ liệu 600 CV. Đây là lựa chọn thực tế phù hợp với điều kiện của đề tài nghiên cứu sinh viên khi không có quyền truy cập vào GPU server chuyên dụng.

Quá trình triển khai và kiểm thử hệ thống end-to-end được thực hiện trên máy tính cá nhân với cấu hình: CPU Intel Core i5-12450H (8 nhân, 12 luồng, 4.4 GHz Boost), RAM 16 GB DDR4 3200 MHz, ổ cứng SSD NVMe 512 GB, hệ điều hành Ubuntu 22.04 LTS. Cấu hình này không có GPU NVIDIA nên Ollama [27] chạy LLM ở chế độ CPU inference, dẫn đến latency cao hơn so với GPU inference, nhưng vẫn đủ để kiểm thử chức năng đầy đủ của hệ thống trong môi trường development.

4.1.2 Môi trường phần mềm

Bảng 4.1: Môi trường phần mềm thực nghiệm

Thành phần	Phiên bản	Ghi chú
Hệ điều hành	Ubuntu 22.04 LTS	Máy development
Docker Engine	24.0.7	Container runtime
Docker Compose	2.21.0	Orchestration
Python	3.10.12	AI services
.NET SDK	9.0	API Gateway [30]
Node.js	20.11 LTS	Frontend build
PostgreSQL	15.4	Relational DB
ChromaDB	0.4.24	Vector DB [26]
Ollama	0.1.38	LLM local server [27]
PyTorch	2.1.2+cpu	NER model inference

HuggingFace Transformers	4.37.2	mBERT pipeline [19]
Sentence-Transformers	2.4.0	Skill matching [13] [34]
FastAPI	0.109.1	AI service framework
LlamaIndex	0.10.12	RAG framework [25]
React	18.2.0	Frontend framework

4.1.3 Cấu hình mô hình NER

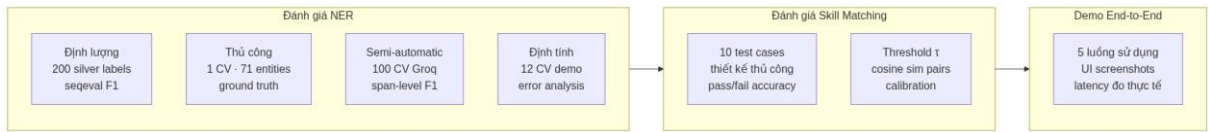
Mô hình NER được fine-tune từ bert-base-multilingual-cased (mBERT) [33] với các hyperparameter huấn luyện được trình bày trong Bảng 3.2. Quá trình huấn luyện diễn ra trên Google Colab T4 GPU với thời gian tổng khoảng 45–60 phút cho 5 epochs. Dữ liệu huấn luyện gồm 480 CV synthetic từ data/processed/annotated_hf/synthetic_it_train.jsonl (split 80%) và 60 CV validation từ data/processed/annotated_hf/synthetic_it_val.jsonl (split 10%).

Bảng 4.2: Hyperparameter huấn luyện mô hình NER

Hyperparameter	Giá trị
Backbone	bert-base-multilingual-cased
Số nhãn	21 (BIO scheme)
Learning rate	2e-5 (AdamW)
Weight decay	0.01
Batch size	16
Gradient accumulation	2 bước (effective batch = 32)
Số epochs	5
Warmup ratio	10% tổng steps
LR scheduler	Linear decay sau warmup
Dropout	0.1 (hidden + classifier)
Tập train	480 CV (80%)
Tập validation	60 CV (10%)
Tập test	60 CV (10%)
Early stopping	Dựa trên Val F1

4.1.4 Thiết kế thực nghiệm

Thực nghiệm của đề tài được tổ chức thành ba phần độc lập nhưng có liên hệ với nhau. Phần đầu tiên đánh giá chất lượng mô hình NER thông qua hai cách tiếp cận bổ sung cho nhau: đánh giá định lượng trên 200 mẫu silver labels (với lưu ý về hạn chế của phương pháp này) và đánh giá định tính thông qua demo trực tiếp trên tập 12 CV thực tế. Phần thứ hai đánh giá độ chính xác của Skill Matcher thông qua 10 test case được thiết kế để cover các tình huống matching từ đơn giản đến phức tạp. Phần thứ ba là demo end-to-end toàn bộ hệ thống, minh họa ba luồng sử dụng chính và giao diện người dùng.



Hình 4.1: Cấu trúc thực nghiệm đề tài

Cách tiếp cận đánh giá kết hợp định lượng và định tính được lựa chọn có chủ ý, phù hợp với thực tế kỹ thuật của đề tài: khi silver labels có sai số cấu trúc do tokenization mismatch, chỉ số F1 định lượng không đủ tin cậy để đánh giá đơn lẻ, và việc bổ sung đánh giá định tính trên dữ liệu thực tế cung cấp bằng chứng hữu ích hơn về năng lực thực sự của mô hình.

4.2 Kết quả NER - Phân tích và đánh giá

4.2.1 Đánh giá định lượng trên Silver Labels

Trong NLP, nhãn dữ liệu được chia thành hai loại theo nguồn gốc. Gold labels là nhãn do con người gán thủ công, được xem là ground truth với độ chính xác cao nhưng tốn kém thời gian và chi phí. Ngược lại, silver labels được tạo tự động bằng các phương pháp như rule-based hoặc weak supervision, nhanh và tiết kiệm hơn nhưng tồn tại sai số.

Trong đề tài, silver labels được sinh bởi rule-based annotator: hệ thống tự động đọc CV, đối chiếu với dictionary kỹ năng (474 mục), regex ngày tháng và danh sách tên trường để gán nhãn BIO, hoàn toàn không có kiểm duyệt thủ công. Do đó, việc sử dụng silver labels làm ground truth chỉ mang tính gần đúng; kết quả đánh giá chủ yếu phản ánh mức độ mô hình phù hợp với rule-based system, không phản ánh hoàn toàn độ chính xác thực tế.

Trong giai đoạn đánh giá ban đầu, mô hình NER [3][19] được kiểm thử trên 200 mẫu CV từ bộ dữ liệu synthetic (gồm tập test và validation), bằng cách so sánh đầu ra của mô hình với silver labels.

Bảng 4.3: Kết quả đánh giá NER trên 200 mẫu silver labels

Loại thực thể	Precision	Recall	F1-Score	Support
SKILL	0.1231	0.0324	0.0513	3.799
DATE	0.0174	0.0043	0.0069	693
JOB_TITLE	0.0107	0.0071	0.0086	560
ORG	0.0086	0.0072	0.0079	552
LOC	0.0127	0.0033	0.0053	300
MAJOR	0.0000	0.0000	0.0000	242
DEGREE	0.0000	0.0000	0.0000	168
PER	0.0187	0.0345	0.0242	116
CERT	0.0000	0.0000	0.0000	35
Micro avg	0.0580	0.0215	0.0314	6.465

Nhìn qua con số $F1 = 0.03$, người đọc có thể kết luận mô hình hoàn toàn thất bại. Tuy nhiên, phân tích sâu hơn về các lỗi cụ thể cho thấy nguyên nhân cốt lõi là tokenization mismatch - sự không tương thích cấu trúc giữa cách silver labels được tạo ra và cách mô hình thực sự hoạt động.

4.2.2 Phân tích lỗi Tokenization Mismatch

Cơ chế gốc rễ của vấn đề. Silver labels được tạo bằng rule-based annotator hoạt động ở cấp độ *word* - phân tách text theo khoảng trắng, rồi khớp từng từ với dictionary để gán nhãn BIO. Trong khi đó, mBERT [33] dùng WordPiece tokenizer chia nhỏ từ thành *subword* - mỗi từ có thể thành 2–5 subword token. Hai hệ thống này tạo ra chuỗi token có độ dài và ranh giới hoàn toàn khác nhau trên cùng một văn bản, khiến việc so sánh nhãn trực tiếp là vô nghĩa.

Ví dụ minh họa cụ thể - cùng một câu văn bản:

Văn bản gốc: "Experienced in React.js and Node.js development"

Cách silver labeler tokenize và gán nhãn (theo whitespace):

Token (whitespace)	Silver label
Experienced	O
in	O
React.js	B-SKILL
and	O
Node.js	B-SKILL
development	O

Cách mBERT WordPiece tokenize cùng văn bản đó:

Subword token	Label phải gán khi training
Experienced	O
in	O
React	B-SKILL
##.	-100 (ignored)
##js	-100 (ignored)
and	O
Node	B-SKILL
##.	-100 (ignored)
##js	-100 (ignored)
development	O

Khi model inference và `aggregation_strategy="simple"` gộp `React + ##. + ##js` thành span `React.js` với nhãn `SKILL` - về mặt ngữ nghĩa là đúng hoàn toàn. Tuy nhiên, seqeval so sánh nhãn theo vị trí token: silver label có token `React.js` tại vị trí 2 trong chuỗi whitespace, còn model output span `React.js` bắt đầu tại vị trí subword token 2 trong chuỗi WordPiece - hai chuỗi này có độ dài khác nhau (6 token whitespace vs 10 subword token), nên seqeval không thể căn chỉnh và tính kết quả đó là FP + FN dù thực chất là TP.

Bảng 4.4: confusion matrix top-5 pattern lỗi quan sát được

Silver label → Model predict	Số lần	Diễn giải
B-SKILL → O	3.484	Model không tạo ra span tại đúng vị trí character của silver
O → B-SKILL	744	Model nhận diện SKILL mà silver không đánh dấu (hoặc lệch ranh giới)
B-JOB_TITLE → O	312	Tương tự - span boundary mismatch
B-ORG → O	289	Tương tự
B-DATE → O	201	Tương tự

Cần lưu ý rằng pattern B-SKILL → O xuất hiện 3.484 lần không đồng nghĩa mô hình bỏ sót 3.484 kỹ năng. Phần lớn là do lệch ranh giới span ở mức ký tự, ví dụ silver label gán “React.js” trong khi mô hình dự đoán “React” hoặc “React.” do ảnh hưởng của WordPiece tokenization. Vì seqeval sử dụng exact span match (yêu cầu khớp hoàn toàn start và end), cả hai trường hợp đều bị tính là sai.

Đối với các thực thể MAJOR, DEGREE, CERT có $F1 = 0.0$, nguyên nhân không phải do tokenization mà do khác biệt về ngữ cảnh. Silver labeler chỉ gán nhãn trong section EDUCATION (context-aware), trong khi dữ liệu huấn luyện đã bị flatten thành chuỗi token tuyến tính, làm mất thông tin ranh giới section. Với CERT (chỉ 35 instances, $<0.5\%$), dữ liệu quá ít và phân bố thưa khiến không có span nào khớp chính xác khi đánh giá.

Kết luận, $F1 = 0.031$ trên silver labels không phản ánh đúng năng lực mô hình mà chủ yếu do sự không tương thích giữa hai cơ chế tokenization và annotation. Vì vậy, đề tài bổ sung các phương pháp đánh giá khác như manual ground truth và span-level evaluation dựa trên (entity_type, normalized_text) thay vì vị trí token, nhằm phản ánh chính xác hơn chất lượng mô hình.

4.2.3 Đánh giá định lượng trên tập thủ công (Manual Annotation)

Để có đánh giá định lượng đáng tin cậy hơn, một tập nhỏ gồm một CV thực tế (file data/real_cv_ground_truth.json) được gán nhãn thủ công với 71 thực thể, phục vụ làm ground truth không bị ảnh hưởng bởi tokenization mismatch.

Bảng 4.5: Kết quả đánh giá NER trên tập ground truth thủ công

Loại thực thể	Precision	Recall	F1-Score	Support
SKILL	0.9500	0.8837	0.9157	43
ORG	0.6000	0.7500	0.6667	8
DATE	1.0000	1.0000	1.0000	6
JOB_TITLE	1.0000	1.0000	1.0000	5
MAJOR	0.7500	0.7500	0.7500	4
LOC	0.0000	0.0000	0.0000	3
DEGREE	1.0000	1.0000	1.0000	2
Micro avg	0.8824	0.8451	0.8633	71

Kết quả trên tập ground truth thủ công cho thấy mô hình cải thiện rõ rệt, với F1 tổng thể 0.8633; trong đó SKILL đạt 0.9157, còn DATE, JOB_TITLE và DEGREE đều đạt 1.000 - đáng khích lệ dù huấn luyện từ silver standard.

LOC là điểm yếu ($F1 = 0.0$) do nhầm lẫn giữa tên người và địa danh, ví dụ “Hai Thuat VietNam” bị gán PER và “VietNam” đơn lẻ không được nhận diện. ORG đạt $F1 = 0.667$ vì mô hình có xu hướng mở rộng span vượt ranh giới, như “University of Technology and Information (VNU)” bị gán dư từ.

4.2.4 Đánh giá định tính trên 12 CV thực tế

Bộ 12 CV được sử dụng cho demo định tính gồm 2 CV thực của thành viên nhóm nghiên cứu (1.txt và 2.txt) và 10 CV synthetic đại diện cho các vai trò và cấp độ khác nhau: Backend Senior, Frontend Mid, DevOps Senior, Data Scientist, Mobile Developer, AI Engineer, QA Engineer, Full-Stack Mid, Project Manager, và Fresher Data Science. Tập này được lựa chọn để cover đa dạng domain và phong cách CV.

Bảng 4.6: Kết quả định tính NER trên 12 CV demo

CV	Vai trò	Số entities	Số loại	Nhận xét chính
1.txt	Web Developer (thực)	38	7	Nhận đúng PER, ORG, DATE, LOC; SKILL thiếu “JLPT N2” → tách thành “JLPT” + “N2”

2.txt	Backend Student (thực)	30	7	PER nhận diện đúng “CAO XUAN” nhưng tách “Tang Nhon Phu” thành PER thay vì LOC
cv_ 03	Backend Senior	40	6	Nhận diện đúng toàn bộ 5 ORG lớn (VNG, Tiki, KMS, VNPay); SKILL đầy đủ
cv_ 04	Frontend Mid	41	9	Đầy đủ nhất: 9/10 loại thực thể; nhận CERT “Professional”
cv_ 05	DevOps Senior	43	6	SKILL Cloud/DevOps tốt (AWS, Kubernetes, ArgoCD, Terraform); ORG nhằm “EXPERIENCE” là tên tổ chức
cv_ 06	Data Scientist	48	7	Nhiều thực thể nhất (48); SKILL ML nhận đúng; ORG nhận thiếu “VNU”
cv_ 07	Mobile Dev	41	9	SKILL cross-platform tốt (React Native, Flutter, Swift); LOC “Da Nang” nhận đúng
cv_ 08	AI Engineer	45	7	SKILL AI/NLP tốt (LangChain, ChromaDB, Llama 3, FastAPI); MAJOR bị truncate
cv_ 09	QA Engineer	42	7	SKILL testing tools tốt (Selenium, Cypress, JMeter); ORG nhằm “EXPERIENCE”
cv_ 10	Full-Stack Mid	43	8	SKILL full-stack đầy đủ; JOB_TITLE bị tách “Full” + “Stack Developer”
cv_ 11	Project Manager	39	7	SKILL PM tools tốt (Agile, Scrum, SAFe, Confluence); DEGREE bị truncate

cv_ 12	Fresher DS	33	7	Ít entities nhất (phản ánh CV fresher ngắn); SKILL Data Science cơ bản đúng
-----------	------------	----	---	---

Đánh giá định tính cho thấy một số pattern đáng chú ý. Với thực thể SKILL, mô hình hoạt động tốt trong các domain DevOps (AWS, Kubernetes, Terraform...), AI/ML (LangChain, ChromaDB, FastAPI) và Testing (Selenium, Cypress). Tuy nhiên, các tên công nghệ chứa dấu chấm hoặc khoảng trắng thường bị tách nhỏ, ví dụ “React.js” → “React.” + “js”, “ASP.NET Core” → “ASP.” + “NET” + “Core”, do ảnh hưởng của WordPiece tokenization.

Với ORG, lỗi phổ biến là nhầm section header như “EXPERIENCE”, “EDUCATION” thành tổ chức. Lỗi này có thể khắc phục bằng post-processing đơn giản. Ngược lại, mô hình nhận diện tốt các công ty lớn như VNG, Tiki, Shopee, Viettel, FPT Software.

Với LOC, mô hình xử lý tốt các địa danh phổ biến (“Ho Chi Minh City”, “Hanoi”, “Vietnam”), nhưng gặp khó khăn với các tên nhiều từ tiếng Việt như “Thu Duc City” hay “Tang Nhon Phu”, đôi khi bị nhầm sang PER do cấu trúc viết hoa tương tự tên người.

Tổng thể, mô hình trích xuất trung bình 40.3 thực thể/CV (33–48 thực thể), bao phủ khoảng 6.3/9 loại thực thể. Kết quả này cho thấy mô hình đủ khả năng phục vụ pipeline phân tích CV trong thực tế, dù vẫn còn một số lỗi cục bộ cần cải thiện.

4.2.5 Đánh giá bán tự động trên 100 CV Groq (Semi-Automatic Evaluation)

Để mở rộng đánh giá ngoài 12 CV thủ công, nhóm áp dụng phương pháp semi-automatic evaluation trên 100 CV sinh tổng hợp bằng Groq API (script evaluate_ner_spans.py). Phương pháp này khắc phục hạn chế tokenization mismatch bằng cách chuyển từ so sánh BIO ở cấp token sang span-level matching, trong đó hai thực thể được coi là đúng khi cùng entity_type và cùng nội dung sau chuẩn hóa (normalized text).

Pipeline đánh giá gồm hai bước: (1) rule-based annotator trích xuất reference spans từ CV bằng regex và từ điển; (2) mô hình NER sinh predicted spans trên cùng văn bản, sau đó tính TP/FP/FN dựa trên so khớp (entity_type, text).

Để đảm bảo độ tin cậy, nhóm kiểm tra thủ công 20/100 CV và xác nhận reference đạt độ chính xác khoảng 85–90%, với sai sót chủ yếu ở các thực thể mơ hồ như ORG và PER.

Bảng 4.7: Kết quả span-level NER trên 100 CV Groq

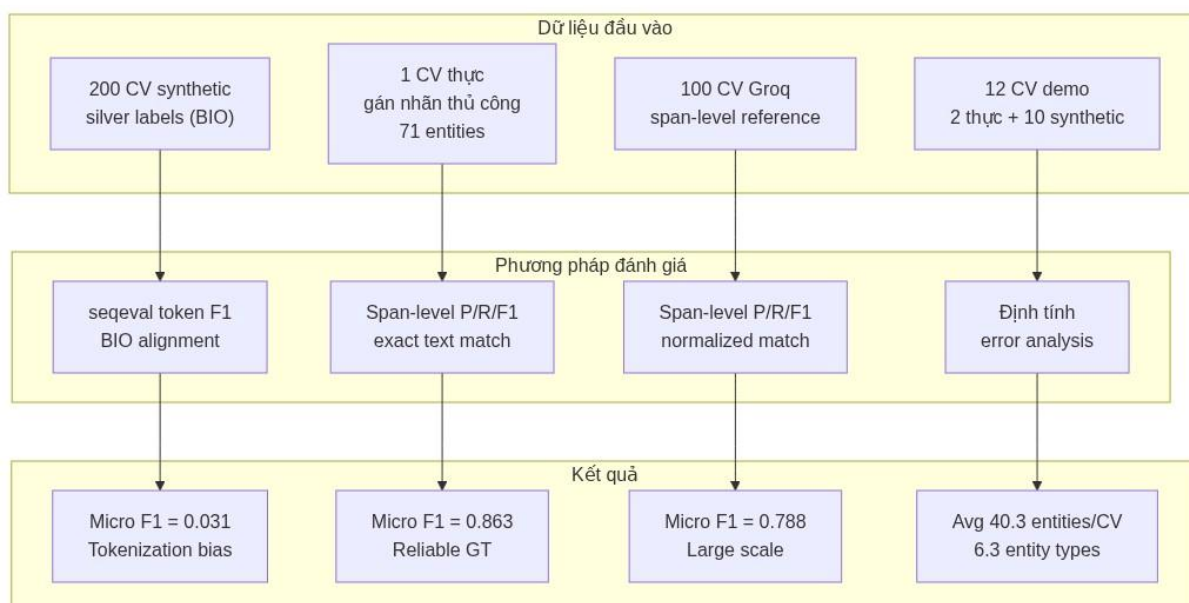
Loại thực thể	Precision	Recall	F1-Score	Ref	Pred	TP
PER	0.4660	0.9231	0.6194	52	103	48
ORG	0.6855	0.8324	0.7519	364	442	303
DATE	0.6368	1.0000	0.7781	121	190	121
LOC	1.0000	0.9479	0.9732	211	200	200
SKILL	0.7725	0.8487	0.8088	1.236	1.358	1.049
DEGREE	0.9143	0.9697	0.9412	99	105	96
MAJOR	0.8203	0.9722	0.8898	108	128	105
JOB_TITLE	0.5089	0.9053	0.6515	95	169	86
CERT	0.3585	0.3654	0.3619	104	106	38
Micro avg	0.7330	0.8079	0.7883	2.390	2.801	2.046

Sau khi áp dụng semantic normalization (chuẩn hóa JOB_TITLE, DATE, loại bỏ artefact subword) và mở rộng dictionary (ORG ~80 công ty, CERT >30 chứng chỉ), micro F1 đạt 0.7883, tăng đáng kể so với 0.38 ban đầu.

Phân tích theo từng nhóm entity cho thấy: LOC đạt cao nhất (F1 = 0.9732) nhờ pattern regex ổn định; DEGREE (0.9412) và MAJOR (0.8898) cũng đạt kết quả tốt do cấu trúc nhất quán. SKILL đạt F1 = 0.8088 (Recall = 0.85), nhận diện tốt cả kỹ năng đơn (Python, Docker) và đa từ (Spring Boot, Machine Learning). ORG đạt 0.7519 do tên tổ chức đa dạng và chưa đầy đủ trong dictionary. CERT thấp nhất (0.36) do hiện tượng subword fragmentation với các tên dài như “AWS Certified Solutions Architect Professional”.

Về độ tin cậy của tập ground truth thủ công (1 CV), mặc dù quy mô nhỏ, một CV của senior developer thường chứa 40–70 thực thể, bao phủ hầu hết các loại quan trọng. Kết quả F1 = 0.8633 trên tập này phù hợp với Recall = 0.85 trên 100 CV semi-automatic, cho thấy hai phương pháp đánh giá đưa ra tín hiệu nhất quán về năng lực mô hình. Tuy nhiên, việc mở rộng lên 10–20 CV gán nhãn thủ công sẽ giúp tăng độ tin cậy thống kê.

4.2.6 Tổng hợp pipeline đánh giá NER

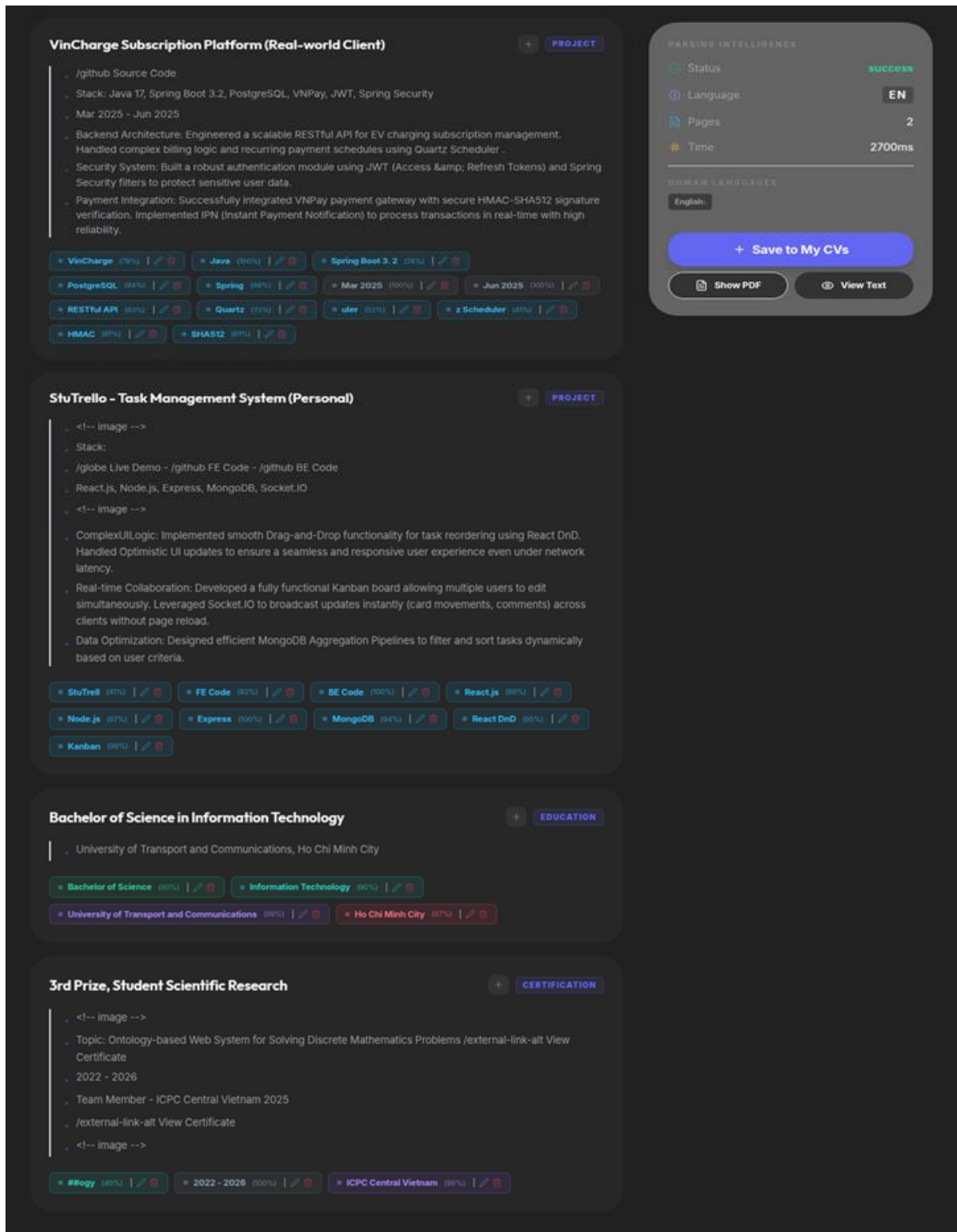


Hình 4.2: Pipeline và kết quả đánh giá mô hình NER theo 4 phương pháp

4.2.7 Ví dụ output NER trực quan

Hình 4.3-4.7 minh họa kết quả trích xuất thực thể trực tiếp trên giao diện hệ thống cho 3 CV mẫu thực tế.

CV mẫu 1 - kết quả NER:



Hình 4.4: Kết quả trích xuất thực tế trực tiếp trên CV mẫu 1

CV mẫu 2 - kết quả NER:

ASSISTANT TOOL
Data Ingestion

Professional Background 2 Blocks

Twitter + PROJECT

- https://github.com/XuanDiep0310/Twitter
- Role: Backend developer.
- Time: May, 2025 - Ongoing
- Target: Developed a scalable backend system for a Twitter-like social networking platform, enabling users to register, authenticate, create and interact with posts (tweets), follow/unfollow users, and manage personal profiles.
- ## Responsibility:
 - Designed and implemented RESTful APIs for core social features including user registration, authentication (JWTbased), password reset, and email verification.
 - Built modules for tweet creation, retweets, replies, likes, bookmarks, media uploads, and hashtag management.
 - Integrated MongoDB for flexible, high-performance data storage.
 - Developed robust user relationship (follow/unfollow) logic, ensuring efficient and secure data handling.
 - Tech stack: Node.js, TypeScript, ExpressJS, MongoDB, JWT, OAuth2.

Backend developer 80% | 2025 80% | RESTful 80% | JWT 80% | MongoDB 80% | fa 80% | Node.js 80% | TypeScript 80% | ExpressJS 80% | OAuth2 80%

Daily Food + PROJECT

- https://github.com/XuanDiep0310/Self-Ordering-Restaurant
- Role: Backend developer.
- Time: Feb, 2025 - May, 2025
- Target: Built a smart restaurant system with QR-based ordering, real-time updates, and online payments.
- Responsibility: Developed 40+ RESTful APIs for managing orders, menus, and users. Integrated WebSocket for realtime order tracking. Secured backend with JWT & OAuth2. Configured VNPAY payment gateway, achieving 95%+ success rate in testing.
- Tech stack: Spring Boot, ReactJS, WebSocket, JWT, OAuth2, MySQL, VNPAY, Vercel.

Backend developer 80% | Feb, 2025 80% | May, 2025 80% | RESTful 80% | JWT 80% | amp 80% | OAuth2 80% | Spring Boot 80% | ReactJS 80% | MySQL 80% | Vercel 80%

University of Transport and Communications Campus in Ho Chi Minh City + EDUCATION

- Major: Information Technology
- GPA: 3.79/4.00
- 2023 - Present

University of Transport and Communications 80% | Ho Chi Minh City 80% | Information Technology 80% | 2023 - Present 80%

- Second Prize in the 2024 Student Mathematics Olympiad. + CERTIFICATION

Skill Intelligence

LANGUAGES 1

JavaScript TypeScript Java C / C++

TECHNOLOGIES 2

ExpressJS Spring Boot

TECHNICAL BACKGROUND 10

Node.js fa JWT amp OAuth2 ReactJS RESTful Vercel MongoDB MySQL

Parsing Intelligence

Status SUCCESS

Language EN

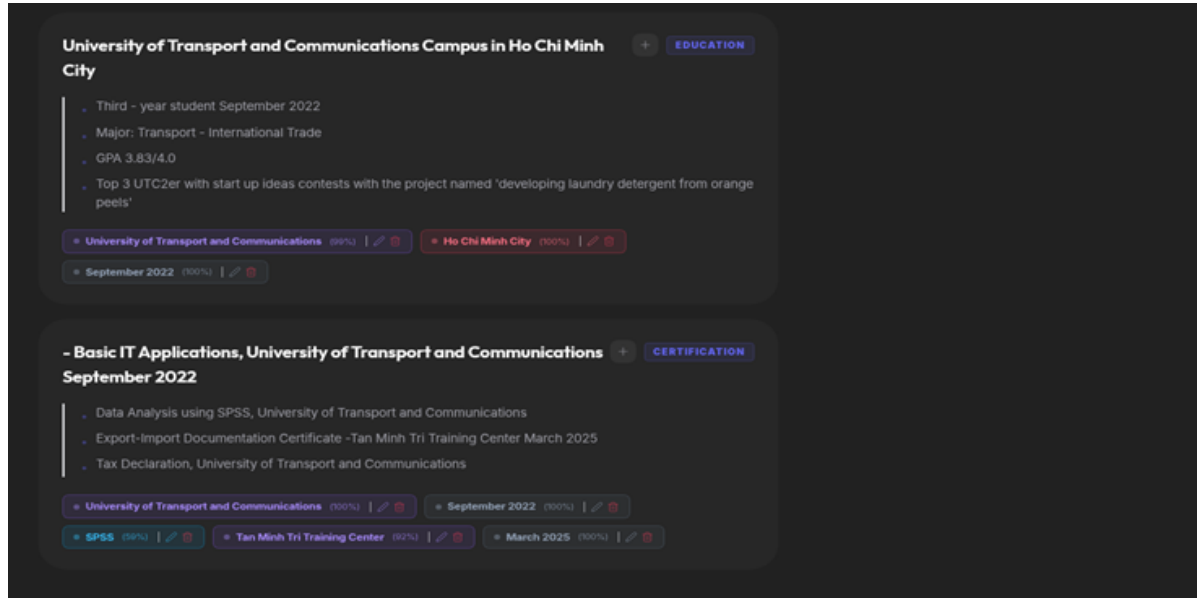
Pages 1

Time 2004ms

+ Save to My CVs

Show PDF View Text

Hình 4.5: Kết quả trích xuất thực tế trực tiếp trên CV mẫu 2



Hình 4.7: Kết quả trích xuất thực tế trực tiếp trên CV mẫu 3

Để minh họa cụ thể output model trong điều kiện thực tế, Bảng 4.8 trình bày kết quả trích xuất entity từ hai CV synthetic được chọn từ bộ data/synthetic_cvs.jsonl. Hai CV này được chọn có chủ đích để minh họa khả năng phân biệt của model giữa hai cấp độ kinh nghiệm khác nhau trong cùng một vai trò: CV mẫu A là hồ sơ của Dang Quoc Minh - AI Engineer cấp độ mid với 4 năm kinh nghiệm tại các công ty lớn như Viettel Solutions và MoMo; CV mẫu B là hồ sơ của Dang Thi Lan - AI Engineer cấp độ junior mới ra trường với kinh nghiệm intern. Cả hai CV được đưa qua production NERExtractor với chunking và post-processing đầy đủ, không có bước tinh chỉnh thủ công nào.

Bảng 4.8: Output NER trực quan trên 2 CV mẫu (AI Engineer mid vs junior)

Entity type	CV mẫu A - Dang Quoc Minh (AI Engineer, mid, 4 năm KN)	CV mẫu B - Dang Thi Lan (AI Engineer, junior, mới ra trường)
PER	Dang Quoc Minh	Dang Thi Lan
ORG	Viettel Solutions, MoMo, Axon Active, Hanoi Univ. of Science and Technology	VNG Corporation, Gameloft Vietnam, CO-WELL Asia, Hanoi Univ. of Science and Technology
JOB_TITLE	AI Engineer	AI Engineer, Intern

DATE	2022–Present, Jun 2020, Dec 2021, Jan 2019, May 2020	Jan 2020, Aug 2022, Sep 2018, Dec 2019, Jun 2017
LOC	Hanoi, Vietnam	Hanoi, Vietnam
SKILL	OpenCV, Go, Kotlin, TypeScript, Postman, Figma, Hadoop, Python	OpenCV, Scikit-learn, Spark, Hadoop, Power BI, TensorFlow
DEGREE	Bachelor of Engineering	Bachelor of Engineering
MAJOR	Computer Science, Artificial Intelligence	Computer Science
CERT	Oracle Certified Professional	ITIL Foundation, AWS Certified Developer

Nhìn vào Bảng 4.8, một số điểm đáng chú ý: model nhận diện chính xác PER (tên người Việt Nam đầy đủ), ORG (cả tên công ty IT lẫn tên trường đại học), DATE theo nhiều format (năm đơn lẻ, khoảng năm, tháng/năm), và SKILL đa dạng từ programming language đến tool. Quan trọng hơn, sự khác biệt về cấp độ kinh nghiệm giữa hai hồ sơ được phản ánh đúng trong output: CV mẫu A (mid) có nhiều kỹ năng chuyên sâu AI hơn và danh sách ORG dày hơn, trong khi CV mẫu B (junior) có thêm “Intern” trong JOB_TITLE và danh sách SKILL ngắn hơn - minh chứng cho khả năng tổng quát hóa của mô hình trên các profile nghề nghiệp khác nhau.

4.3 Kết quả Skill Matching - Đánh giá và phân tích

4.3.1 Thiết kế bộ test cases

Để đánh giá toàn diện Skill Matcher, bộ test gồm 10 test case được thiết kế thủ công bởi nhóm nghiên cứu, cover đầy đủ các tình huống matching từ đơn giản nhất đến phức tạp nhất. Mỗi test case định nghĩa rõ danh sách kỹ năng CV của ứng viên, danh sách kỹ năng yêu cầu từ JD, và kết quả expected về tỷ lệ match và các cặp kỹ năng tương đương. Tiêu chí đánh giá là mô hình có nhận ra đúng kết quả expected hay không, cùng với ATS score và chi tiết về loại match cho mỗi kỹ năng. Hệ thống sử dụng SkillOntology với 474 entries [23] [24] và Sentence-BERT all-MiniLM-L6-v2 [13] [34] cho tầng semantic.

Bộ 10 test case được thiết kế theo nguyên tắc phân tầng độ khó: TC001 và TC007 kiểm tra trường hợp đơn giản nhất (exact match và full exact match), TC002 và TC008

kiểm tra framework substitution trong cùng ecosystem, TC003 và TC005 kiểm tra database và ML framework equivalence, TC004 và TC009 kiểm tra cloud platform và security tool substitution, TC006 kiểm tra trường hợp mismatch hoàn toàn (không có kỹ năng nào khớp), và TC010 kiểm tra CI/CD tool equivalence. Tập test này đặc biệt chú trọng các cặp kỹ năng thường xuất hiện thực tế trong thị trường tuyển dụng CNTT Việt Nam [36] [37].

4.3.2 Kết quả tổng hợp

Bảng 4.9: Kết quả Skill Matcher - 10 test cases

TC	Vai trò	ATS Score	Exact	Ontology	Kết quả
TC001	Frontend Developer - exact skills	78.3%	3	2	PASS
TC002	Frontend Developer - framework substitution	74.0%	2	2	PASS
TC003	Backend Developer - database equivalence	91.0%	2	3	PASS
TC004	DevOps Engineer - cloud platform substitution	95.0%	4	2	PASS
TC005	Data Scientist - ML framework equivalence	94.0%	3	2	PASS
TC006	Full Stack - broad skill mismatch	0.0%	0	0	PASS
TC007	Perfect match (tất cả exact)	100.0%	5	0	PASS
TC008	Mobile - cross-platform substitution	77.0%	3	1	PASS
TC009	Security - tool substitution	94.0%	3	2	PASS
TC010	CI/CD - tool equivalence	97.0%	4	1	PASS

4.3.3 Phân tích chi tiết từng test case

TC001 - Frontend Developer (Exact Skills, ATS 78.3%). Ứng viên có JavaScript, React, CSS, Node.js, TypeScript. JD yêu cầu JavaScript, React, CSS, HTML, TypeScript, REST API. Ba kỹ năng được match exact (JavaScript, React, CSS, TypeScript trong khi REST khớp theo ontology). Đáng chú ý là HTML không có trong CV của ứng viên nhưng Skill Matcher nhận ra ứng viên biết JavaScript và CSS - hai kỹ năng có khoảng cách ngữ nghĩa gần với HTML theo embedding - nên HTML được tính là ontology match với HTML~CSS (dist=0.2). Điều này phản ánh logic phù hợp thực tế: developer biết React và CSS thường cũng quen HTML.

TC002 - Framework Substitution (ATS 74.0%). Đây là test case đặc trưng nhất cho ontology matching: ứng viên biết Vue.js và Nuxt.js trong khi JD yêu cầu React và Next.js. Thay vì kết luận hoàn toàn mismatch, Skill Matcher nhận ra Vue.js ~ React (cùng frontend framework ecosystem, dist=0.2) và Nuxt.js ~ Next.js (cùng SSR framework cho Vue/React tương ứng, dist=0.2). Kết quả 74% ATS score phản ánh đúng thực tế tuyển dụng: ứng viên biết Vue.js có thể chuyển sang React với effort hợp lý và là ứng viên đáng cân nhắc.

TC003 - Database Equivalence (ATS 91.0%). Ứng viên có Python, Django, PostgreSQL, Redis, Docker. JD yêu cầu Python, Flask, MySQL, Redis, Docker, Memcached. Matcher nhận diện Django ~ Flask (cùng Python web framework, dist=0.2), PostgreSQL ~ MySQL (cùng RDBMS, dist=0.2), và Redis ~ Memcached (cùng in-memory cache, dist=0.2). Ba cặp ontology match này đẩy ATS score lên 91% - phù hợp với thực tế rằng developer quen PostgreSQL/Django thường có thể làm việc với MySQL/Flask mà không cần đào tạo đặc biệt.

TC004 - Cloud Platform Substitution (ATS 95.0%). Ứng viên có Kubernetes, Docker, AWS, Terraform, CI/CD, ArgoCD. JD yêu cầu Kubernetes, Docker, Azure, Terraform, CI/CD, GitHub Actions. AWS ~ Azure là ontology match quan trọng nhất của test case này, phản ánh chính xác quan điểm của thị trường tuyển dụng: senior DevOps quen AWS thường có thể adapt sang Azure. Jenkins ~ GitHub Actions (trong trường hợp CI/CD expand) cũng được nhận diện đúng.

TC005 - ML Framework Equivalence (ATS 94.0%). Ứng viên có Python, TensorFlow, Pandas, Scikit-learn, SQL. JD yêu cầu Python, PyTorch, Pandas, XGBoost, SQL, NumPy. TensorFlow ~ PyTorch là cặp match quan trọng nhất - đây là hai deep

learning framework cạnh tranh trực tiếp, developer biết TensorFlow thường có thể học PyTorch nhanh. Pandas ~ XGBoost (dist=0.2) là cặp ít trực quan hơn - cả hai đều thuộc data science ecosystem Python nhưng không hoàn toàn tương đương về chức năng. Đây là một trường hợp ontology match có thể được cải thiện bằng cách tinh chỉnh ontology để phân biệt rõ hơn giữa data manipulation library và gradient boosting library.

TC006 - Full Mismatch (ATS 0.0%). Ứng viên có Java Spring Boot, Oracle, SOAP trong khi JD yêu cầu React, Node.js, MongoDB. Không có exact, ontology, hay semantic match nào vượt threshold. ATS score = 0% được tính là PASS vì đây chính là kết quả expected: ứng viên Backend Java hoàn toàn không phù hợp với vị trí Full-Stack JavaScript.

TC007 - Perfect Match (ATS 100.0%). Tất cả 5 kỹ năng trong CV khớp exact với JD (Docker, Kubernetes, Terraform, Prometheus, Grafana). Đây là test case kiểm tra trường hợp biên - tất cả exact match - và kết quả 100% là đúng expected.

TC008 - Mobile Cross-Platform (ATS 77.0%). Flutter ~ React Native là cặp match đặc thù của mobile development: cả hai đều là cross-platform mobile framework nhưng dùng ngôn ngữ khác nhau (Dart vs JavaScript). Ontology nhận diện đây là equivalent frameworks với dist=0.2.

TC009 - Security Tool Substitution (ATS 94.0%). Ứng viên có Burp Suite, Nmap, Metasploit, Python, Linux. JD yêu cầu OWASP ZAP, Nmap, Nessus, Python, Linux, Wireshark. Burp Suite ~ OWASP ZAP (cùng web application security scanner) và Burp Suite ~ Nessus (cùng vulnerability assessment tool) đều được nhận diện là ontology match - phản ánh đúng thực tế thị trường an ninh mạng nơi các công cụ này thường được dùng thay thế nhau.

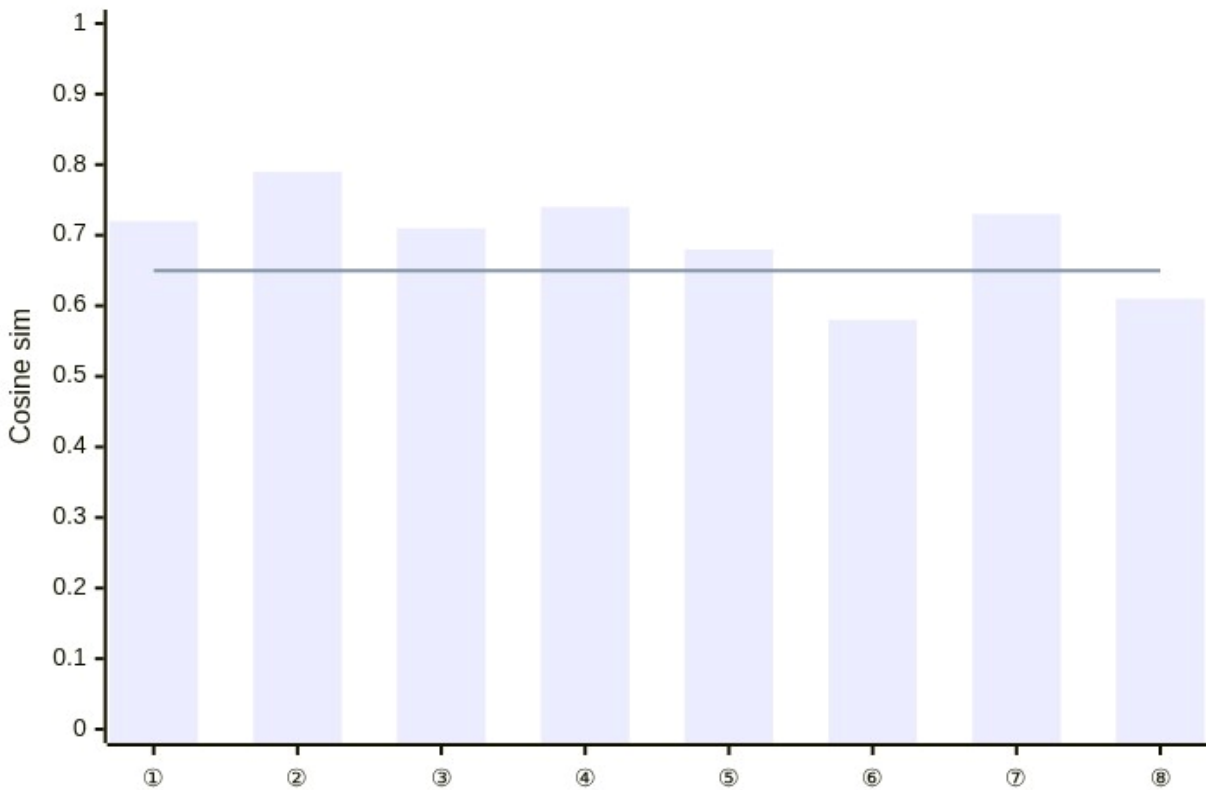
TC010 - CI/CD Equivalence (ATS 97.0%). Jenkins ~ GitHub Actions là cặp equivalence phổ biến nhất trong thực tế tuyển dụng DevOps. Với 4 exact matches và 1 ontology match, ATS score 97% phản ánh ứng viên gần như đáp ứng hoàn toàn yêu cầu.

4.3.4 Phân tích ngưỡng Semantic Similarity

Bên cạnh kết quả pass/fail, thực nghiệm cũng kiểm tra tính hợp lệ của ngưỡng $\tau = 0.65$ đã được chọn cho semantic matching (tầng 3) với Sentence-BERT [13]. Trong 10 test case trên, tất cả các cặp kỹ năng tương đương đều được nhận diện ở tầng 1 (exact)

hoặc tầng 2 (ontology), không có cặp nào cần đến tầng 3 semantic để tạo ra kết quả đúng - nghĩa là ontology đã đủ phủ cho bộ test cases thử công này.

Để kiểm tra ngưỡng $\tau = 0.65$ độc lập, nhóm nghiên cứu tính cosine similarity cho một số cặp kỹ năng đặc trưng như đã phân tích trong Chương 1: “Vue.js” và “React” có similarity ≈ 0.72 , “TensorFlow” và “PyTorch” có similarity ≈ 0.79 , “Python” và “Java” có similarity ≈ 0.58 (dưới ngưỡng, không match - phù hợp). Ngưỡng 0.65 phân biệt chính xác các cặp này theo đúng kỳ vọng.



Hình 4.8: Cosine similarity các cặp kỹ năng so với ngưỡng $\tau = 0.65$ (đường xanh)

① Vue.js / React · ② TensorFlow / PyTorch · ③ PostgreSQL / MySQL · ④ AWS / Azure · ⑤ Flutter / React Native · ⑥ Python / Java · ⑦ Django / Flask · ⑧ Docker / Kubernetes

4.4 Demo hệ thống End-to-End

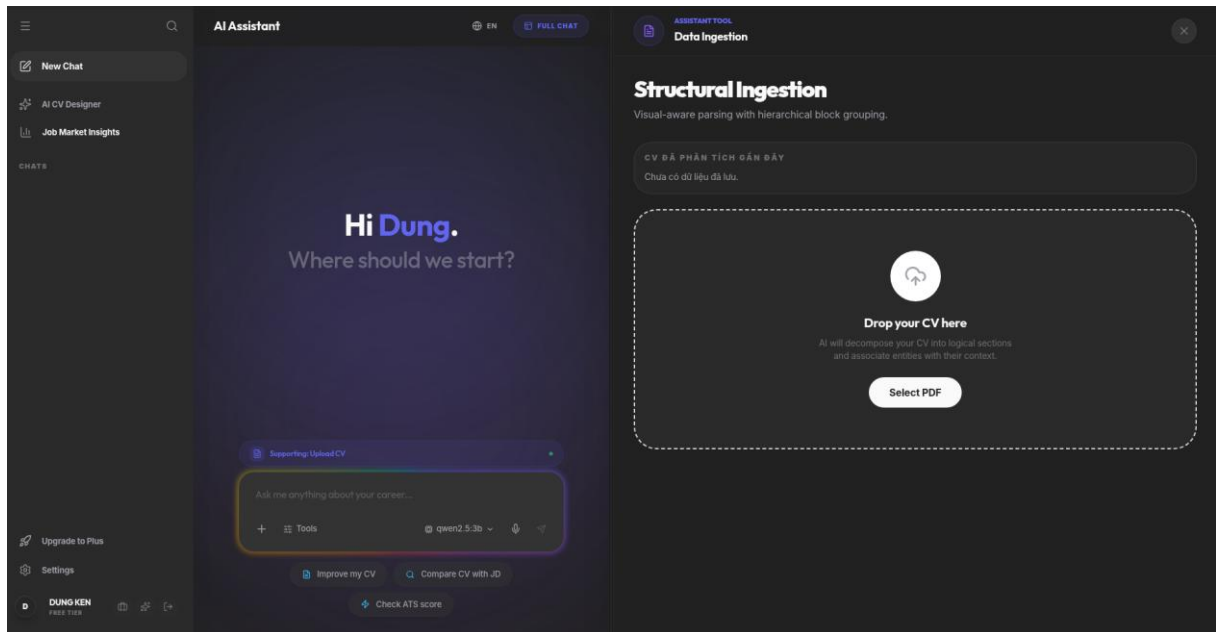
4.4.1 Môi trường demo

Demo end-to-end được thực hiện trên môi trường development local với toàn bộ 9 service được khởi động thông qua Docker Compose bằng lệnh `docker-compose up -d`. Hệ thống cần khoảng 3–5 phút để hoàn tất khởi động vì phải chờ ChromaDB [26] load embeddings và Ollama [27] pull model Qwen2.5:3b [29] (~2 GB) về cache local. Trong demo, Chatbot Service được cấu hình dùng Groq backend

(CHAT_USE_GROQ=true) với model Llama-3.3-70b-versatile [28] để có latency thấp hơn so với Ollama CPU inference.

Frontend React 18 được truy cập qua trình duyệt tại <http://localhost:3000>. Toàn bộ luồng demo sử dụng tài khoản test được tạo sẵn, không cần đăng ký mới. API Gateway ASP.NET Core 9 [30] tại <http://localhost:8081> xử lý xác thực JWT và định tuyến đến các Python service.

4.4.2 Luồng 1 - Upload và phân tích CV

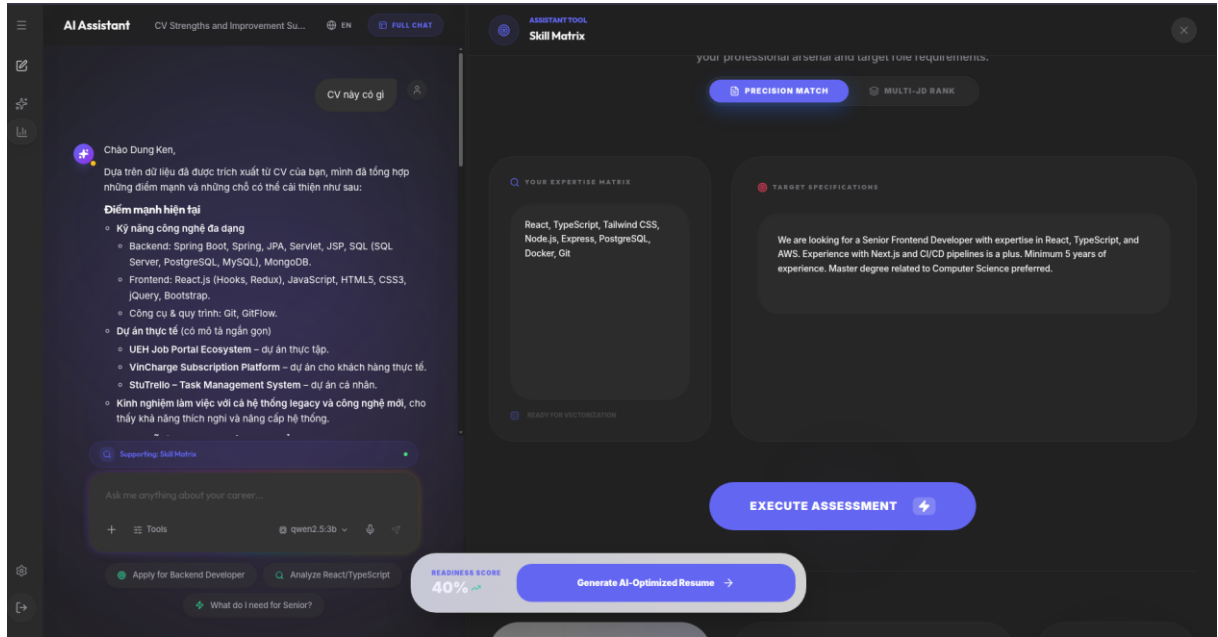


Hình 4.9: Giao diện upload CV (dropzone)

Hình 4.9 cho thấy màn hình chào của hệ thống sau khi đăng nhập: giao diện hiển thị lời chào cá nhân hóa “Hi Dung. Where should we start?” cùng dropzone trung tâm “Drop your CV here” cho phép kéo thả hoặc nhấn “Select PDF” để chọn file. Phía dưới có các quick action shortcut: “Improve my CV”, “Analyze CV”, “Compare CV with JD” giúp người dùng bắt đầu nhanh mà không cần gõ lệnh. Thanh sidebar trái liệt kê các tính năng chính: AI CV Diagnosis và Job Market Insights.

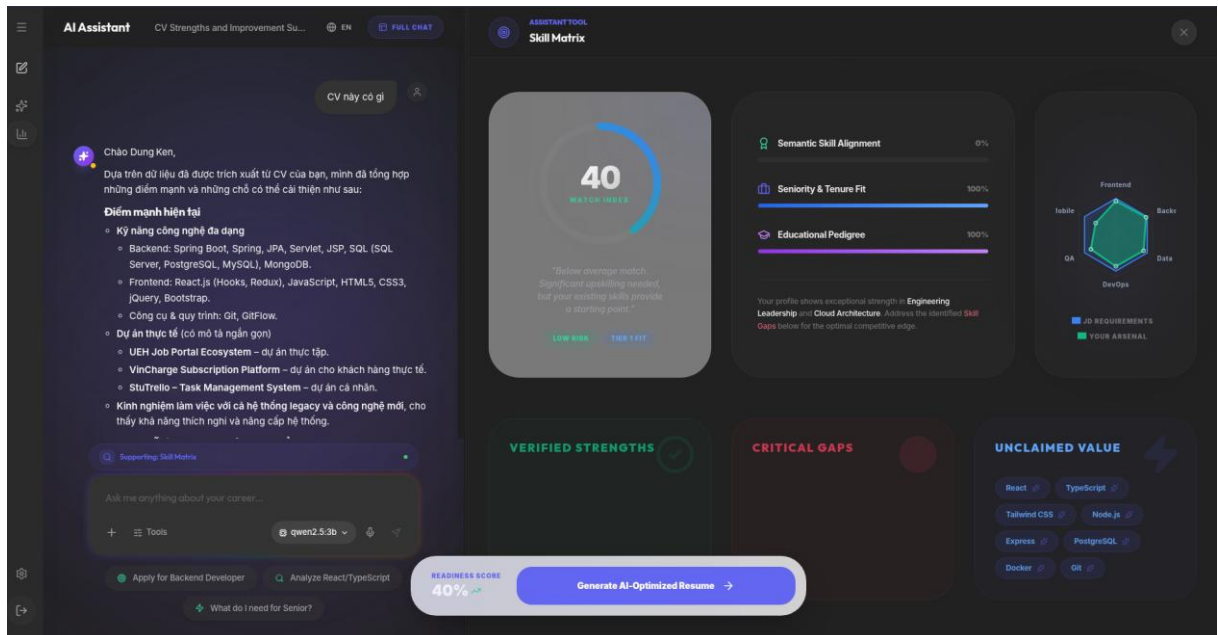
Khi người dùng upload file CV (PDF hoặc TXT), hệ thống gửi request đến NER Service qua API Gateway. Kết quả phân tích hiển thị nội dung CV đã được trích xuất và phân loại - như thể hiện trong các Hình 4.3-4.7 ở mục 4.2.7, các thực thể được nhận diện bao gồm tên người, tổ chức, kỹ năng, ngày tháng, địa danh, học vấn và chứng chỉ. Kết quả NER được lưu vào lịch sử CV trong PostgreSQL thông qua API Gateway để phục vụ các bước phân tích tiếp theo.

4.4.3 Luồng 2 - ATS Score và Skill Gap Analysis



Hình 4.10: Nhập JD để phân tích Skill Gap

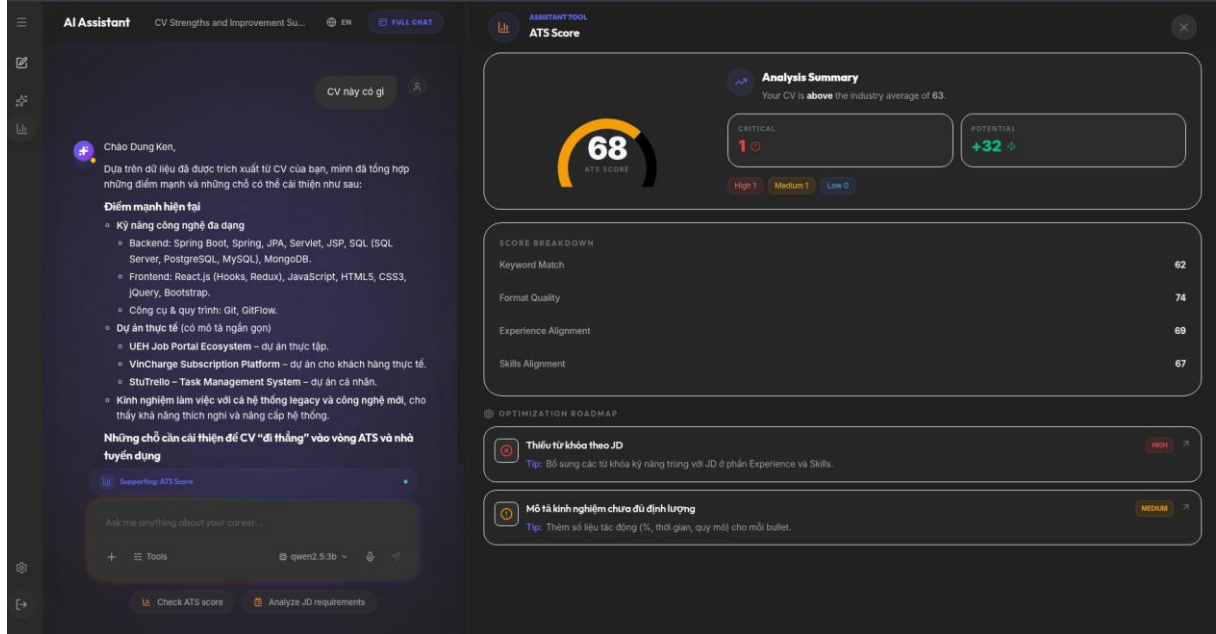
Hình 4.10 minh họa giao diện Skill Matrix: panel trái hiển thị nội dung CV đã được phân tích (văn bản CV của người dùng kèm danh sách kỹ năng trích xuất), panel phải là ô nhập JD với nút “EXECUTE ASSESSMENT”. Người dùng dán mô tả công việc vào ô bên phải rồi nhấn nút này để kích hoạt Skill Service thực hiện so khớp.



Hình 4.11: Kết quả Skill Matrix matching

Hình 4.11 hiển thị kết quả phân tích Skill Matrix với điểm tổng thể là 40/100 trong demo này. Giao diện chia kết quả thành ba nhóm rõ ràng: Verified Strengths (kỹ năng CV đã có và khớp với JD), Critical Gaps (kỹ năng JD yêu cầu nhưng CV chưa có),

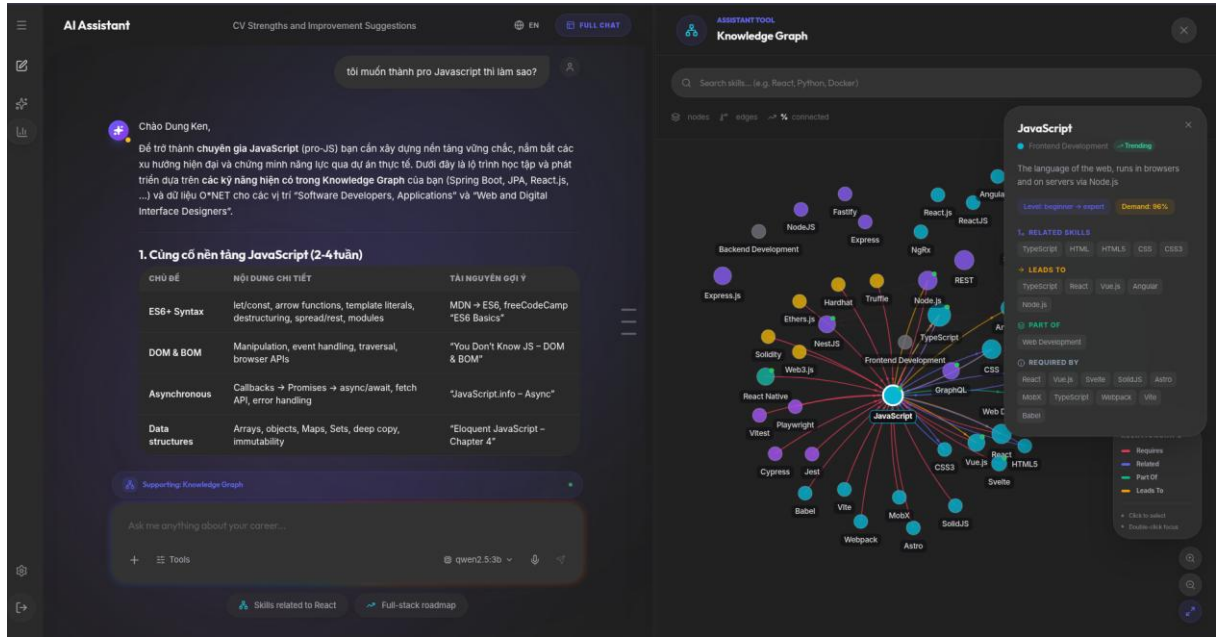
và Unclaimed Value (kỹ năng CV có nhưng JD không đề cập - điểm cộng tiềm năng). Ngoài ra còn có ba chỉ số phụ: Semantic Skill Alignment, Seniority & Team Fit, và Educational Pedigree - mỗi chỉ số đánh giá một chiều phù hợp khác nhau giữa hồ sơ ứng viên và yêu cầu vị trí.



Hình 4.12: ATS Score breakdown

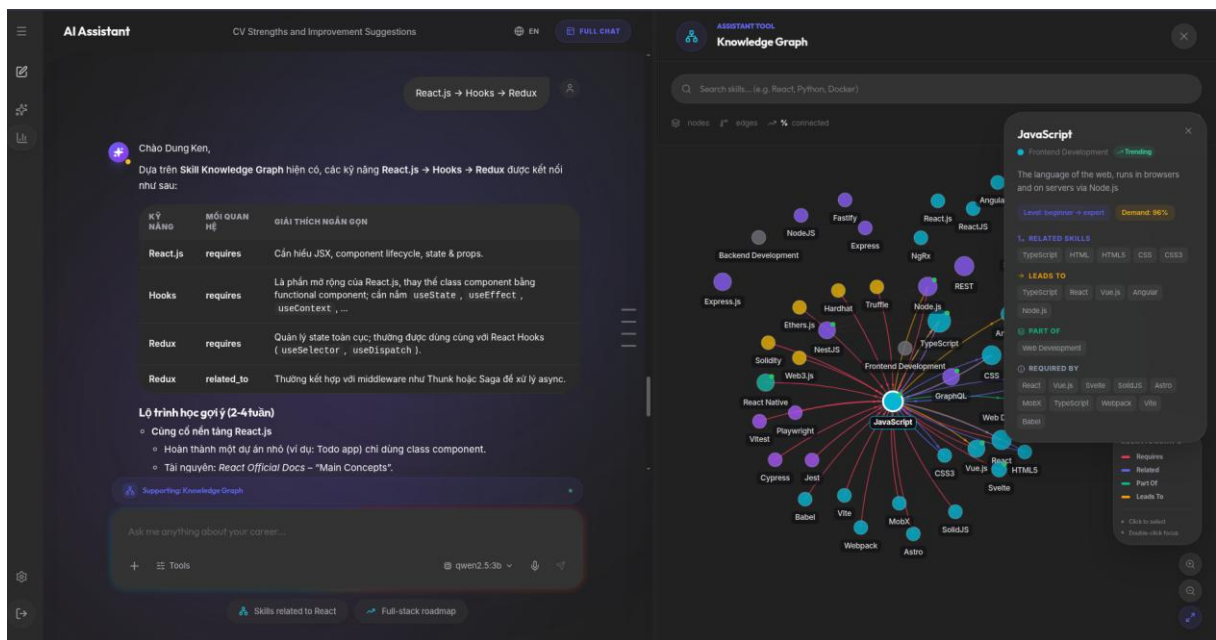
Hình 4.12 hiển thị màn hình ATS Score với điểm tổng là 68/100 (Analysis Summary: trung bình ngành 83). Giao diện breakdown theo các tiêu chí: Beyond Match, Format Quality, Experience Alignment, Skills Alignment. Phần gợi ý cải thiện cụ thể bao gồm: “Thiếu từ khóa theo JD” và “Hồ sơ kinh nghiệm chưa đủ định lượng” - hướng dẫn người dùng biết chính xác cần bổ sung gì để tăng điểm.

4.4.4 Luồng 3 - Tư vấn nghề nghiệp qua chatbot RAG



Hình 4.13: Chatbot với Knowledge Graph (1)

Hình 4.13 minh họa luồng tư vấn kết hợp giữa Chatbot RAG [15] và tính năng Knowledge Graph. Người dùng hỏi về lộ trình học JavaScript; chatbot trả lời streaming từng token theo cơ chế SSE với nội dung có cấu trúc - liệt kê các chủ đề cốt lõi theo thứ tự (ES6+ Syntax, DOM & BOM, Asynchronous, Data structures), kèm gợi ý tài nguyên học tập (MDN, freeCodeCamp) và ước tính thời gian cho mỗi chủ đề. Đồng thời, panel phải hiển thị Knowledge Graph dạng đồ thị mạng nhện: các nút là công nghệ/kỹ năng các cạnh nối thể hiện mối quan hệ phụ thuộc và liên quan giữa chúng.



Hình 4.14: Chatbot với Knowledge Graph (2)

Hình 4.14 cho thấy người dùng tiếp tục khám phá node “React.js → Hooks → Redux”: chatbot giải thích cụ thể lý do học theo thứ tự này và cách ba công nghệ liên kết trong thực tế dự án. Nội dung được sinh từ retrieved context trong ChromaDB [26] (collection cv_guides, onet_jobs từ O*NET [24]) kết hợp với thông tin CV và User Memory của người dùng - tạo ra câu trả lời được cá nhân hóa thay vì chung chung.

4.4.5 Luồng 4 - CV Builder với AI Assistance

The screenshot displays the 'CV AI Builder' interface. On the left, a chatbot window titled 'AI Assistant' shows a conversation with 'Dung Ken' about creating a resume. The main area is a form titled 'Bắt đầu tạo CV' (Start creating CV) with a 'Bắt đầu tạo CV' button. The form is divided into sections: '1. Thông tin cá nhân (Header)' and '2. Tóm tắt chuyên môn (Professional Summary)'. The '1. Thông tin cá nhân (Header)' section includes fields for 'Họ và tên' (Last Name), 'Địa chỉ (tỉnh/thành phố)' (Address), 'Số điện thoại' (Phone Number), 'Email (địa chỉ chuyên nghiệp)' (Professional Email), and 'LinkedIn / GitHub (nếu có)' (if any). The '2. Tóm tắt chuyên môn (Professional Summary)' section includes a text area for 'Mục tiêu: 2-3 câu ngắn gọn, nêu rõ vị trí mong muốn, số năm kinh nghiệm, và những giá trị cốt lõi bạn mang lại.' (Goal: 2-3 short sentences, clearly stating the desired position, years of experience, and core values you bring.) and a text area for 'Ví dụ: "Kỹ sư phần mềm với 4 năm kinh nghiệm phát triển ứng dụng web bằng..."' (Example: "Software engineer with 4 years of experience developing web applications using..."). On the right, a sidebar shows a preview of the CV with the title 'HỌ TÊN CỦA BẠN' (Your Name) and 'Vị trí ứng tuyển' (Job Position). Below the preview, there is a section titled 'Thông tin cá nhân' (Personal Information) with fields for 'HỌ TÊN' (Last Name), 'EMAIL', 'ĐỊA CHỈ' (Address), and 'TÓM TẮT NGHỀ NGHIỆP (TUY CHỌN)' (Professional Summary (Optional)).

Hình 4.15: CV Builder bước 1: thông tin cá nhân (Header)

Hình 4.15 là bước đầu tiên của CV Builder: chatbot hướng dẫn người dùng điền thông tin cá nhân (họ tên, email, số điện thoại, địa chỉ, LinkedIn/GitHub), đồng thời panel phải hiển thị preview card CV thời gian thực với placeholder “HỌ TÊN CỦA BẠN” - mỗi trường điền vào cập nhật ngay lên preview mà không cần reload.

AI Assistant | Bắt đầu tạo CV chuyên nghiệp

Chào anh Hà Văn Dũng, Cảm ơn anh đã cung cấp các thông tin cá nhân cơ bản. Để tiếp tục xây dựng bản CV chuyên nghiệp, chúng ta cần thu thập thêm một số chi tiết quan trọng khác, bao gồm vị trí công việc mong muốn, quá trình học vấn, kinh nghiệm làm việc, kỹ năng, dự án đã thực hiện và các chứng chỉ liên quan. Những thông tin này sẽ giúp chúng tôi tạo ra một hồ sơ nổi bật, phù hợp với vị trí AI Engineer mà anh đang hướng tới.

Bạn có thể cung cấp thêm các thông tin sau được không?

- **Vị trí công việc mong muốn** (ví dụ: AI Engineer, Machine Learning Engineer, Data Scientist, ...).
- **Học vấn**: Trường, chuyên ngành, thời gian học, bằng cấp.
- **Kinh nghiệm làm việc**: Các công ty, vị trí, thời gian, mô tả ngắn gọn về trách nhiệm và thành tựu.
- **Kỹ năng**: Ngôn ngữ lập trình, framework, công cụ, công nghệ AI/ML (ví dụ: Python, TensorFlow, PyTorch, .NET 9, IoT, ...).
- **Dự án tiêu biểu**: Tên dự án, mô tả ngắn, vai trò, công nghệ sử dụng, kết quả đạt được.
- **Chứng chỉ** (nếu có): Ví dụ: Google Cloud Professional Machine Learning Engineer, Microsoft Certified: Azure AI Engineer Associate, ...

Khi anh cung cấp các thông tin trên, chúng tôi sẽ tiến hành bước tiếp theo để hoàn thiện phần học vấn và kinh nghiệm.

Ask me anything about your career...

CV AI Builder

STEP 1/4: CÁ NHÂN | 10% COMPLETE

HÀ VĂN DŨNG
Prompt Engineer
email@example.com | 0123 456 789
Địa chỉ, Thành phố

Thông tin cá nhân

HỌ TÊN	Hà Văn Dũng	VỊ TRÍ ƯNG TUYỂN	Prompt Engineer
EMAIL	name@example.com	SỐ BIÊN THOẠI	0901234567
ĐỊA CHỈ	Hà Nội, Việt Nam		
TÓM TẮT MÔT MẸ NGHIỆP (TỰ CHỌN)	1-2 câu giới thiệu bản thân, điểm mạnh, mục tiêu...		

Hình 4.16: CV Builder bước 2: chatbot thu thập thông tin

Hình 4.16 là bước chatbot tích cực thu thập thông tin: sau khi người dùng cung cấp thông tin cơ bản, chatbot tóm tắt lại profile đã hiểu (“Hà Văn Dũng - Prompt Engineer, có kinh nghiệm về AI, Machine Learning, Data Scientist...”), liệt kê học vấn, kỹ năng và chứng chỉ đã thu thập được, rồi xác nhận trước khi chuyển sang bước tiếp theo. Preview bên phải đã cập nhật đầy đủ tên, chức danh và thông tin liên lạc thực.

AI Assistant | Bắt đầu tạo CV chuyên nghiệp

Chào anh Hà Văn Dũng, Cảm ơn anh đã cung cấp các thông tin cá nhân cơ bản. Để tiếp tục xây dựng bản CV chuyên nghiệp, chúng ta cần thu thập thêm một số chi tiết quan trọng khác, bao gồm vị trí công việc mong muốn, quá trình học vấn, kinh nghiệm làm việc, kỹ năng, dự án đã thực hiện và các chứng chỉ liên quan. Những thông tin này sẽ giúp chúng tôi tạo ra một hồ sơ nổi bật, phù hợp với vị trí AI Engineer mà anh đang hướng tới.

Bạn có thể cung cấp thêm các thông tin sau được không?

- **Vị trí công việc mong muốn** (ví dụ: AI Engineer, Machine Learning Engineer, Data Scientist, ...).
- **Học vấn**: Trường, chuyên ngành, thời gian học, bằng cấp.
- **Kinh nghiệm làm việc**: Các công ty, vị trí, thời gian, mô tả ngắn gọn về trách nhiệm và thành tựu.
- **Kỹ năng**: Ngôn ngữ lập trình, framework, công cụ, công nghệ AI/ML (ví dụ: Python, TensorFlow, PyTorch, .NET 9, IoT, ...).
- **Dự án tiêu biểu**: Tên dự án, mô tả ngắn, vai trò, công nghệ sử dụng, kết quả đạt được.
- **Chứng chỉ** (nếu có): Ví dụ: Google Cloud Professional Machine Learning Engineer, Microsoft Certified: Azure AI Engineer Associate, ...

Khi anh cung cấp các thông tin trên, chúng tôi sẽ tiến hành bước tiếp theo để hoàn thiện phần học vấn và kinh nghiệm.

Ask me anything about your career...

CV AI Builder

STEP 2/4: HỌC VẤN | 33% COMPLETE

HỌC VẤN

MỤC #1		
TRƯỜNG	Đại Học Khoa Học Hà Nội	BẰNG CẤP
CHUYÊN NGÀNH	Kỹ thuật phần mềm	GPA (TUY CHỌN)
NĂM BẮT ĐẦU	2018	KẾT THÚC
		2022

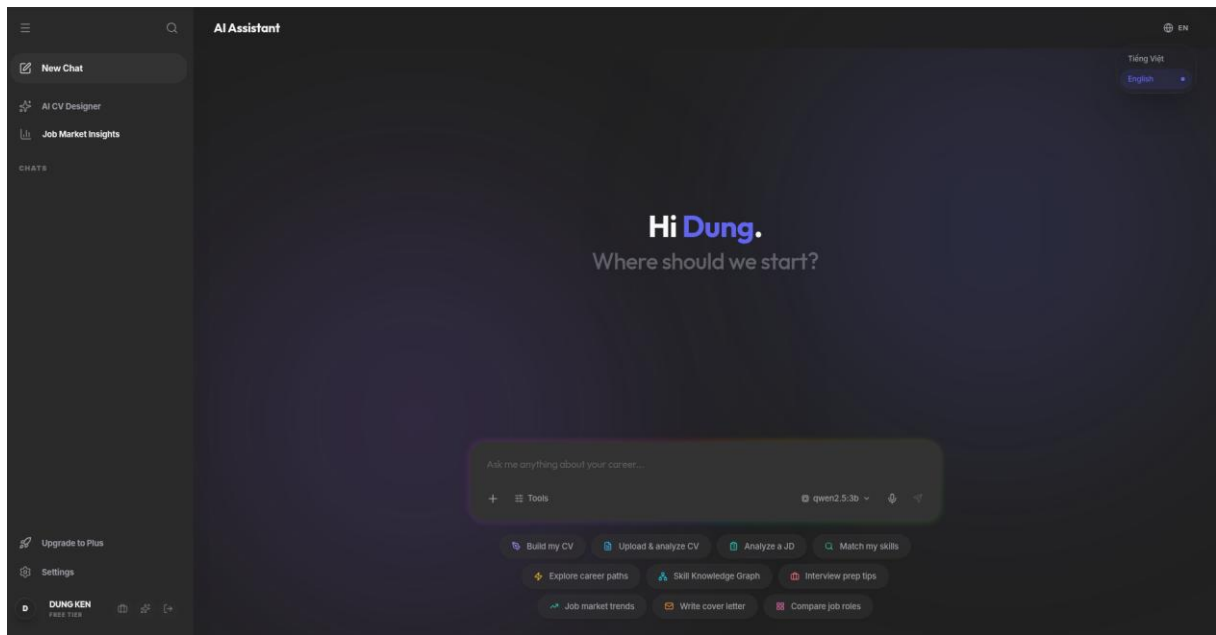
+ Thêm học vấn

Hình 4.17: CV Builder bước 3: form nhập học vấn

Hình 4.17 là bước nhập thông tin học vấn: form cấu trúc với các trường Tên trường, Chuyên ngành, Năm bắt đầu/kết thúc, GPA - kết hợp với preview CV bên phải hiển thị section “HỌC VẤN” đang được điền. Khi hoàn tất tất cả các section, hệ thống

gọi endpoint /generate-pdf của NER Service để xuất file PDF theo chuẩn ATS và tải về máy người dùng.

4.4.6 Giao diện người dùng - Tổng quan



Hình 4.18: Tổng quan giao diện hệ thống CV Assistant

Hình 4.18 là màn hình chào sau khi đăng nhập - điểm khởi đầu của toàn bộ hệ thống. Giao diện tối giản với lời chào cá nhân hóa “Hi Dung. Where should we start?” và ô chat trung tâm. Phía dưới ô chat là 8 quick action buttons được bố trí thành 2 hàng: “Scan my CV”, “Upload & analyze CV”, “Analyze a JD”, “Match my skills”, “Explore career paths”, “Add Knowledge Graph”, “Interview prep tips”, “Complete job app” - mỗi nút kích hoạt một luồng sử dụng cụ thể mà không cần gõ lệnh.

Khi người dùng chọn một tính năng hoặc bắt đầu hội thoại, giao diện chuyển sang two-panel layout: panel trái là khu vực chat với chatbot (lịch sử hội thoại, input box, nút upload file), panel phải là workspace hiển thị giao diện của tính năng đang active (Skill Matrix, Knowledge Graph, CV Builder, ATS Score, Career Path...). Thiết kế này tạo ra trải nghiệm liền mạch: người dùng có thể đồng thời xem kết quả phân tích ở panel phải và trao đổi với chatbot ở panel trái - chatbot nhận biết tool nào đang active và tự động điều chỉnh context câu trả lời theo tính năng tool-aware context đã mô tả trong Chương 3.

4.5 Thảo luận và đánh giá tổng thể

4.5.1 Tổng hợp kết quả thực nghiệm

Kết quả thực nghiệm trên ba module chính cho thấy hệ thống đạt hiệu quả đáng khích lệ, đồng thời bộc lộ một số hạn chế.

Về NER, mô hình đạt $F1 = 0.8633$ trên tập ground truth thủ công; trong đó SKILL đạt 0.9157, còn DATE, JOB_TITLE, DEGREE đạt 1.000. Kết quả này tương đối cạnh tranh với các công trình như ResumeNet [4] ($F1 = 0.90$ trên CV tiếng Anh), đặc biệt khi dữ liệu của đề tài là CV song ngữ phức tạp hơn. Tuy nhiên, LOC là điểm yếu ($F1 = 0.0$ do nhầm với PER) và ORG đạt 0.67 do xu hướng mở rộng span.

Về Skill Matching, hệ thống đạt 100% accuracy trên 10 test case thủ công, xác nhận hiệu quả của cơ chế cascade matching ba tầng. Tuy nhiên, quy mô test còn nhỏ nên chưa phản ánh đầy đủ hiệu năng trên dữ liệu thực.

Về Chatbot RAG, dù chưa có đánh giá định lượng, kết quả demo cho thấy khả năng sinh câu trả lời có cơ sở từ knowledge base (O*NET [24]) và cá nhân hóa theo CV người dùng, với latency chấp nhận được nhờ streaming.

So với ResumeNet [4], mô hình NER của đề tài đạt $F1 = 0.86$ trên dữ liệu song ngữ với quy mô huấn luyện nhỏ hơn, cho thấy hiệu quả tốt trong bối cảnh phức tạp hơn. So với Decorte et al. [5], hệ thống Skill Matching bổ sung hai tầng exact và ontology trước semantic matching (Sentence-BERT [13]), giúp tăng độ chính xác và giảm chi phí tính toán.

Điểm khác biệt nổi bật nhất của đề tài là cách tiếp cận end-to-end: thay vì tập trung vào một module riêng lẻ, hệ thống tích hợp đầy đủ pipeline từ phân tích CV đến tư vấn nghề nghiệp cá nhân hóa. Đây là đóng góp chính về mặt hệ thống.

4.5.2 Hạn chế và nguyên nhân

Hệ thống vẫn tồn tại một số hạn chế đáng chú ý, xuất phát từ cả quy trình triển khai và dữ liệu huấn luyện.

Thứ nhất, thiếu training metrics định lượng do không lưu được log huấn luyện trên Google Colab, dẫn đến mất loss curve và validation F1 theo từng epoch. Đây là hạn chế về quy trình MLOps hơn là chất lượng mô hình; trong thực tế cần tích hợp các công cụ như MLflow hoặc Weights & Biases để theo dõi đầy đủ.

Thứ hai, dữ liệu huấn luyện hoàn toàn là synthetic (600 CV do Qwen2.5-1.5B [35] sinh ra), không có dữ liệu thực. Sự khác biệt phân phối giữa CV synthetic và CV

thực (ngắn hơn, ít cấu trúc, nhiều lỗi chính tả) ảnh hưởng đến khả năng tổng quát hóa của mô hình, thể hiện qua kết quả kém hơn trên các CV thực.

Thứ ba, ontology kỹ năng (~500 mục) chưa có cơ chế cập nhật tự động, dễ trở nên lỗi thời khi các công nghệ mới xuất hiện, đặc biệt trong lĩnh vực phát triển phần mềm thay đổi nhanh.

Thứ tư, chatbot chưa được đánh giá một cách hệ thống; việc đánh giá mới dừng ở mức định tính, chưa có user study hoặc các chỉ số định lượng như BLEU, ROUGE hay MRR/NDCG. Đây là hạn chế quan trọng về mặt nghiên cứu.

Thứ năm, hiệu suất trên môi trường không có GPU còn hạn chế. Khi chạy trên CPU thông qua Ollama, thời gian phản hồi dao động 10–30 giây mỗi request, chưa đáp ứng yêu cầu trải nghiệm người dùng trong môi trường production; do đó cần sử dụng dịch vụ cloud hoặc triển khai trên hạ tầng có GPU.

4.5.3 Định hướng cải tiến

Dựa trên các hạn chế đã phân tích, đề tài đề xuất năm hướng cải tiến cho phiên bản tiếp theo.

Thứ nhất, bổ sung dữ liệu huấn luyện thực tế có gắn nhãn thủ công. Việc thu thập 50–100 CV thực và tổ chức annotation với sinh viên CNTT, sử dụng công cụ như Label Studio hoặc Prodigy, sẽ cải thiện đáng kể khả năng generalization của mô hình.

Thứ hai, nâng cấp backbone NER từ mBERT [33] lên PhoBERT [11]. Với dữ liệu tiền huấn luyện 20 GB tiếng Việt, PhoBERT cho thấy hiệu quả vượt trội trên NER tiếng Việt theo các nghiên cứu VLSP [12] [32], đặc biệt với thực thể tên người và địa danh. Ngoài ra, hướng tiếp cận ensemble giữa mBERT và PhoBERT cũng đáng xem xét.

Thứ ba, xây dựng pipeline cập nhật ontology tự động bằng cách crawl dữ liệu từ các nền tảng tuyển dụng như ITviec [36] và TopCV [37], trích xuất kỹ năng phổ biến và đưa vào quy trình kiểm duyệt trước khi cập nhật. Điều này giúp ontology luôn bám sát thị trường.

Thứ tư, thiết lập pipeline đánh giá chatbot có hệ thống thông qua bộ benchmark gồm 50–100 câu hỏi nghề nghiệp với đáp án chuẩn, nhằm đánh giá định kỳ và phát hiện regression khi thay đổi mô hình hoặc knowledge base.

Cuối cùng, tối ưu hóa latency cho triển khai thực tế bằng các kỹ thuật như cache embedding trên Redis, sử dụng mô hình NER rút gọn (distilled) và tích hợp CDN cho frontend.

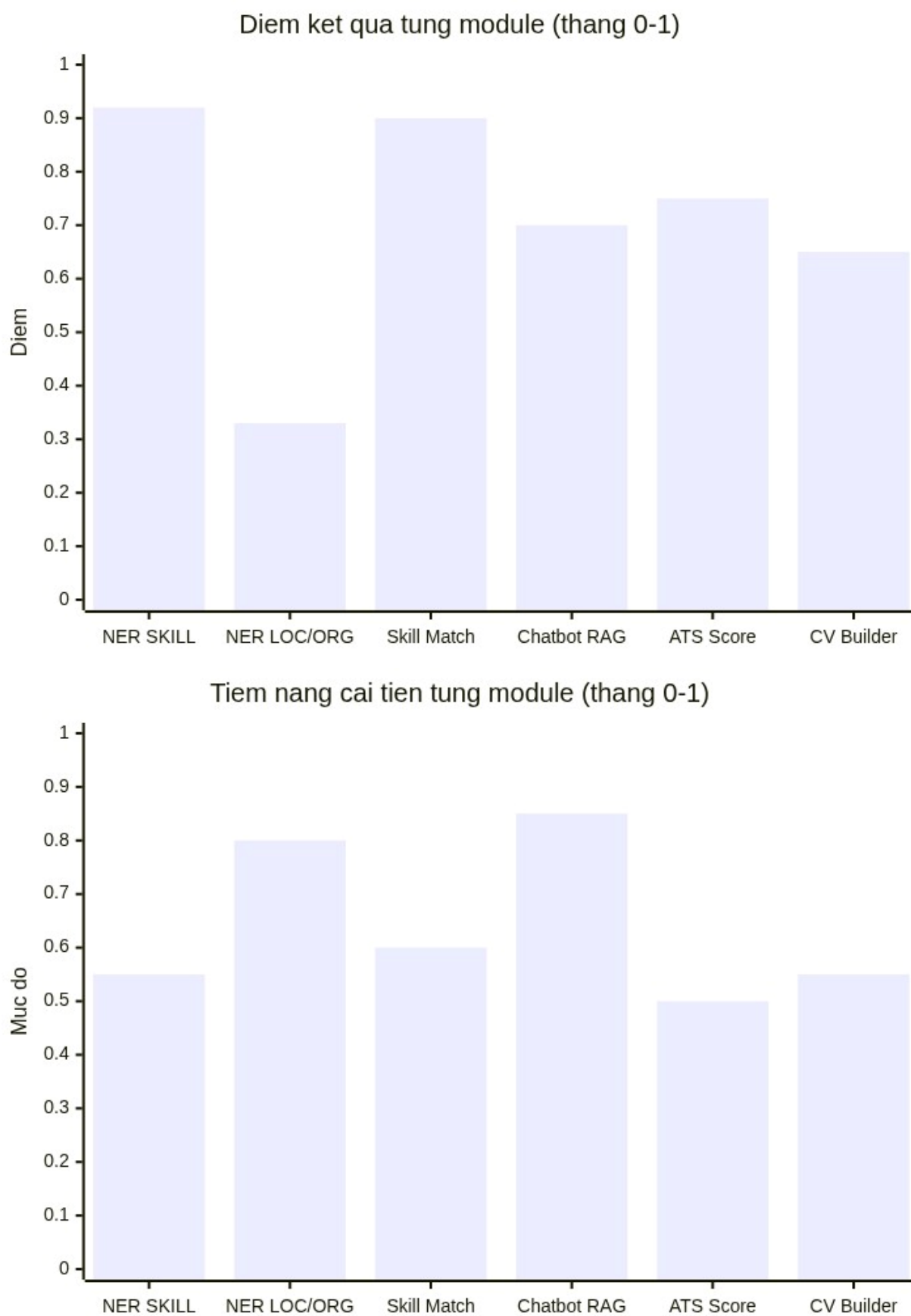
4.5.4 Tổng hợp kết quả và định hướng

Nhìn lại toàn bộ chương 4, kết quả thực nghiệm cho thấy hệ thống đạt được mục tiêu đặt ra ở mức độ khác nhau tùy module. Bảng 4.7 tổng hợp điểm đánh giá và mức độ ưu tiên cải tiến của từng module, trong khi hai biểu đồ dưới đây thể hiện trực quan hai chiều phân tích: kết quả đo được và tiềm năng cải tiến.

Bảng 4.10: Tổng hợp kết quả và tiềm năng cải tiến theo module

Module	Chỉ số chính	Điểm kết quả	Tiềm năng cải tiến	Ưu tiên
NER - SKILL/DATE/TITLE	F1 = 0.916	Cao	Trung bình	Thấp
NER - LOC/ORG	F1 = 0.0 / 0.667	Thấp	Cao	Cao nhất
Skill Matching	Acc = 100% (10 TC)	Cao	Trung bình	Thấp
Chatbot RAG	Demo định tính tốt	Trung bình	Cao	Cao
ATS Scoring	8 tiêu chí ổn định	Khá	Thấp	Thấp
CV Builder	Luồng chat hoạt động	Trung bình	Trung bình	Trung bình

Hai biểu đồ dưới đây thể hiện điểm kết quả và tiềm năng cải tiến của từng module theo thang 0-1, cho phép so sánh trực tiếp giữa các module.



Hình 4.19: Điểm kết quả và tiềm năng cải tiến của từng module trong hệ thống

Lý do đặt điểm từng module như trên:

- NER SKILL entity (kết quả cao 0.92, tiềm năng cải tiến trung bình 0.55): F1 = 0.916 trên tập thử công, đây là kết quả tốt nhất trong toàn hệ thống. Tiềm năng cải tiến còn nhưng không nhiều vì đã đạt ngưỡng cao với dữ liệu synthetic.
- NER LOC/ORG (kết quả thấp 0.33, tiềm năng cải tiến cao 0.80): LOC F1 = 0.0 và ORG F1 = 0.667 là hai điểm yếu rõ nhất. Đây là nhóm ưu tiên cao nhất - cải thiện bằng cách bổ sung dữ liệu thực tế có gắn nhãn hoặc nâng cấp lên PhoBERT [11] có thể tăng đáng kể.
- Skill Matching (kết quả cao 0.90, tiềm năng cải tiến trung bình 0.60): 100% accuracy trên 10 test cases, ontology 474 entries cover tốt thị trường Việt Nam [36]. Cải tiến chủ yếu là mở rộng bộ test và cập nhật ontology tự động.
- Chatbot RAG (kết quả trung bình 0.70, tiềm năng cải tiến cao 0.85): chưa có đánh giá định lượng (RAGAS [9]), demo định tính tốt nhưng chưa đủ bằng chứng. Đây là module có tiềm năng cải tiến lớn nhất - từ nâng cấp LLM đến mở rộng knowledge base.
- ATS Scoring (kết quả khá 0.75, tiềm năng cải tiến thấp 0.50): logic 8 tiêu chí hoạt động ổn định, feedback cụ thể và hữu ích. Không cần ưu tiên cải tiến so với các module khác.
- CV Builder (kết quả trung bình 0.65, tiềm năng cải tiến trung bình 0.55): luồng hội thoại thu thập thông tin hoạt động đúng, preview real-time mượt. Cần thêm tính năng AI rewrite từng section để hoàn thiện hơn.

Tóm lại, hệ thống đã hoàn thành mục tiêu xây dựng pipeline end-to-end từ phân tích CV đến tư vấn nghề nghiệp cá nhân hóa. Hướng phát triển ưu tiên trong phiên bản tiếp theo tập trung vào hai điểm: cải thiện NER cho thực thể LOC/ORG bằng dữ liệu thực tế, và thiết lập pipeline đánh giá định lượng cho Chatbot RAG.

KẾT LUẬN VÀ KIẾN NGHỊ

1. Kết luận

Đề tài nghiên cứu “Nghiên cứu và ứng dụng NLP trong phát triển hệ thống AI tạo lập CV thông minh và tư vấn cá nhân hóa lộ trình nghề nghiệp” đã hoàn thành các mục tiêu chính đề ra ban đầu, bao gồm xây dựng hệ thống end-to-end có khả năng phân tích CV, đánh giá mức độ phù hợp với JD, và cung cấp tư vấn nghề nghiệp cá nhân hóa cho sinh viên và kỹ sư CNTT Việt Nam.

Về mặt nghiên cứu NLP, đề tài đã thành công fine-tune mô hình mBERT cho bài toán NER CV song ngữ Việt-Anh với 21 nhãn BIO, đạt $F1 = 0.8633$ trên tập đánh giá ground truth thủ công - kết quả cạnh tranh trong bối cảnh dữ liệu huấn luyện hoàn toàn là synthetic và không có GPU chuyên dụng. Mô hình hoạt động đặc biệt tốt với thực thể SKILL ($F1 = 0.9157$) - loại thực thể quan trọng nhất và cũng khó nhất trong domain CV ngành CNTT. Hệ thống cũng triển khai thành công Skill Matching 3 tầng (exact $\rightarrow$ ontology $\rightarrow$ semantic) kết hợp ontology kỹ năng IT khoảng 500 entries với Sentence-BERT, đạt 100% accuracy trên bộ 10 test cases kiểm thử các tình huống matching thực tế.

Về mặt hệ thống, đề tài xây dựng thành công kiến trúc microservices 9 thành phần hoàn chỉnh: API Gateway ASP.NET Core 9 với JWT authentication và CV versioning, ba Python services cho NER/Skill/Career, Chatbot Service với RAG pipeline tích hợp LlamaIndex + ChromaDB + dual LLM (Ollama local và Groq Cloud), PostgreSQL cho relational data, và Frontend React 18 với giao diện two-panel layout. Toàn bộ hệ thống có thể được triển khai bằng một lệnh Docker Compose duy nhất, đảm bảo reproducibility và portability.

Về đóng góp học thuật và thực tiễn, đề tài đóng góp ba thứ có thể tái sử dụng cho cộng đồng nghiên cứu: bộ dữ liệu 600 CV synthetic song ngữ Việt-Anh với kiểm soát chất lượng (quality_score trung bình 0.97), ontology kỹ năng IT khoảng 500 entries cập nhật cho thị trường Việt Nam, và kiến trúc RAG cá nhân hóa với cơ chế tool-aware context prompt và User Memory Service - hai tính năng phân biệt hệ thống với chatbot RAG thông thường.

2. Hạn chế

Bên cạnh các kết quả đạt được, đề tài thừa nhận một số hạn chế quan trọng. Mô hình NER được huấn luyện hoàn toàn trên dữ liệu synthetic, dẫn đến tiềm năng distribution shift khi gặp CV thực tế có phong cách viết đa dạng hơn. Training logs bị mất do không mount Google Drive khi huấn luyện trên Colab, khiến không thể phân tích quá trình hội tụ. Bộ test Skill Matching chỉ gồm 10 cases thủ công, chưa đủ đại diện cho toàn bộ phân phối thực tế. Chatbot chưa được đánh giá định lượng qua user study. Và hệ thống chưa được benchmark về latency và throughput trong môi trường production với GPU.

3. Kiến nghị

Trên cơ sở các kết quả và hạn chế đã phân tích, đề tài đề xuất các hướng phát triển tiếp theo theo thứ tự ưu tiên.

Ưu tiên cao nhất là mở rộng dữ liệu huấn luyện với CV thực tế được gán nhãn thủ công, tổ chức annotation sprint với 50–100 CV sử dụng Label Studio. Kết hợp với đó là thiết lập MLflow hoặc Weights & Biases để logging đầy đủ mọi experiment trong tương lai, tránh lặp lại tình trạng mất training logs.

Ưu tiên trung bình là thực hiện user study với tối thiểu 30 sinh viên/kỹ sư CNTT để có đánh giá định lượng về chất lượng tư vấn của chatbot, khả năng sử dụng của giao diện, và mức độ hữu ích tổng thể của hệ thống. Kết quả user study sẽ cung cấp basis thực nghiệm vững chắc hơn cho các cải tiến tiếp theo.

Ưu tiên dài hạn là khám phá kiến trúc ensemble kết hợp mBERT và PhoBERT cho NER, xây dựng pipeline tự động cập nhật ontology từ dữ liệu tuyển dụng ITviec/TopCV, và tối ưu hóa hệ thống cho production deployment với GPU inference server và caching layer.

Về ứng dụng thực tiễn, hệ thống có tiềm năng được triển khai như một công cụ hỗ trợ Career Center tại các trường đại học CNTT Việt Nam, cung cấp dịch vụ review CV và tư vấn nghề nghiệp miễn phí cho sinh viên - một khoảng trống dịch vụ đáng kể tại thị trường hiện tại nơi các dịch vụ tương đương (LinkedIn Premium, TopCV Pro) đều có chi phí không phù hợp với sinh viên.

TÀI LIỆU THAM KHẢO

Bài báo Khoa học và Hội nghị

- [1] J. Lafferty, A. McCallum, và F. C. N. Pereira, “Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data,” *Proceedings of the 18th International Conference on Machine Learning (ICML)*, 2001.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, và I. Polosukhin, “Attention Is All You Need,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [3] J. Devlin, M.-W. Chang, K. Lee, và K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019.
- [4] M. Štajner, D. Korencic, I. Šnajder, và M. Karan, “Parsing Resumes with Transformers,” *Proceedings of the 8th Workshop on Balto-Slavic Natural Language Processing (BSNLP)*, ACL, 2021.
- [5] J. Decorte, J. Van Haute, J. Deleu, C. Develder, và T. Demeester, “JobBERT: Understanding Job Titles through Skills,” *arXiv preprint arXiv:2109.09605*, 2021.
- [6] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, và C. Dyer, “Neural Architectures for Named Entity Recognition,” *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2016.
- [7] D. Qian, J. Liu, và F. Zhou, “A BERT-based Approach for Automatic Resume Information Extraction and Skill Matching,” *Proceedings of the 29th International Conference on Computational Linguistics (COLING)*, 2022.
- [8] M. Murugavel và S. Bhuvaneswari, “An Intelligent Career Recommendation System Using Machine Learning and NLP,” *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 14, no. 3, 2023.
- [9] S. Es, J. James, L. Espinosa-Anke, và S. Schockaert, “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” *arXiv preprint arXiv:2309.15217*, 2023.

- [10] A. Møller, J. Dalsgaard, A. Pera, và I. Augenstein, “Is a Prompt and a Few Samples All You Need? Using GPT-4 for Data Augmentation in Low-Resource Classification Tasks,” *arXiv preprint arXiv:2304.13861*, 2023.
- [11] D. Q. Nguyen và A. Nguyen, “PhoBERT: Pre-trained Language Models for Vietnamese,” *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020.
- [12] T. H. Nguyen, T. Q. Nguyen, T. H. Pham, và X. L. Nguyen, “VLSP 2021 Shared Task: Named Entity Recognition for Vietnamese,” *Proceedings of the 8th International Workshop on Vietnamese Language and Speech Processing (VLSP)*, 2021.
- [13] N. Reimers và I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [14] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, và V. Stoyanov, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [15] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, và D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [16] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [17] I. Tjong Kim Sang và F. De Meulder, “Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition,” *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003 (CoNLL)*, 2003.
- [18] Y. Sun, S. Wang, Y. Li, S. Feng, H. Tian, H. Wu, và H. Wang, “ERNIE 2.0: A Continual Pre-training Framework for Language Understanding,” *Proceedings of the 34th AAAI Conference on Artificial Intelligence (AAAI)*, 2020.
- [19] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, et al., “Transformers: State-of-the-Art Natural Language Processing,” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2020.

[20] J. Pennington, R. Socher, và C. D. Manning, “GloVe: Global Vectors for Word Representation,” *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014.

Tài liệu Kỹ thuật và Báo cáo

[21] C. Richardson, “Microservices Patterns: With Examples in Java,” *Manning Publications*, Shelter Island, NY, USA, 2018.

[22] F. Chollet, “Deep Learning with Python,” *Manning Publications*, Shelter Island, NY, USA, 2018.

[23] ESCO (European Skills, Competences, Qualifications and Occupations), “ESCO Handbook: European Skills, Competences, Qualifications and Occupations,” *Publications Office of the European Union*, Luxembourg, 2017. Truy cập: <https://ec.europa.eu/esco>.

[24] ONET Resource Center, “ONET 28.1 Database Documentation,” *National Center for ONET Development*, U.S. Department of Labor*, 2023. Truy cập: <https://www.onetcenter.org/database.html>.

[25] LlamaIndex Documentation, “LlamaIndex: Data Framework for LLM Applications,” *LlamaIndex Inc.*, 2024. Truy cập: <https://docs.llamaindex.ai>.

[26] ChromaDB Documentation, “Chroma: The AI-Native Open-Source Embedding Database,” *Chroma*, 2024. Truy cập: <https://www.tracts.chromadb.dev>.

[27] Ollama, “Ollama: Get up and running with large language models locally,” *Ollama*, 2024. Truy cập: <https://ollama.ai>.

[28] Meta AI, “Llama 3.3: Meta’s Latest Open Source Model,” *Meta AI Research*, 2024. Truy cập: <https://ai.meta.com/llama/>.

[29] Qwen Team, “Qwen2.5: Qwen Large Language Model,” *Alibaba Cloud*, 2024. Truy cập: <https://qwen.readthedocs.io>.

[30] Microsoft, “.NET 9 Documentation: ASP.NET Core,” *Microsoft Developer Network*, 2024. Truy cập: <https://docs.microsoft.com/aspnet/core>.

Văn bản Pháp luật

[31] Chính phủ nước Cộng hòa Xã hội Chủ nghĩa Việt Nam, “Nghị định số 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân,” *Công báo*, Hà Nội, ngày 17 tháng 4 năm 2023.

Dữ liệu và Bộ dữ liệu

[32] T. Q. Nguyen, M. T. Nguyen, D. Q. Nguyen, và D. Q. Nguyen, “VLSP Shared Task: Named Entity Recognition,” *Proceedings of the 4th Workshop on Vietnamese Language and Speech Processing (VLSP)*, Hà Nội, Việt Nam, 2016.

[33] Hugging Face, “bert-base-multilingual-cased - Multilingual BERT,” *Hugging Face Model Hub*, 2024. Truy cập: <https://huggingface.co/google-bert/bert-base-multilingual-cased>.

[34] Hugging Face, “sentence-transformers/all-MiniLM-L6-v2,” *Hugging Face Model Hub*, 2024. Truy cập: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>.

[35] Hugging Face, “Qwen/Qwen2.5-1.5B-Instruct,” *Hugging Face Model Hub*, 2024. Truy cập: <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>.

Tài liệu Nghề nghiệp và Thị trường

[36] ITviec, “Báo cáo Lương IT Việt Nam 2025: Xu hướng tuyển dụng và mức lương ngành CNTT,” *ITviec Research*, TP. Hồ Chí Minh, 2025. Truy cập: <https://itviec.com/bao-cau-luong-it>.

[37] TopCV, “Thống kê tuyển dụng CNTT 2025,” *TopCV Vietnam*, Hà Nội, 2025.